



Microprocessor & Computer Architecture (μ pCA)

Session - 1.1



Microprocessor & Computer Architecture (μ pCA)

Introduction

Microprocessor & Computer Architecture (μ pCA)

Overview



Microprocessor & Computer Architecture (μpCA)

What we Know? [studied in 3 sem]

Digital Design!

- In DDCO course, *increase logic circuit speed, decrease logic resources required and decrease power consumed*
- *Goal is to Optimize fundamental physical quantities of time, space and energy.*

Computer Organization !

- Hardware Abstraction above Digital Logic and below Operating System.
- Hardware details which are required by programmers, such as Control signals, Interfaces and Memory

Microprocessor & Computer Architecture (μpCA)

What we study - Syllabus



Unit 1: Architecture

Introduction, ISA Classification - RISC and CISC, Memory Addressing, Operands - Types and Size, Instruction Set - Operations, Control Flow, Instruction Encoding, Case Study - ARM/ MIPS/ x86 Processor. **12 Hours**

Unit 2: Pipeline

3- Stage Pipelining, 5 - Stage Pipelining, Pipeline Datapath and Control, Data Hazards – Forwarding vs. Stalling, Control Hazards, Branch Prediction Mechanisms and Exceptions, Performance Metrics. **12 Hours**

Unit 3: Memory Hierarchy

Basics of Caches - Fully Associative, Direct Mapped and Set Associative, Cache Performance, Basic Cache Optimizations. **12 Hours**

Unit 4: Flash Storage, Connecting and Interfacing devices and Bus Architecture

Introduction to flash storage, Connecting Processors, Memory and I/O devices, Interfacing I/O Devices to the Processor, Memory and Operating System. **10 Hours**

Unit 5: Advances in Architecture

Parallel Computing : Introductory concepts and terminology-Flynn's taxonomy, parallel computing memory architectures, parallel programming models, parallel examples: matrix multiplication, Amdahl's Law, Gustafson Law, Hardware Multi threading, Multi-Core Architecture **10 Hours**

Microprocessor & Computer Architecture (μpCA)

Evaluation



As per University Policy

ISA - 50% - 4 ISAs – Unit1, Unit2, Unit3, Unit4&5

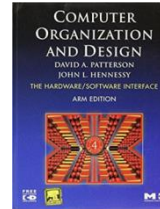
ESA - 50% - 100 marks reduced to 50 marks

Microprocessor & Computer Architecture (μpCA)

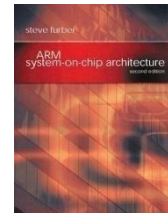
Text and Reference Book(s)



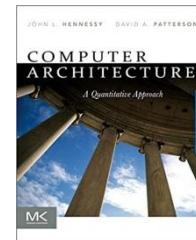
T1: “Computer Organization and Design”, Patterson, Hennessey, 5th Edition, Morgan Kaufmann, 2014.



T2: “ARM System-on-Chip Architecture”, Steve Furber, 2nd Edition, 2015, Pearson India.



R1: “Computer Architecture: A Quantitative Approach”, Hennessey, Patterson, 5th Edition, Morgan Kaufmann, 2011.



R2: “The Definitive Guide to the ARM Cortex-M0 and Cortex M0+ processors”, Joseph Yiu, 2nd Edition, Newnes, 2015.

Job Profiles....

Microprocessor & Computer Architecture (μpCA)

Job Description 1

Job Description

Software Developer for Microprocessor based embedded system



- Perform Board Bring-up on ARM/Intel SoC architecture.
- Reverse engineer HW-SW issues by referring schematics & data sheet.
- Realize digital design on Intel/Xilinx FPGA.
- Optimize device drivers (Ethernet, PCIe, DMA, CAN) taking into consideration security aspects.
- Migrate software components (Device Drivers & Middleware) from Linux to QNX.
- Design systems using SoC, PCIe Switch, FPGA & SoC's.
- Design software architecture using state of the art tools (like Rapsody).
- Port open source software & benchmark performance on development kits.
- Positively influence Bosch position by contributing to open source software.
- Create quick demonstrators/PoC & represent Bosch offerings in trade shows

Qualifications

- Qualification: B.E./B.Tech./ M.E./ M.Tech. – Electronics & Communication or Computer Science
- Work Experience: 1 to 4 years

Technical Know-how:

- Background experience in using Digital Storage Oscilloscope.
- Awareness of Intel and ARM SoC architecture from secure boot point of view.
- Awareness of ARM Instruction Set Architecture & Vectorization techniques.
- Practical experience in real-time Linux/ RTOS based embedded software development using C & C++.
- Awareness of latest technical happenings in open source software (OSADL, Linux mainline, Autoware, Apollo, Adaptive AUTOSAR).
- Practical experience in debugging software for embedded systems.
- Practical overview (via. academic literatures) of technology trends in SoC, Ethernet Switch, PCIe & FPGAs.

Microprocessor & Computer Architecture (μpCA)

Job Description 2

NVIDIA is searching for a highly motivated, creative engineer with experience in system software to join the GPU Software team. As someone who is hardworking and passionate about their work, you will design key aspects of our production GPU kernel drivers and embedded Software. You should demonstrate the ability to excel in an environment with complex software and hardware designs.



What you'll be doing:

- Define, design, develop and verify features for our GPUs; collaborating with hardware engineers and fellow software engineers
- You will follow the devices all the way through the development process to the customer desktops, notebooks, workstations, and gaming console products that are used throughout the world
- Heavily involved with the early modeling and simulation required to produce our world-class products
- Multiple opportunities to collaborate and communicate effectively with teams from all around the globe

What we need to see:

- BS or MS degree in Computer Engineering, Computer Science, or related degree
- Strong C programming skills as well as having shown initiative in pursuing independent coding projects
- Familiarity with computer system architecture, microprocessor, and microcontroller fundamentals (caches, buses, memory controllers, DMA, etc.)
- Kernel experience with Linux, Android, Chrome, or Windows systems
- 2+ years of meaningful software development experience

Ways to stand out from the crowd:

- Background and strength with complex system-level debugging is invaluable
- Deep understanding of memory management and virtualization concepts
- Familiarity with kernel level security concepts
- Experience with embedded system SW concepts, e.g.: RTOS and overlay programming models

Microprocessor & Computer Architecture (μpCA)

Job Description 3



Capgemini
Firmware Developer

Role Description (Role & Responsibilities)

1. **Mandatory skills 16bit 32bit Microcontroller Microprocessor**
2. Embedded systems Firmware Device driver development experience Programming Strong in C
3. Communication protocols UART CAN SPI Ethernet Modbus TCP IP IDE usage
4. Code composer studio IAR workbench Code warrior RTOS VxWorks FreeRTOS Ti RTOS Bootloader
5. Multi threading concepts Preferred skills Strong in Cplusplus Assembly language experience

Mobiveil: CPU Processor Design

Job Summary:

Bachelors or Masters degree in **Computer Science** or Electrical/Computer Engineering.

Understanding of general purpose CPU micro architecture, including knowledge of areas such as processor pipelines, caches, memory hierarchy, and multi-processor systems.

Knowledge and or Experience in RTL Design hardware development using Verilog, ideally block design in a CPU design project or similar high performance project.

Understanding of CPU instruction set architecture and assembly language.

Familiarity with ARM architecture and micro-architecture for current ARM CPU cores is helpful but not required.

Software development skills and/ or experience is helpful (C/C++, Python/Perl, Shell scripting)

Experience modelling microprocessors using higher-level languages, like C/C++, is helpful but not required

Effective communication skills and the ability to collaborate with a team

Microprocessor & Computer Architecture (μpCA)

Job Description 4



Open-Silicon: Software Engineer

Job Description:

- Solid programming experience in C or C++.
 - Adequate knowledge of Object oriented programming and design concepts.
 - **Should have good knowledge of System software, Microprocessors/Microcontrollers, Memory subsystem, Hardware IPs .**
 - Proven experience in embedded systems design with low level driver programming , bootloader, preemptive, multitasking real-time operating systems
 - Should have exposure to scripting language e.g. Perl/Shell/Python etc.
 - Experience in hands-on development and troubleshooting on embedded boards.
 - Familiarity with software configuration management tools, defect tracking tools, and peer review
- Good team player, should be able to handle task independently





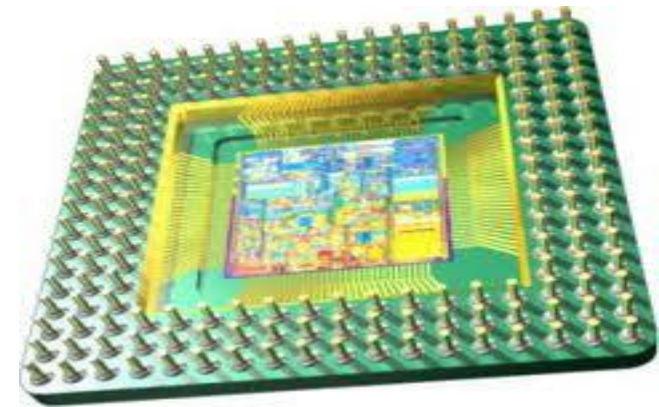
Microprocessor & Computer Architecture (μ pCA)

Introduction to Microprocessor

Session - 1.11



Is **MICROPROCESSOR**
Brain or Heart
of a computer System
???



Statutory Warning: does not resemble anyone over here

Microprocessor & Computer Architecture (μpCA)

What we study?

What is a microProcessor?

- Brain of the Computer
- Basic Computing Unit on an Integrated Chip (IC)
- Three basic characteristics differentiate microprocessors:
Instruction set , Bandwidth, Clock speed

What is Computer Architecture ?

- The engineering decisions and tradeoffs that must be made in order to produce a **“Good” Design**.

i.e Execution Time, Space and Resource



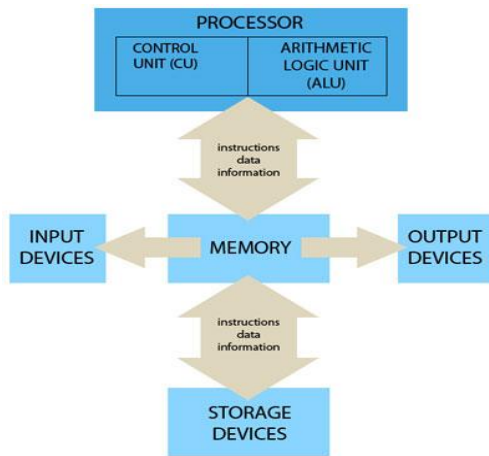
Why Study Microprocessor?

- Everywhere we have devices which are controlled by “Microprocessor” or “Microcontroller”
- Because its part of the Curriculum!!! 😊

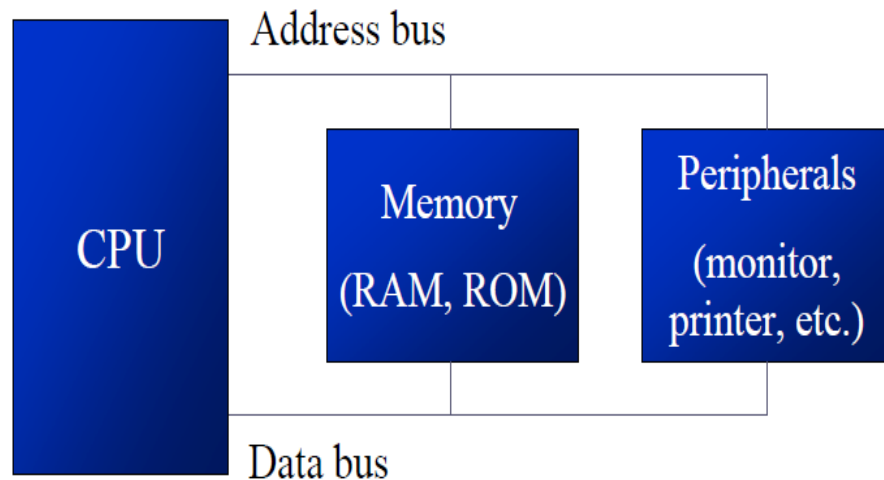


Microprocessor & Computer Architecture

Computer Organization & Computer Architecture

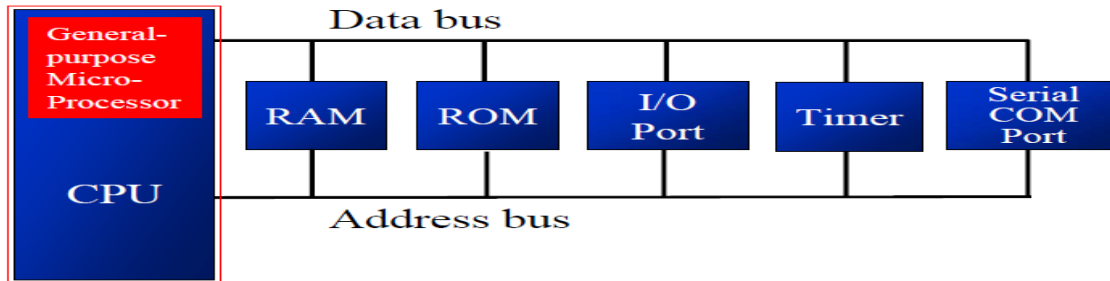


- Computer Organization refers to the **Operational Units and there interconnections** that realize or recognize the specifications of Computer Architecture.
- Organizational **attributes** includes **Hardware details transparent** to the programmer such as **control signals** , **Interface** between computers and peripherals and the memory technology used.



- **Computer Architecture** refers to those attributes of a system visible to a programmer, the other way those attributes that have direct impact on the logical execution of the program.
- Architectural attributes include the **instruction set**, **the no of bits used to represent the data types**, **Input Output mechanism and technique** for addressing memories.

Microprocessor



- Accepts [digital data](#) as input, processes it according to instructions stored in its memory, and provides results as output.
- Don't have RAM, ROM, and other peripheral on the chip.
- Memory , I/O and other components need to be connected externally.
- Cannot be used in compact systems.
- Power Consumption is High.

Microcontroller

Microcontroller



- Comprises of CPU, in addition with a fixed amount of RAM, ROM and other peripherals all embedded on a single chip.
- Also termed as a mini computer or a computer on a single chip.
- Designed to perform **specific tasks**.
- Requires **small resources** like RAM, ROM, I/O ports etc. and hence can be embedded on a single chip.
- Reduces the size and the cost.

Microprocessor & Computer Architecture (μ pCA)

Microprocessor, Where am I?



Microprocessor & Computer Architecture (μ pCA)

Microprocessor, Where am I?



Microprocessor & Computer Architecture (μpCA)

Microprocessor - Evolution

4004	Nov. 15,1971
8008	April 1972
8080	April 1974
8085	March 1976
8086	June 8, 1978
8088	June 1979
80286	Feb. 1982
i80386	1985 - 1990
i80486	1989 - 1992
Intel Pentium	1993 - 1999
Intel Pentium MMX	1996 - 1999
Intel Atom	2008 - 2009 (as Centrino Atom), 2008–present (as Atom)
Intel Celeron	1998–present
Intel Pentium Pro	1995 - 1998
Intel Pentium II	1997 - 1999
Intel Pentium III	1999 - 2003
Intel Xeon	1998–present
Pentium 4	2000 - 2008
Pentium 4	2000 - 2008
Pentium M	2003 - 2008
Pentium D/EE	2005 - 2008

Microprocessor & Computer Architecture (μpCA)

Microprocessor - Evolution

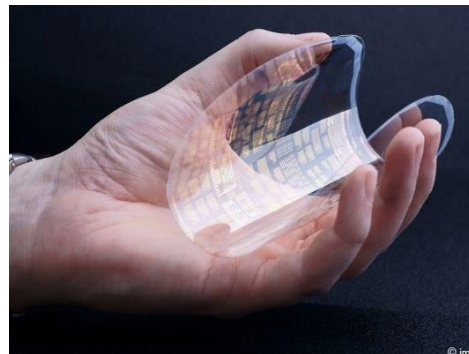


Intel Pentium Dual-Core	2006 - 2009
Intel Pentium (2009)	2009–present
Intel Core	2006 - 2008
Intel Core 2	2006 - 2011
Intel Core i3	2010–present
Intel Core i5	2009–present
Intel Core i7	2008–present
Intel Core i7 (Extreme Edition)	2011–present
Intel Core i9	2018–present
Intel Core i9 (Extreme Edition)	Q3 2017–present

Microprocessor & Computer Architecture (μ pCA)

Why Study Microprocessor ?

- Because its part of the Curriculum!!! 😊
- Everywhere we have devices which are controlled by “Microprocessor” or “Microcontroller”



Microprocessor & Computer Architecture (μpCA)

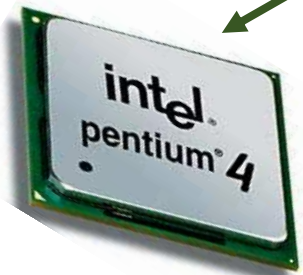
Classification of Microprocessors

CISC: Complex Instruction Set Computer

RISC: Reduced Instruction Set Computer



CISC



RISC



**Special
Purpose**



How to Classify Microprocessors?

Three basic characteristics differentiate microprocessors:

- Instruction set**: The set of instructions that the microprocessor can execute.
- Bandwidth** : The number of bits processed in a single instruction.
- Clock Speed** : How many instructions per second the processor can execute.

Given in megahertz (MHz),

Microprocessor & Computer Architecture (μpCA)

Instruction Set Architecture (ISA)

- The complete collection of instructions that are understood by a CPU

Example: Data Movement, Data Processing, Branch Instruction.....etc

- Designers decide the ISA for respective Microprocessor.
- It is responsibility of Compiler to generate correct and Optimized code for respective Microprocessor.
- Main criteria to classify CISC & RISC Processor

Consider the program fragments:

$a = b \times c ;$

$b = 10;$ and

$c = 5$

To Execute the Program

Load **b** from Memory

Load **c** from Memory

mul **b** and **c**

Store the result **a** back in
Memory

Microprocessor & Computer Architecture (μpCA)

CISC: Complex Instruction Set Computer



- CISC processors were evolved in the 1970s.
- During this period, the computer memory used to be ‘small’ and ‘very expensive’.
- Philosophy of CISC processors is to simplify the code and make it **shorter** in order to reduce the memory requirement.
- In a CISC processor, a single instruction has ‘several low-level operations. This makes the CISC instructions short but ‘complex’.

Microprocessor & Computer Architecture (μpCA)

CISC: Complex Instruction Set Computer

- Consider the program fragments: $a = b \times c$; $b = 10$ and $c = 5$

CISC

Program 1
`mul bx`

Here, One operand is implicit.

Mul instruction will multiply 5 and 10 and store it in the register.

Program 2
`mov ax, 10`
`mov bx, 5`
`mul bx, ax`

Mul instruction will fetch 5 and 10 from memory, multiply 5 and 10 and store it in the register.

Microprocessor & Computer Architecture (μpCA)

RISC: Reduced Instruction Set Computer



- Idea of RISC processors was originated in the 1974 and Implemented in 1980's.
- Philosophy of RISC processors is to simplify operation of Individual Instruction.
- Separate Instruction for Load and Store (Data Movement)
- ***Number of lines of code*** in the program ***increases*** but Amount of work done by individual instruction is ***REDUCED***.

Microprocessor & Computer Architecture (μpCA)

RISC: Reduced Instruction Set Computer

- Consider the program fragments: $a = b \times c$; $b = 10$ and $c = 5$

RISC

```
Begin  mov ax, 0
       mov bx, 10
       mov cx, 5
       add ax, bx
       dec cx
       bne Begin
```

Microprocessor & Computer Architecture (μpCA)

RISC vs CISC

- Consider the program fragments:

CISC

```
mov ax, 10  
mov bx, 5  
mul bx
```

RISC

Begin

```
mov ax, 0  
mov bx, 10  
mov cx, 5  
add ax, bx  
dec cx  
bne Begin
```

The total clock cycles for the CISC version might be:

$(2 \text{ movs} \times 1 \text{ cycle}) + (1 \text{ mul} \times 30 \text{ cycles}) = 32 \text{ cycles}$

While the clock cycles for the RISC version is:

$(3 \text{ movs} \times 1 \text{ cycle}) + (5 \text{ adds} \times 1 \text{ cycle}) + (5 \text{ dec} \times 1 \text{ cycle}) + (5 \text{ loops} \times 1 \text{ cycle}) = 18 \text{ cycles}$

With RISC clock cycle being shorter, RISC gives us much faster execution speeds.

Microprocessor & Computer Architecture (μpCA)

RISC vs CISC



RISC

- Simple instructions, few in number.
- Fixed length instructions.
- Multiple register sets.
- Three operands per instruction.
- Parameter passing through register windows.
- Single-cycle instructions.
- Hardwired control.
- Highly pipelined.
- Complexity in compiler.
- Only **LOAD/STORE** instructions access memory.
- Few addressing modes.

CISC

- Many complex instructions.
- Variable length instructions.
- Single register set.
- One or two register operands per instruction.
- Parameter passing through memory.
- Multiple cycle instructions.
- Microprogrammed control.
- Less pipelined.
- Many instructions can access memory.
- Many addressing modes.

Microprocessor & Computer Architecture (μpCA)

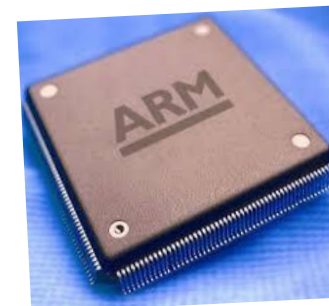
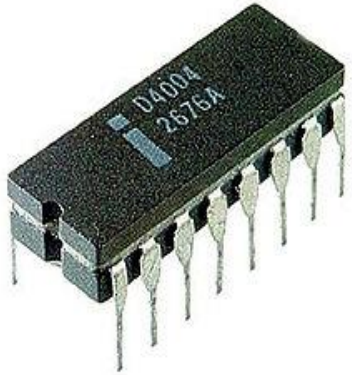
RISC Vs CISC Machines Today

- With the rise of embedded systems and mobile computing, the terms RISC and CISC have lost their significance
- Both architectures almost seem to have **adopted** the strategies of the other.
- Some RISC systems provide more extravagant instruction sets than some CISC systems.
- CISC chips are now able to execute more than one instruction within a single clock, including Pipelining.
- The RISC Vs CISC debate started when chip-area and processor design complexity were issues – now energy and power are the issues.
- The two top competitors today are ARM and Intel – Intel focuses on performance and ARM focuses on efficiency (*British company* – **Acron** **RISC Machine**)



Microprocessor & Computer Architecture (μ pCA)

Our Choice-> ARM



Mostly on-----?



THANK YOU



Team MPCA

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE19CS252

Dr. D. C. Kiran

Department of
Computer Science and Engineering

Microprocessor & Computer Architecture (μ pCA)

Register Bank

Dr. D. C. Kiran

Department of Computer Science and Engineering

Microprocessor & Computer Architecture (μpCA)

Syllabus



Unit 1: Basic Processor Architecture and Design

- ~~Microprocessor Overview~~
- ~~CISC VS RISC~~
- ~~Introduction to ARM Processor & Applications~~
- ~~ARM Architecture Overview~~
- ~~Different ARM processor Modes~~
- Register Bank

Unit 2: Pipelined Processor and Design

Unit 3: Memory Design

Unit 4: Input/Output Device Design

Unit 5: Advanced Architecture

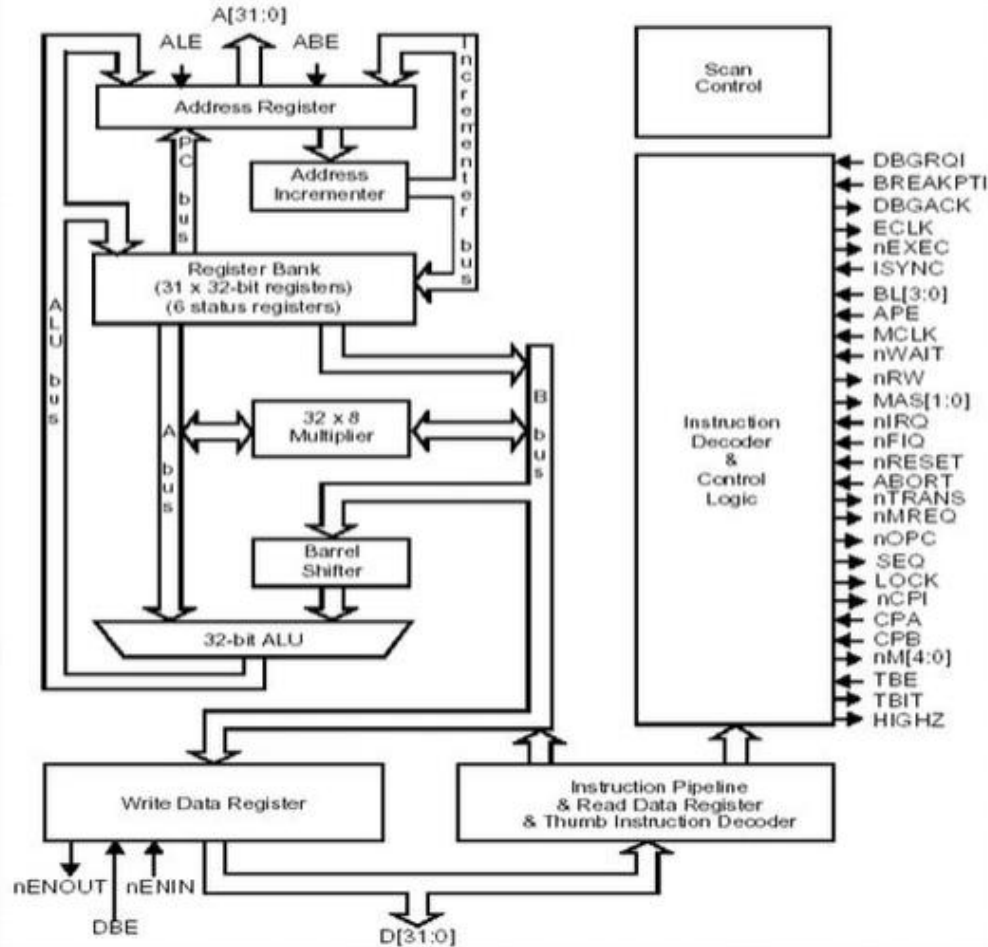
Microprocessor & Computer Architecture (μpCA)

ARM: [ACORN RISC MACHINE]

ARM7TDMI Processor



- Von Neumann Architecture
- 3-stage pipeline
 - fetch, decode, execute
- 32-bit Data Bus
- 32-bit Address Bus
- 37 32-bit registers
- 32-bit ARM instruction set
- 16-bit THUMB instruction set
- 32x8 Multiplier
- Barrel Shifter



- ARM has 37 registers all of which are 32-bits long.
 - 1 dedicated Program Counter (PC)
 - 1 dedicated Current Program Status Register (CPSR)
 - 5 dedicated Saved program Status Registers (SPSR)
 - 30 general purpose registers
- The current processor mode governs which of several banks is accessible. Each mode can access
 - a particular set of r0-r12 registers
 - a particular r13 (the stack pointer, SP) and r14 (the link register, LR)
 - the program counter, r15 (PC)
 - the current program status register, CPSR

Reference:

<https://www.sciencedirect.com/topics/computer-science/banked-register>

Microprocessor & Computer Architecture (μpCA)

The ARM has Seven basic operating modes:

User & System

Common R0-R12, Dedicated R13 (SP), Dedicated R14(LR), Common R15 (PC) & CPSR

IRQ / Supervisor / Abort / Undef

Common R0-R12, Dedicated R13 (SP), Dedicated R14(LR), Common R15 (PC) & CPSR and
Dedicated SPSR

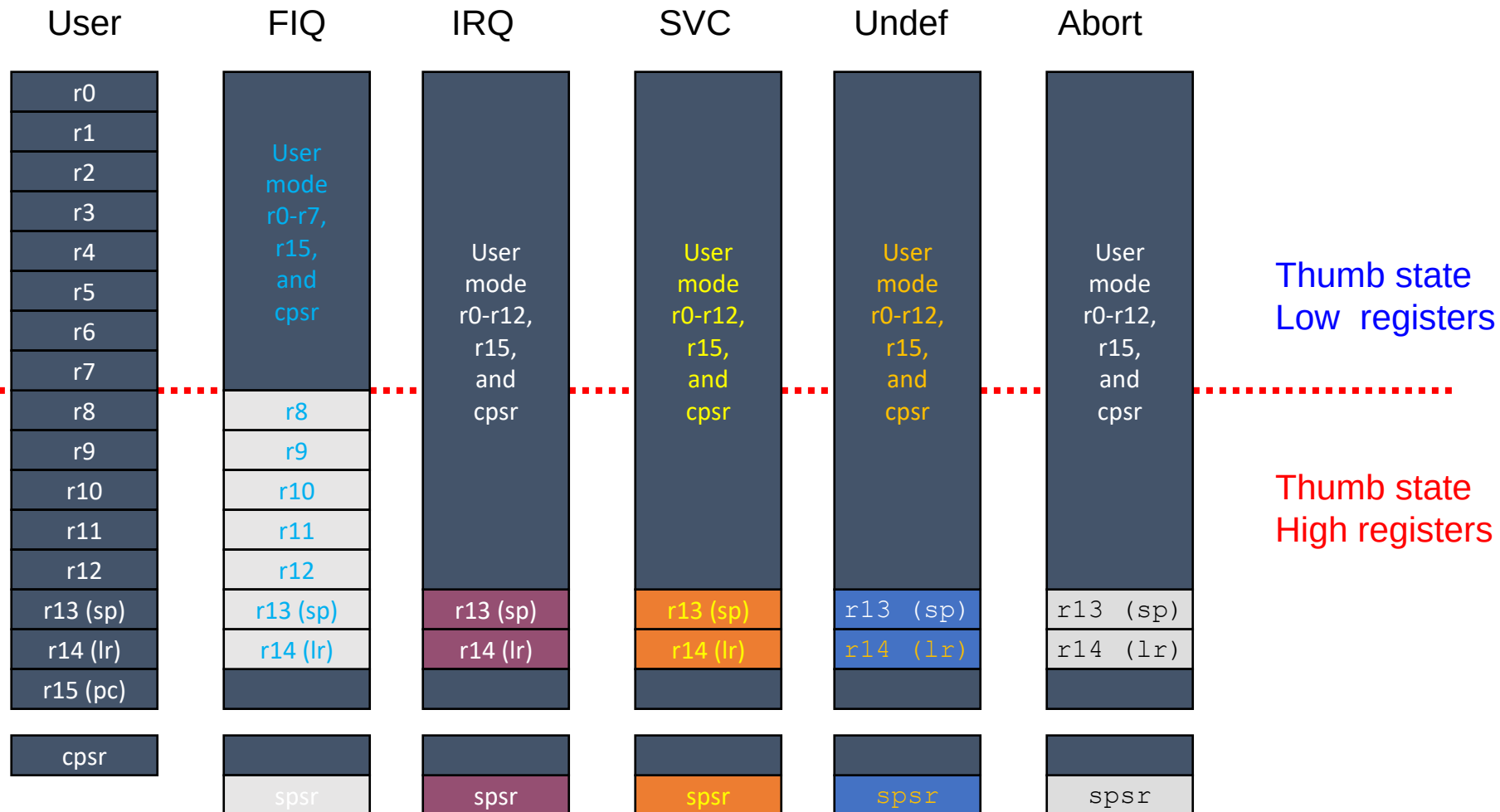
FIQ

Common R0-R7, Dedicated R8-R12, Dedicated R13 (SP), Dedicated R14(LR), Common R15 (PC) &
CPSR and Dedicated SPSR

PC	1	Common
CPSR	1	Common
SPSR	5	All Privileged mode
GPR	8	Common R0-R7
GPR	5	Common R8-R12
Spl GPR	5	FIQ R8-R12
SP	6	All Mode
LR	6	All Mode
	37	

Microprocessor & Computer Architecture (μpCA)

Register Organization Summary



Note: System mode uses the User mode register set

Microprocessor & Computer Architecture (μpCA)

Status Register : CPSR & SPSR



Current Program Status Register

- ARM core uses CPSR to monitor & control internal operations.
- The current status of the program under execution, such as, result of current execution instruction is zero/ -ve are captured here.

Saved Program Status Register

- Processor, While Shifting one mode to another mode, CPSR will be copied to SPSR.
- SPSR will be copied back to CPSR when it return back to previous mode.

Example:

Suppose Processor is in IRQ mode of operation and if FIQ request arrives then processor has to switch itself to FIQ mode of operation. First CPSR get copied to SPSR of IRQ mode, then processor will serve FIQ mode. After serving FIQ mode processor should return to IRQ mode and should resume its working. So SPSR of IRQ mode gets copied again into CPSR to serve in the IRQ mode.

[Click here for Reference](#)

Microprocessor & Computer Architecture (μpCA)

The ARM Register Set

Current Visible Registers

Abort Mode

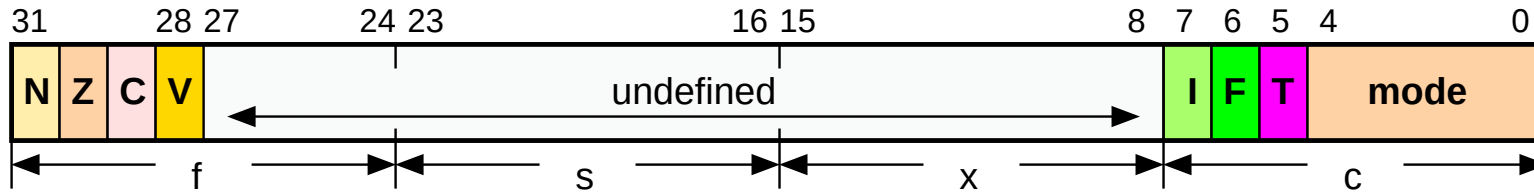
r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

Banked out Registers

User	FIQ	IRQ	SVC	Undef
	r8			
	r9			
	r10			
	r11			
	r12			
r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)
	spsr	spsr	spsr	spsr

Microprocessor & Computer Architecture (μpCA)

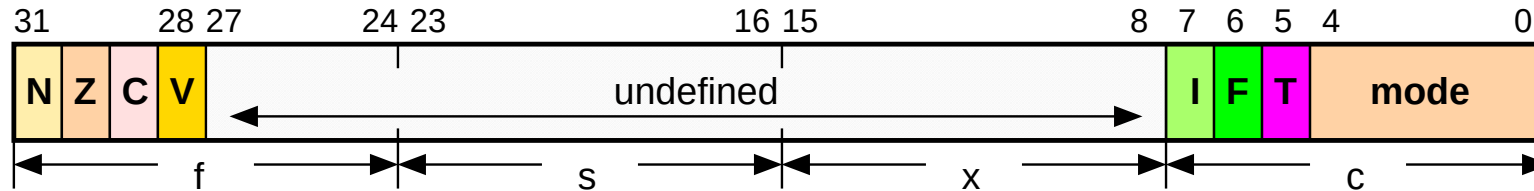
Status Register (SR)



SR fields are divided into four fields, each 8-bit wide:

- Flags (f)
- Status (s) : *Reserved for Future*
- Extension (x) : *Reserved for Future*
- Control (c).

Microprocessor & Computer Architecture (μpCA)



- **N: Negative**: Set to 1 if the Result of previous instruction is Negative.
- **Z: Zero**: Set to 1 if the Result of previous instruction is Zero.
- **C: Carry**: Set to 1, if result of either an Arithmetic or Shifter produce a carry-out,
- **V: oVerflow**: Set to 1, if previous instruction produce an overflow into the sign bit.



Mode bits

10000	User
10001	FIQ
10010	IRQ
10011	Supervisor
10111	Abort
11011	Undefined
11111	System

Interrupt Disable bits.

I = 1: Disables the IRQ.

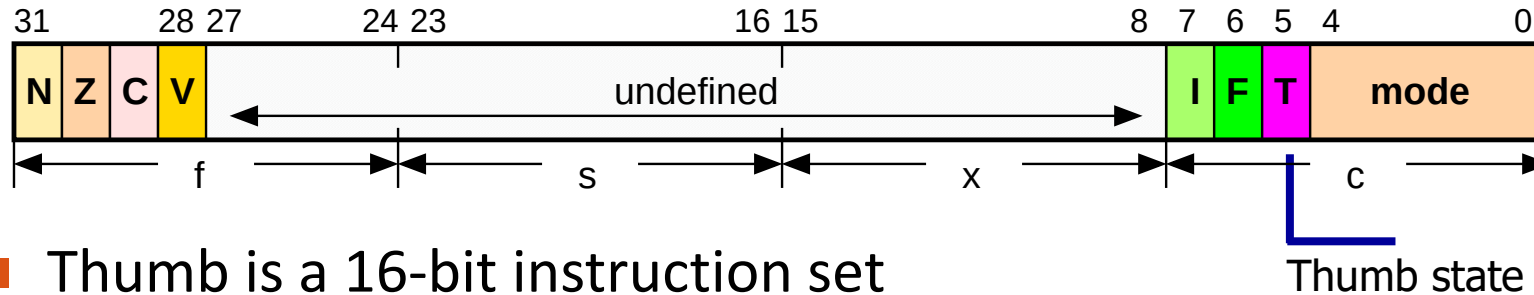
F = 1: Disables the FIQ.

T Bit (Arch. with Thumb mode only)

T = 0: Processor in ARM state

T = 1: Processor in Thumb state

Thumb Mode



- Thumb is a 16-bit instruction set
- Optimized for **code density** from C code (~65% of ARM code size)
- Improved performance from memory with a **narrow data bus**
- **Subset** of the functionality of the ARM instruction set
- Only Low Registers R0-R7 are used
- Constants are of limited size.
- Inline barrel shifter not used.

Microprocessor & Computer Architecture (μpCA)

I & F Bits



I F

- 1 1 FIQ is served, IRQ and FIQ is disabled
- 1 0 IRQ is served, IRQ is disabled & FIQ is enabled
- 0 1 **FIQ is served, IRQ is enabled & FIQ is disabled (Not Allowed)**
- 0 0 USER program is served, IRQ and FIQ both are enabled

Unit 1: Basic Processor Architecture and Design

- ~~Microprocessor Overview~~
- ~~CISC VS RISC~~
- ~~Introduction to ARM Processor & Applications~~
- ~~ARM Architecture Overview~~
- ~~Different ARM processor Modes~~
- ~~Register Bank~~
- ARM Program structure
- ARM Instruction Format



THANK YOU

Dr. D. C. Kiran

Department of Computer Science and Engineering

dckiran@pes.edu

9829935135



Microprocessor & Computer Architecture (μ pCA)

ARM Program Structure & Instruction Format

Microprocessor & Computer Architecture (μpCA)

ARM Program Structure



Address	Instruction & Data
.text	
Address of Instruction 1	ARM Instruction_1
Address of Instruction 2	ARM Instruction_3
Address of Instruction 3	ARM Instruction_3
.....	
.....	
Address of Instruction n	ARM Instruction_n
.data	
Address of Data1	Declaration of variable 1
Address of Data2	Declaration of variable 2
.....	
.....	
Address of Data n	Declaration of variable n

Note: Blue color depict the code written by the programmer
Red color depict the address assigned during execution

Microprocessor & Computer Architecture (μpCA)

ARM Program Structure



Address	Instruction	Meaning
	.text	
00001000:EF9F0014	LDR R0, =a	Load the Address of a to R0
00001004:EF9F1014	LDR R1,=b	Load the Address of b to R1
00001008:EF9F3014	LDR R3, =c	Load the Address of c to R3
0000100C:E5D14000	LDR R4, [r1]	Load the value (100) to R4
00001010:E5D05000	LDR R5, [r0]	Load the value (200) to R5
00001014:E0846005	Add R6, R4, R5	Add R4 & R5
00001018:E00360B0	STR R6, [r3]	Store the result(300) in the address specified in R3
	.data	
00001028:	a: .word 100	Variable a of data type word
00001029:	b: .word 200	Variable b of data type word
0000102A:	c: .word 0	Variable c of data type word

Note: the datatype may be byte, half word, asciz

Microprocessor & Computer Architecture (μpCA)

Register View (ARMSIM Simulator)



```
.text
00001000:EF9F0014  LDR R0, =a
00001004:EF9F1014  LDR R1,=b
00001008:EF9F3014  LDR R3, =c
0000100C:E5D14000  LDR R4, [r1]
00001010:E5D05000  LDR R5, [r0]
00001014:E0846005  ADD R6, R4, R5
00001018:E00360B0  STR R6, [r3]

.data
00001028:  a: .word 100
00001029:  b: word 200
0000102A:  c: word 0
```

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:00001028
R1	:00001029
R2	:00000000
R3	:0000102a
R4	:000000c8
R5	:00000064
R6	:0000012c
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001060

CPSR Register	
Negative (N)	:0
Zero (Z)	:0
Carry (C)	:0
Overflow (V)	:0
IRQ Disable	:1
FIQ Disable	:1
Thumb (T)	:0
CPU Mode	:System

0x000000df	

Microprocessor & Computer Architecture (μpCA)

Salient features of ARM Instructions



- Each Instruction is 32 bit wide
- Load-store architecture
- 3-address instructions.
- Conditional execution of every instruction.
- Possible to load/store multiple register at once.
- Possible to combine shift and ALU operations in a single instruction.
- No Memory-Memory Operations

MNEMONIC{condition}{S} {Rd},Operand1,Operand2

MNEMONIC - Short name of the instruction. *Eg: ADD,SUB....*

{condition} - Condition that is needed to be met in order for the instruction to be executed *EG: EQ, MI,GT,LT,LE,AL,NE*

{S} - An optional suffix. If S is specified, the condition flags are updated on the result of the operation . *Eg: To set N,O, C, V of CPSR*

{Rd} - Register (destination) for storing the result of the instruction

Operand1 - First operand. Either a register or an immediate value

Operand2 - Second (flexible) operand. Can be an immediate value (number) or a register with an optional shift

Microprocessor & Computer Architecture (μpCA)

General Format of Instruction Example

Example 1:

MOV r0, #-10

MOV r1, #-10

ADD r2, r1,r0

```
R0      :-10
R1      :-10
R2      :-20
R3      :0
R4      :0
R5      :0
R6      :0
R7      :0
R8      :0
R9      :0
R10 (s1) :0
R11 (fp) :0
R12 (ip) :0
R13 (sp) :21504
R14 (lr) :0
R15 (pc) :70656
-----
CPSR Register
Negative (N) :0
Zero (Z)      :0
Carry (C)     :0
Overflow (V)  :0
IRQ Disable:1
FIQ Disable:1
Thumb (T)     :0
CPU Mode      :System
-----
```

Microprocessor & Computer Architecture (μpCA)

General Format of Instruction Example

Example2:

MOV r0, #-10

MOV r1, #-10

ADDS r2, r1,r0

```
R0      :-10
R1      :-10
R2      :-20
R3      :0
R4      :0
R5      :0
R6      :0
R7      :0
R8      :-10
R9      :0
R10 (s1) :0
R11 (fp) :0
R12 (ip) :0
R13 (sp) :21504
R14 (lr) :0
R15 (pc) :70656
-----
CPSR Register
Negative (N) :1
Zero (Z)     :0
Carry (C)    :1
Overflow (V) :0
IRQ Disable :1
FIQ Disable :1
Thumb (T)    :0
CPU Mode     :System
-----
```

Microprocessor & Computer Architecture (μpCA)

General Format of Instruction Example

Example 3

```
.text
MOV r0,#-44
MOV r1,#4
SUBS r2,r0,r1
ADDMIS r0,r0,r1
.end
```

- Subtract r1 from r0
- Add only if the result of SUB is -ve
(Here result of SUB is -ve so add is executed)

```
R0      :-40
R1      : 4
R2      :-48
R3      : 0
R4      : 0
R5      : 0
R6      : 0
R7      : 0
R8      : 0
R9      : 0
R10 (s1) : 0
R11 (fp) : 0
R12 (ip) : 0
R13 (sp) : 21504
R14 (lr) : 0
R15 (pc) : 70656
-----
CPSR Register
Negative (N) : 1
Zero (Z)     : 0
Carry (C)    : 0
Overflow (V) : 0
IRQ Disable  : 1
FIQ Disable  : 1
Thumb (T)    : 0
CPU Mode     : System
-----
```

Microprocessor & Computer Architecture (μpCA)

General Format of Instruction Example

Example 4

```
.text
MOV r0,#44
MOV r1,#-4
SUBS r2,r0,r1
ADDMIS r0,r0,r1
.end
```

- Subtract r1 from r0
- Add only if the result of SUB is -ve
(Here result of SUB is not -ve so add is not executed)

```
R0      : 44
R1      : -4
R2      : 48
R3      : 0
R4      : 0
R5      : 0
R6      : 0
R7      : 0
R8      : 0
R9      : 0
R10 (s1) : 0
R11 (fp) : 0
R12 (ip) : 0
R13 (sp) : 21504
R14 (lr) : 0
R15 (pc) : 70656
-----
CPSR Register
Negative (N) : 0
Zero (Z)     : 0
Carry (C)    : 0
Overflow (V) : 0
IRQ Disable  : 1
FIQ Disable  : 1
Thumb (T)    : 0
CPU Mode     : System
-----
```

Microprocessor & Computer Architecture (μpCA)

General Format of Instruction Example

Example 5

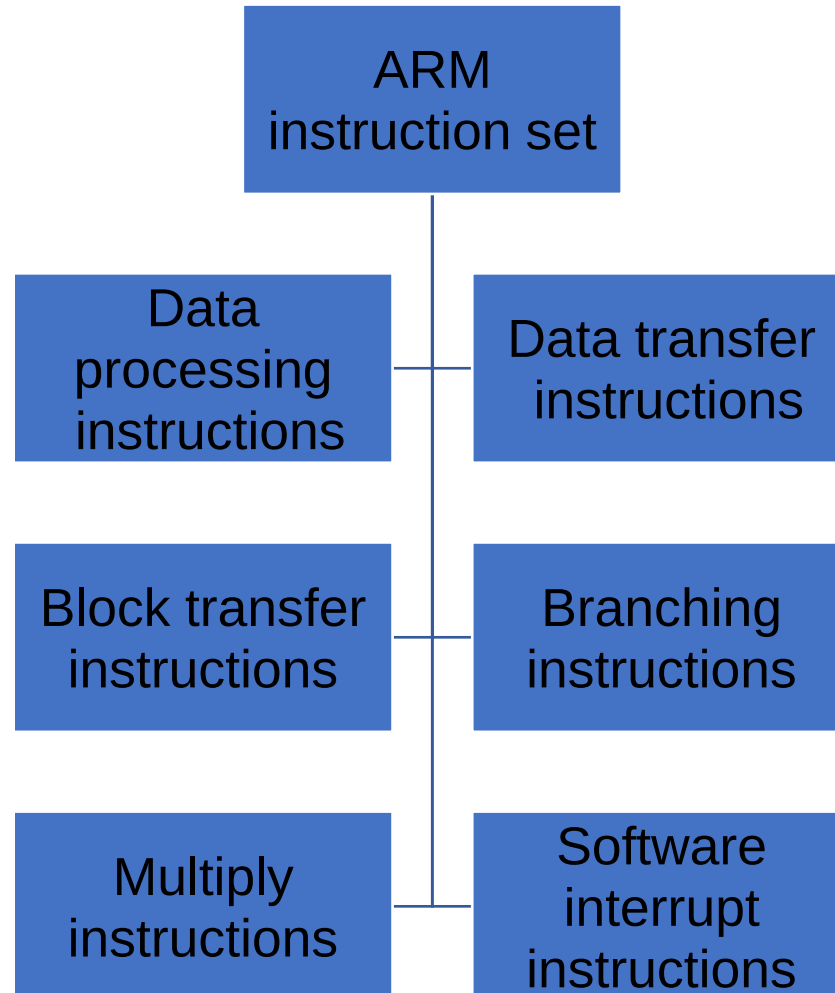
```
.text
MOV r0,#44
MOV r1,#4
CMP r1,r0
ADDEQ r0,r0,r1
.end
```

- Compare r1 and r0
- Add only if the comparison is true i.e N=0
(Here comparison fails i.e N=1)

```
R0      : 44
R1      : 4
R2      : 0
R3      : 0
R4      : 0
R5      : 0
R6      : 0
R7      : 0
R8      : 0
R9      : 0
R10 (s1) : 0
R11 (fp) : 0
R12 (ip) : 0
R13 (sp) : 21504
R14 (lr) : 0
R15 (pc) : 70656
-----
CPSR Register
Negative (N) : 1
Zero (Z)     : 0
Carry (C)    : 0
Overflow (V)  : 0
IRQ Disable  : 1
FIQ Disable  : 1
Thumb (T)    : 0
CPU Mode     : System
-----
```

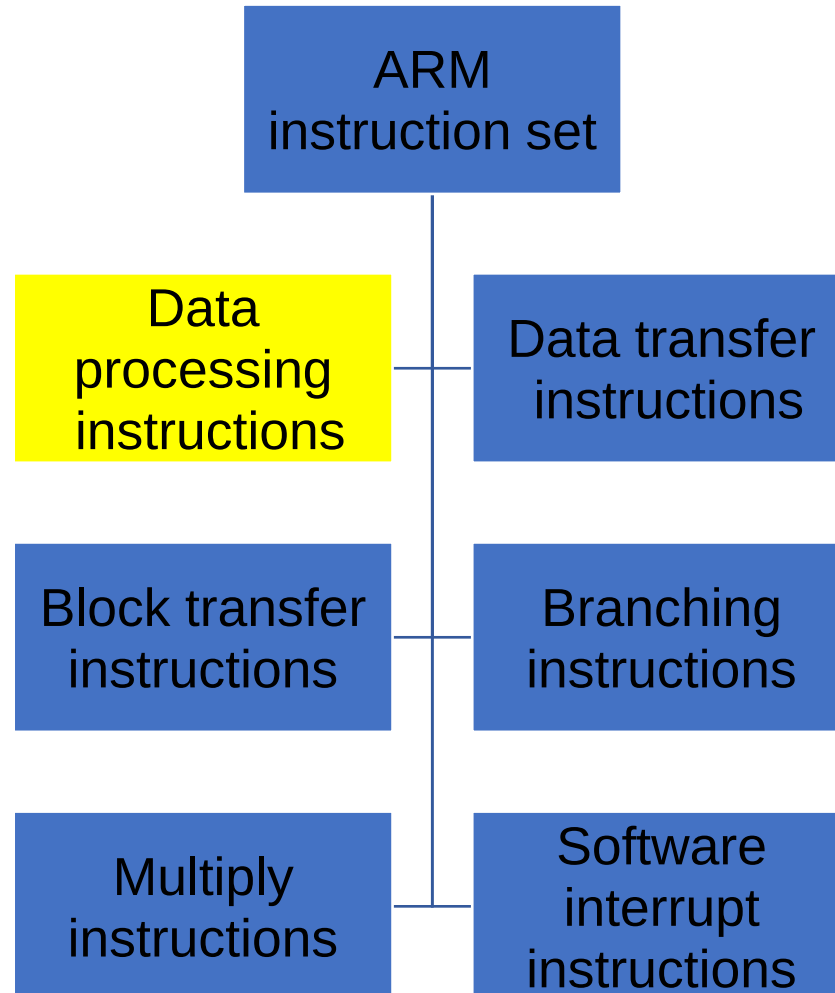
Microprocessor & Computer Architecture (μpCA)

ARM Instruction Set



Microprocessor & Computer Architecture (μpCA)

Next Session



[Click Here for reference](#)



THANK YOU



Team MPCA

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

Session - 4

ARM instructions are grouped as follows.

1. Data Processing Instructions.

Ex: MOV, ADD, SUB, CMP, CMN, etc.,

2. Branch Instructions.

Ex: B, BL, BGT, BLT, BGE, BLE, etc.,

3. Data Transfer Instructions.

Ex: LOAD – STORE Instructions- LDR, STR, LDM, STM, etc.,

4. Software Interrupt Instructions.

Ex: SWI 0x11, etc.,

5. Program Status Register Instructions.

Ex: MSR, MRS, etc.,

1. Data Processing Instructions.

MOVE instruction

Syntax: <mnemonic><cond><s> <dest Rd>, <src op1 Rn>

Ex1: **MOV** **R0 , R1**
 R0 ← R1

***Moves** the **contents** of the source operand registers **R1** and **copies** to the destination register **R0**.*

Ex2: **MVN** **R0, R1**
 R0 ← negate(R1)

***Moves** the **complemented value** of the source operand register **R1** and **stores** to the destination register **R0**.*

Arithmetic Instruction:

Syntax: <mnemonic><cond><s> <dest Rd>, <src op1 Rn >, <src op2 Rm>

Ex3: **ADD** **R3,** **R1,** **R2**
 R3 **←** **R1** **+** **R2**

Adds the contents of the source operand registers R1 & R2 and stores the result in the destination register R3.

Compare Instruction:

Syntax:

 <mnemonic><cond><s> <dest Rd>, <src op1 Rn >,
Ex4: **CMP** **R3,** **R1**

Neither source nor destination fields are affected.

Ex4: **CMN** **R3,** **R1**

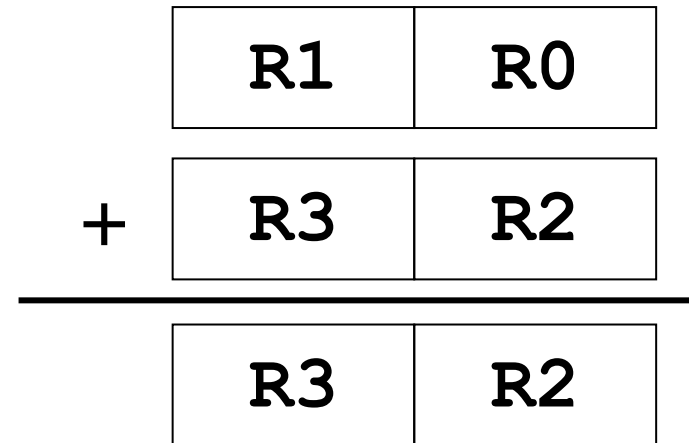
Performs the **operation R3 + R1** and **sets** the condition codes / flags.

Neither source nor destination fields are affected.

Setting the condition codes

- Any data processing instruction can set the condition codes if the programmers wish it to

64-bit addition



ADD_S R2 , R2 , R0

ADC R3 , R3 , R1

Comparison

compare

- **CMP** R1, R2 @ set cc on R1-R2

compare negated

- **CMN** R1, R2 @ set cc on R1+R2

bit test

- **TST** R1, R2 @ set cc on R1 and R2

test equal

- **TEQ** R1, R2 @ set cc on R1 xor R2

Arithmetic Operations

- **MOV R4, #10**
- **MVN R5, R4**
- **MOV R1, #5**
- **MOV R2, #2**

- **ADD r0, r1, r2** ;r0:=r1+r2
- **ADC r0, r1, r2** ;r0:=r1+r2+C
- **SUB r0, r1, r2** ;r0:=r1-r2
- **SBC r0, r1, r2** ;r0:=r1-r2+C-1
- **RSB r0, r1, r2** ;r0:=r2-r1, **Reverse Subtraction**
- **RSC r0, r1, r2** ;r0:=r2-r1+C-1

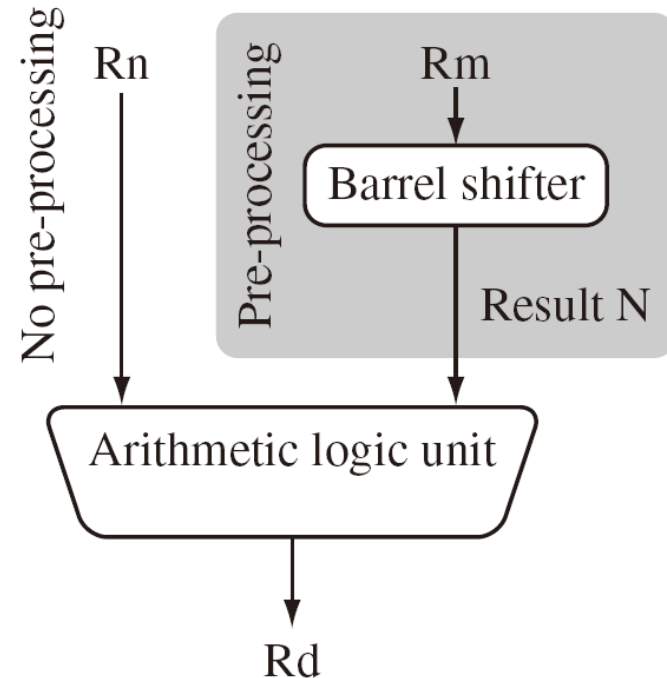
By default data processing operations do not affect the condition flags

Bit-wise Logical Operations

- **AND** r0, r1, r2 ;r0 := r1 **AND** r2
- **ORR** r0, r1, r2 ;r0 := r1 **OR** r2
- **EOR** r0, r1, r2 ;r0 := r1 **XOR** r2
- **BIC** r0, r1, r2 ;r0 := r1 **AND** (**NOT** r2), bit clear

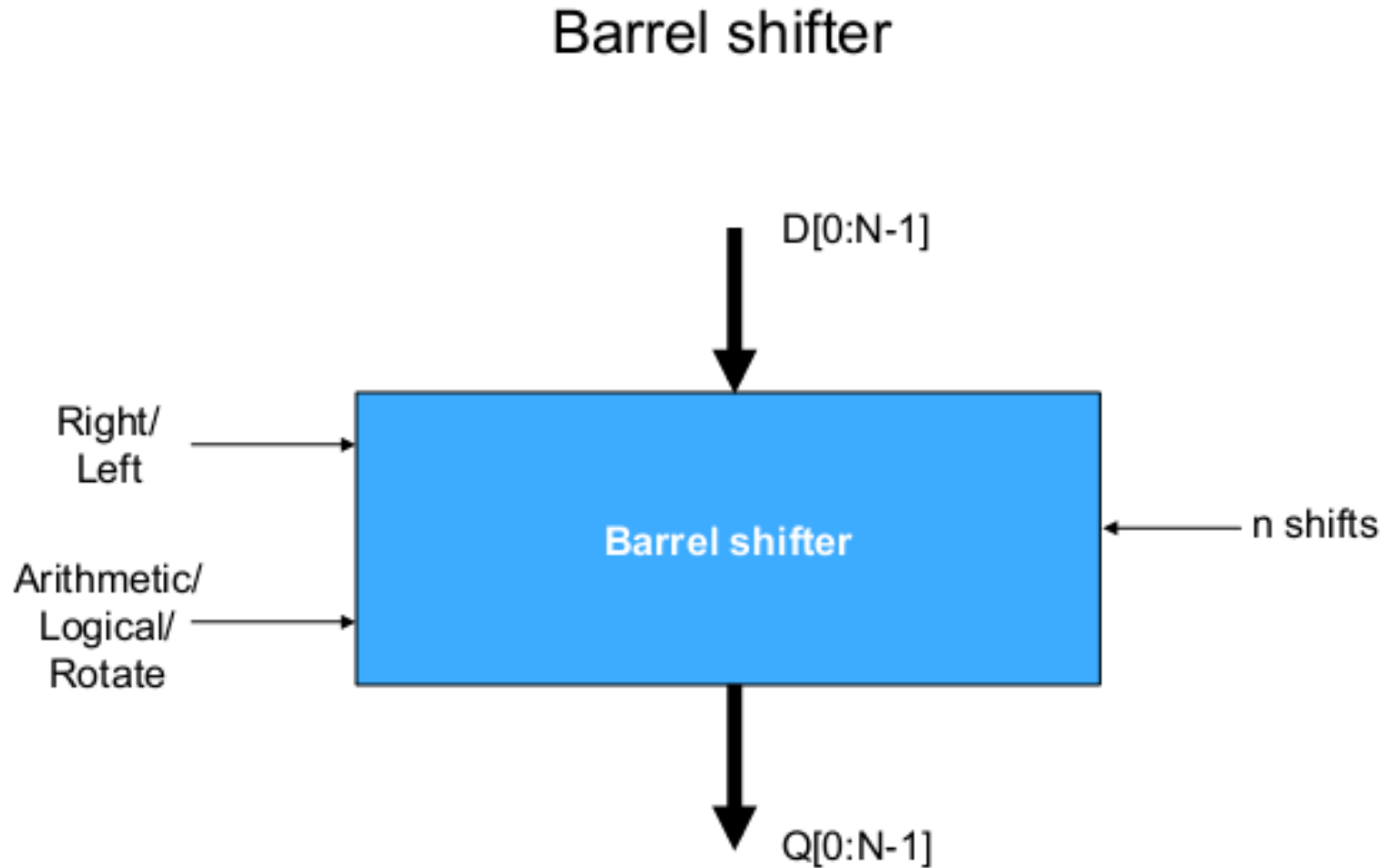
Shifted register operands

- One operand to ALU is routed through the **Barrel shifter**. Thus, the operand can be modified before it is used. Useful for fast multiplication and dealing with lists, table and other complex data structure. (similar to the displacement addressing mode in CISC.)



- Some instructions (e.g. **MUL**, **CLZ**, **QADD**) do not read barrel shifter.

BARREL SHIFTER - BLOCK DIAGRAM



BARREL SHIFTER

- ARM architecture supports instructions which perform a shift with ALU operation.
- The shifter performance is therefore critical as it contributes directly to the datapath cycle time.
- To minimize the delay through the shifter, A cross-bar switch matrix is used to steer each input to appropriate output.
- Example shown is 4 x 4 matrix cross bar switch.
- ARM uses a 32 x 32 matrix cross bar switch.
- Each input is connected to each output through a switch.

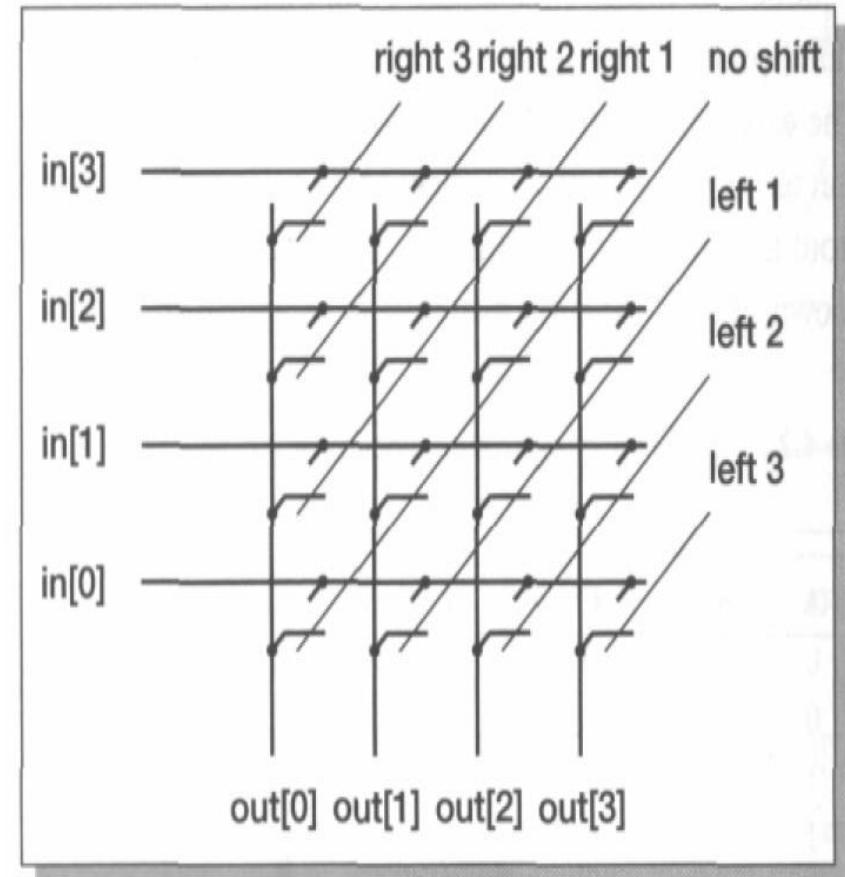


Figure 4.15 The cross-bar switch barrel shifter principle.

BARREL SHIFTER - FUNCTION

- Shift Operation – Left
 - One left shift diagonal line is turned on.
- Shift Operation – Right
 - One right shift diagonal line is turned on.
- Arithmetic Shift right
 - Uses sign extension rather than zero filling for unconnected output.
- Rotate operation
 - The right shift diagonal is enabled together with complementary left shift diagonal.

Mnemonic	Description	Shift	Result
LSL	logical shift left	$x\text{LSL } y$	$x \ll y$
LSR	logical shift right	$x\text{LSR } y$	$(\text{unsigned})x \gg y$
ASR	arithmetic right shift	$x\text{ASR } y$	$(\text{signed})x \gg y$
ROR	rotate right	$x\text{ROR } y$	$((\text{unsigned})x \gg y) (x \ll (32 - y))$
RRX	rotate right extended	$x\text{RRX}$	$(c\text{ flag} \ll 31) ((\text{unsigned})x \gg 1)$

MULTIPLICATION BY CONSTANT USING THE BARREL SHIFTER

- **Multiplication by 2^n (1,2,4,8,16,32..)**

`MOV Ra, Rb, LSL #n`

- **Multiplication by 2^{n+1} (3,5,9,17..)**

`ADD Ra,Ra,Ra,LSL #n`

- **Multiplication by 2^{n-1} (3,7,15..)**

`RSB Ra,Ra,Ra,LSL #n`

- **Multiplication by 6**

`ADD Ra,Ra,Ra,LSL #1` ; multiply by 3

`MOV Ra,Ra,LSL#1` ; and then by 2

- **Multiply by 10 and add in extra number**

`ADD Ra,Ra,Ra,LSL#2` ; multiply by 5

`ADD Ra,Rc,Ra,LSL#1` ; multiply by 2 and add in next digit

BARREL SHIFTER - EXAMPLES

- Shift Operation – Left
 - One left shift diagonal line is turned on.

Example 1: ADD R0, R1, R2, LSL #3

- Since LSL is used, it selects left diagonal #3.
- Rotate operation
 - The right shift diagonal is enabled together with complementary left shift diagonal.

Example 2: ADD R0, R1, R2, ROR #3

- Selects Right diagonal #3 and left diagonal #29.

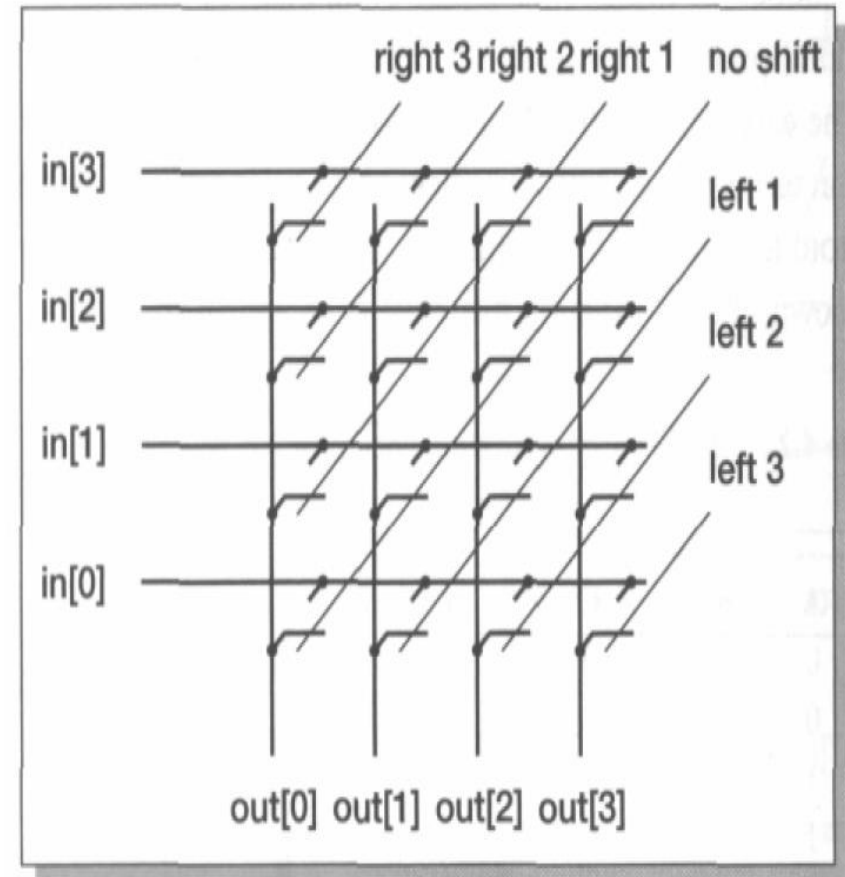


Figure 4.15 The cross-bar switch barrel shifter principle.

Logical shift left



```
MOV  R0, R2, LSL #2 @ R0:=R2<<2  
                        @ R2 unchanged
```

Example: 0...0 0011 0000

Before R2=0x00000030

After R0=0x000000C0

R2=0x00000030

Logical shift right



```
MOV  R0, R2, LSR #2 @ R0:=R2>>2  
                        @ R2 unchanged
```

Example: 0...0 0011 0000

Before R2=0x00000030

After R0=0x0000000C

R2=0x00000030

Arithmetic shift right



```
MOV  R0, R2, ASR #2 @ R0 := R2 >> 2  
                        @ R2 unchanged
```

Example: **1010 0...0 0011 0000**

Before R2=0xA0000030

After R0=0xE800000C

R2=0xA0000030

Rotate right



```
MOV  R0, R2, ROR #2 @ R0:=R2 rotate  
                        @ R2 unchanged
```

Example: 0...0 0011 0001

Before R2=0x00000031

After R0=0x4000000C

R2=0x00000031

Rotate right extended



MOV R0, R2, **RRX** @ R0:=R2 rotate
@ R2 unchanged

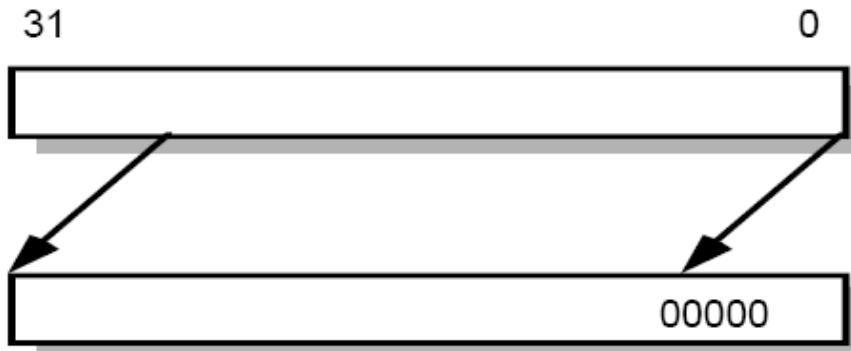
Example: **0...0 0011 0001**

Before R2=0x00000031, C=1

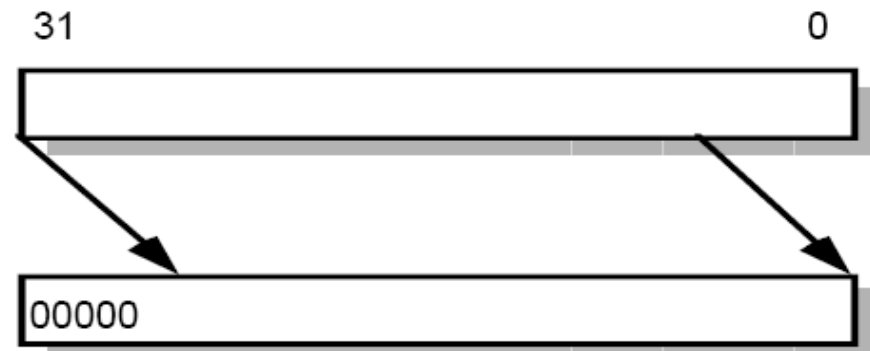
After R0=0x80000018, C=1

R2=0x00000031

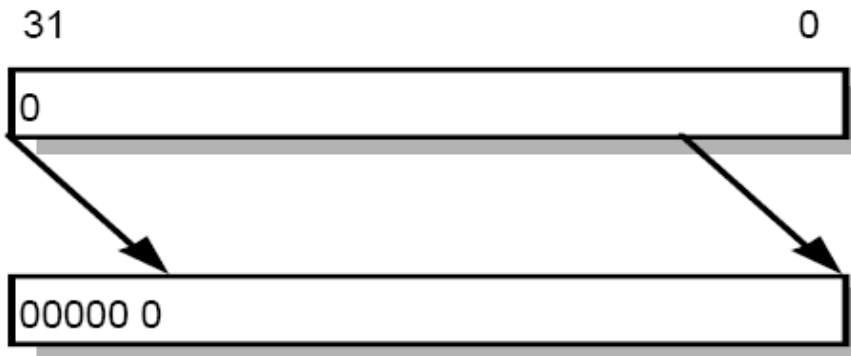
Shifted register operands



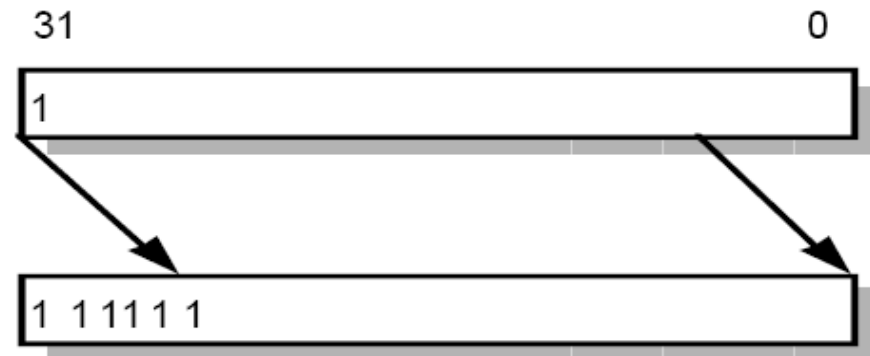
LSL #5



LSR #5

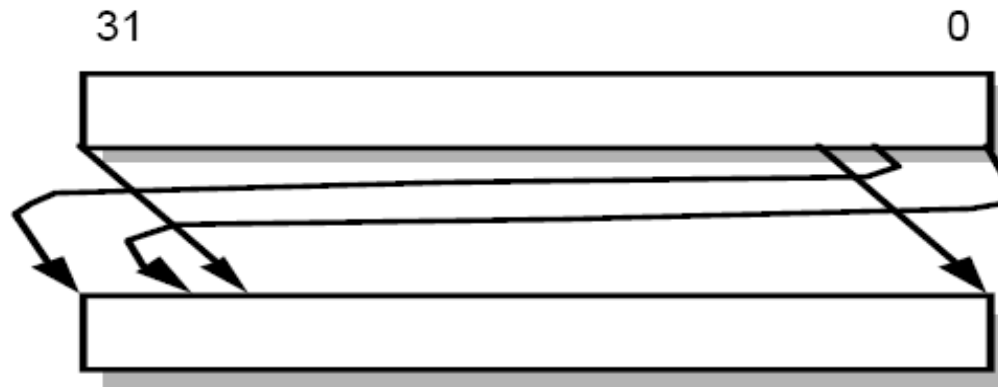


ASR #5 , positive operand

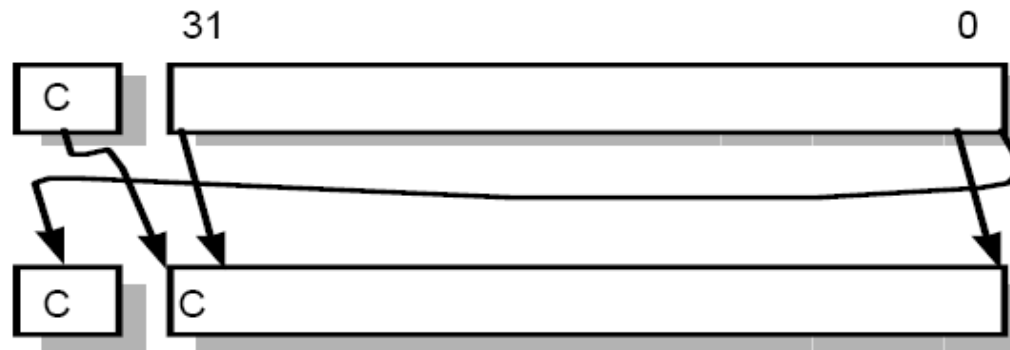


ASR #5 , negative operand

Shifted register operands



ROR #5



RRX

Shifted register operands

- It is possible to use a register to specify the number of bits to be shifted; only the bottom 8 bits of the register are significant.

@ array index calculation

ADD R0, R1, R2, LSL R3 @ $R0 := R1 + R2 * 2^{R3}$

@ fast multiply $R2 = 35 \times R0$

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

Shifted register operands

<i>N</i> shift operations	Syntax
Immediate	#immediate
Register	Rm
Logical shift left by immediate	Rm, LSL #shift_imm
Logical shift left by register	Rm, LSL Rs
Logical shift right by immediate	Rm, LSR #shift_imm
Logical shift right with register	Rm, LSR Rs
Arithmetic shift right by immediate	Rm, ASR #shift_imm
Arithmetic shift right by register	Rm, ASR Rs
Rotate right by immediate	Rm, ROR #shift_imm
Rotate right by register	Rm, ROR Rs
Rotate right with extend	Rm, RRX

Multiplication as Addition & Subtraction

MOV R1, #35

MUL R2, R0, R1

or

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

Using the ARM instruction set, write Assembly level language instructions / sequence / programs to perform the following as directed.

1. Compile the following C assignment statements into ARM.
 - i. $A = B + C;$
 - ii. $D = A - C;$
 - iii. Perform $D = A - C$ using RSB instruction.
2. Convert the following assignment into ARM.
 - i. $F = (G + H) - (I + J).$
use the register **R0 to R4** as operands **F to J** respectively.
3. Convert the statement into ARM where the operand is in memory.
 - i. $G = H + A[10].$

Assignment.

Note: User different addressing modes both at source & destination operand positions.



Arm Instruction Set – data processing group



THANK YOU



Team MPCA

Department of Computer Science
and Engineering



Microprocessor & Computer Architecture (μ pCA)

Branch Instructions & Addressing Modes

Session - 5

Microprocessor & Computer Architecture (μpCA)

Branch / Flow Control Instructions



Syntax: B{<cond>} Label
BL{<cond>} Label
BX{<cond>} Rm
BLX{<cond>} Rm

B	Branch	Program Counter = Label
BL	Branch & Link	Step1: PC will be copied to R14 the Link Register (LR) before branch is taken. Step2: Program Counter = Label
BX	Branch Exchange	Used for changing ARM to Thumb mode or from Thumb mode to ARM mode.
BLX	Branch Exchange with link	

Flow Control Instructions

- Branch Instruction- (Unconditional)

```
B    label
```

```
...
```

```
label:  ...
```

- Conditional Branch Instruction

```
MOV    R0, #0
```

```
loop:  ...
```

```
ADD    R0, R0, #1
```

```
CMP    R0, #10
```

```
BNE   loop
```

Flow Control Instructions

- **Exercise-1:** Program to find the sum of N NUMBERS (use conditional branch).
- Store the result in memory location.

Program:

.DATA		L1:	LDR R3, [R1]
	A: .WORD 10,20,30,40,50,60,70,80,90,100		ADD R4,R4,R3 ; Add next element in the array.
	SUM: .WORD 0		ADD R1, R1, #4 ; address to the next data
.TEXT			ADD R5, R5, #1 ; increment the count register
LDR R1,=A			CMP R5, #11 ; Check whether all numbers are added
LDR R2,=SUM			BNE L1 ; Else branch to L1 location
MOV R4,#0 ; INITIALISATION			STR R4,[R2] ; store the result in location SUM.
MOV R5,#1 ; COUNT register			SWI 0X011 ; logical end of the program.

Microprocessor & Computer Architecture (μpCA)

Conditional Branch Options

Branch	Interpretation	Normal uses
B BAL	Unconditional Always	Always take this branch Always take this branch
BEQ	Equal	Comparison equal or zero result
BNE	Not equal	Comparison not equal or non-zero result
BPL	Plus	Result positive or zero
BMI	Minus	Result minus or negative
BCC	Carry clear	Arithmetic operation did not give carry-out
BLO	Lower	Unsigned comparison gave lower
BCS	Carry set Higher	Arithmetic operation gave carry-out
BHS	or same	Unsigned comparison gave higher or same
BVC	Overflow clear	Signed integer operation; no overflow occurred
BVS	Overflow set	Signed integer operation; overflow occurred
BGT	Greater than	Signed integer comparison gave greater than
BGE	Greater or equal	Signed integer comparison gave greater or equal
BLT	Less than	Signed integer comparison gave less than
BLE	Less or equal	Signed integer comparison gave less than or equal
BHI	Higher	Unsigned comparison gave higher
BLS	Lower or same	Unsigned comparison gave lower or same

Addressing Half Words

Program to find the sum of N numbers using **half word**

.DATA

A: **.HWORD** 0x10,0x20,0x30,0x40,0x50,0x60,0x70,
0x80,0x90, 0x0100

SUM: **.WORD** 00

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ;INITIALISATION

MOV R5,#1 ;COUNT

L1: **LDRH R3,[R1]**

ADD R4,R4,R3

ADD R1,R1,#2

ADD R5,R5,#1

CMP R5, #11

BNE L1

STRH R4, [R2]

SWI 0X011

Addressing Byte Data

Program to find the sum of N numbers using **Byte Data**

; SUM OF N NUMBERS

; DATA GIVEN

.DATA

A: **.BYTE** 1,2,3,4,5,6,7,8,9,10

SUM: .word 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ;INITIALISATION

MOV R5,#1 ;COUNT

L1: **LDRB R3,[R1]**

ADD R4,R4,R3

ADD R1,R1,#1

ADD R5,R5,#1

CMP R5, #11

BNE L1

STRB R4,[R2]

SWI 0X011

- Memory is addressed by **a register** and **an offset**.

```
LDR R0, [R1] @ mem[R1]
```

- Three ways to specify offsets:

- Immediate

```
LDR R0, [R1, #4] @ mem[R1+4]
```

- Register

```
LDR R0, [R1, R2] @ mem[R1+R2]
```

- Scaled Register

```
LDR R0, [R1, R2, LSL #2] @ mem[R1+4*R2]
```

1. Preindexing or Preindexing without writeback
2. Preindexing with Writeback or Autoindexing
3. Post indexing

- Pre Indexing Without Write Back

Syntax: **LDR Rd, [Rn, OFFSET]**

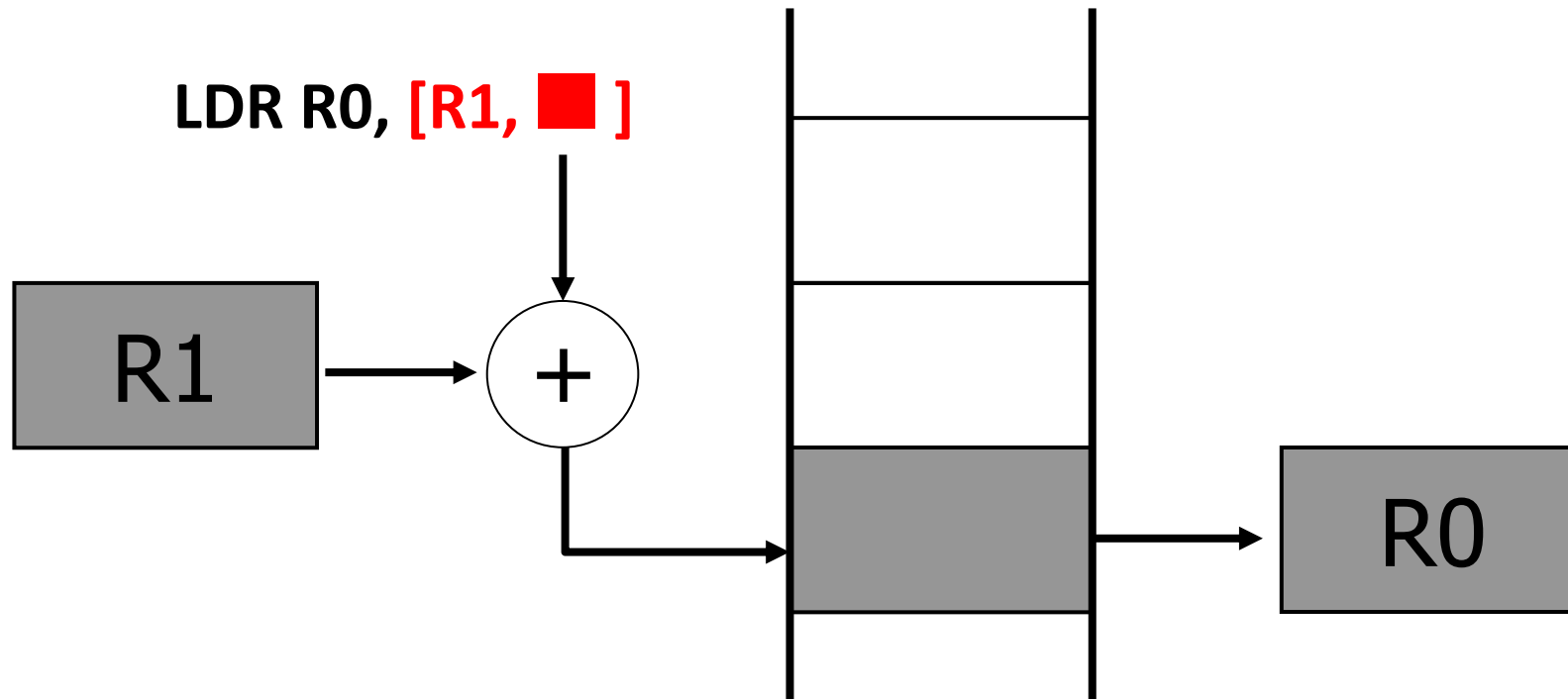
- Pre Indexing With Write Back

Syntax: **LDR Rd, [Rn, OFFSET]!**

- Poste Indexing

Syntax: **LDR Rd, [Rn] ,OFFSET**

LDR R0, [R1, #4] @ R0=mem[R1+4]
@ R1 unchanged



Microprocessor & Computer Architecture (μpCA)



Example – Using preindexing addressing mode

- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program:

.DATA		L1:	LDR R3, [R1, #4]
A: .WORD 10,20,30,40,50,60,70,80,90,100			ADD R4,R4,R3 ; Add next element in the array.
SUM: .WORD 0			ADD R1,R1,#4 ; useless
.TEXT			ADD R5, R5, #1 ; increment the count register
LDR R1,=A			CMP R5, #11 ; Check whether all numbers are added
LDR R2,=SUM			BNE L1 ; Else branch to L1 location
MOV R4,#0 ; INITIALISATION			STR R4,[R2] ; store the result in location SUM.
MOV R5,#1 ; COUNT register			SWI 0X011 ; logical end of the program.

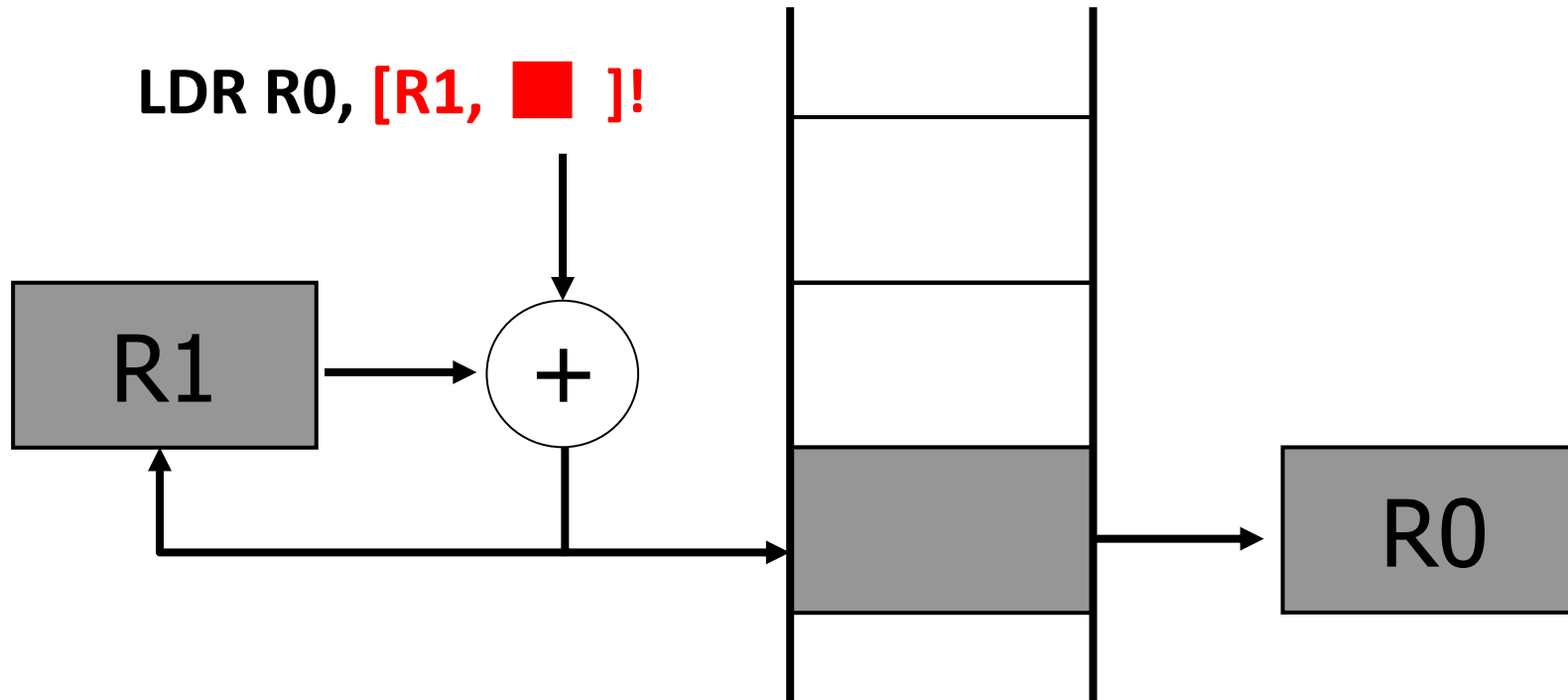
Has no impact on the preindexing mode....

Microprocessor & Computer Architecture (μpCA)

Auto-indexing addressing or Preindexing with writeback

LDR R0, [R1, #4]! @ R0=mem[R1+4]
@ R1=R1+4

No extra time; Fast;



Microprocessor & Computer Architecture (μpCA)

Example – Using preindexing with writeback or autoindexing addressing mode



- Program to find the sum of N NUMBERS
- Store the result in memory location.

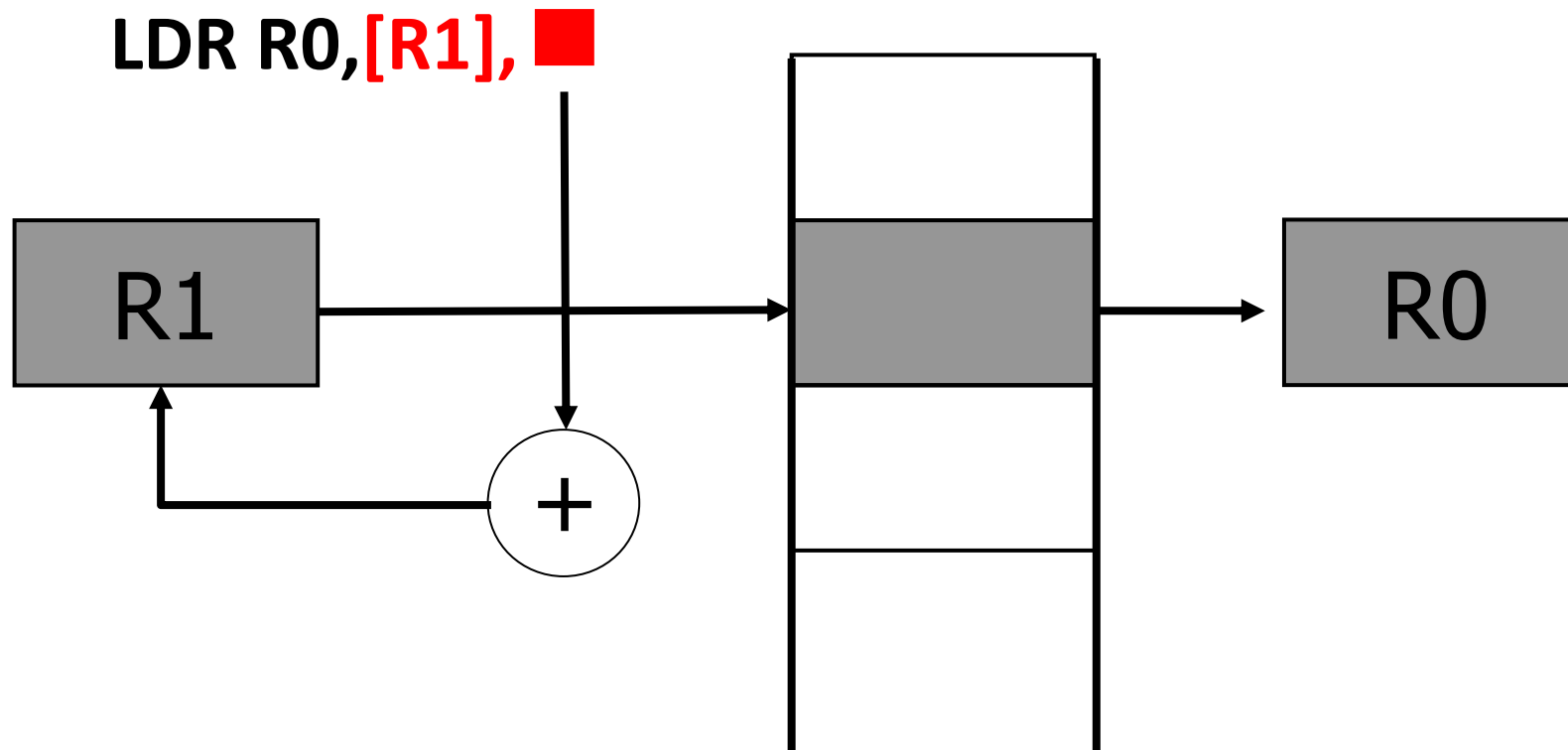
Program:

<pre>.DATA A: .WORD 10,20,30,40,50,60,70,80,90,100 SUM: .WORD 0 .TEXT LDR R1,=A LDR R2,=SUM MOV R4,#0 ; INITIALISATION MOV R5,#1 ; COUNT register</pre>	<pre>L1: LDR R3, [R1,#4]! ADD R4,R4,R3 ; Add next element in the array. ADD R5, R5, #1 ; increment the count register CMP R5, #11 ; Check whether all numbers are added BNE L1 ; Else branch to L1 location STR R4,[R2] ; store the result in location SUM. SWI 0X011 ; logical end of the program.</pre>
---	--

Microprocessor & Computer Architecture (μpCA)

Post-index addressing

LDR R0, [R1], #4 @ R0=mem[R1]
 @ R1=R1+4



Microprocessor & Computer Architecture (μpCA)

Example – Using postindexing Addressing Mode



- Program to find the sum of N NUMBERS and Store the result in memory location.

Program:

.DATA

A: .WORD 10,20,30,40,50,60,70,80,90,100

SUM: .WORD 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ; INITIALISATION

MOV R5,#1 ; COUNT register

L1: **LDR R3, [R1], #4**

ADD R4,R4,R3

ADD R5,R5,#1

CMP R5, #11

BNE L1

STR R4,[R2]

SWI 0X011



THANK YOU



Team MPCA

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE19CS252

Dr. D. C. Kiran

Department of
Computer Science and Engineering

Microprocessor & Computer Architecture (μ pCA)

Data Transfer Instructions

Dr. D. C. Kiran

Department of Computer Science and Engineering

Microprocessor & Computer Architecture (μpCA)

Syllabus

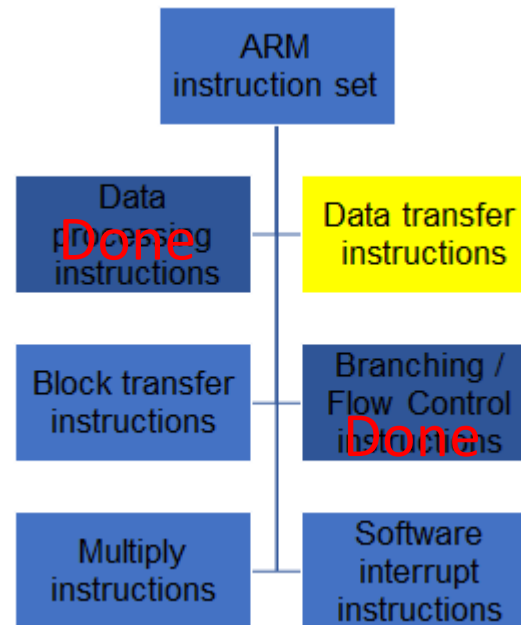
Unit 1: Basic Processor Architecture and Design

- ~~Microprocessor Overview~~
- ~~CISC VS RISC~~
- ~~Introduction to ARM Processor & Applications~~
- ~~ARM Architecture Overview~~
- ~~Different ARM processor Modes~~
- ~~Register Bank~~
- ~~ARM Program structure~~
- ~~ARM Instruction Format~~
- **ARM INSTRUCTION SET**

~~Data Processing Instructions~~

~~Flow Control Instructions~~

Data Transfer Instructions



Microprocessor & Computer Architecture (μpCA)

Memory system

- Memory is a linear array of bytes addressed from 0 to $2^{32}-1$
- Word, half-word, byte
- Little-endian

From ARMSIM

```
.text
00001000:E3A00013  mov r0,#19
00001004:E59F5018  ldr r5,=a
00001008:E5850000  str r0,[r5]
0000100C:E1A00000  mov r0,r0
00001010:E1A00000  mov r0,r0
00001014:E1A00000  mov r0,r0
00001018:E1A00000  mov r0,r0
0000101C:E1A00000  mov r0,r0
00001020:EF000011  swi 0x011
      .data
00001028:      a: .word 0
```

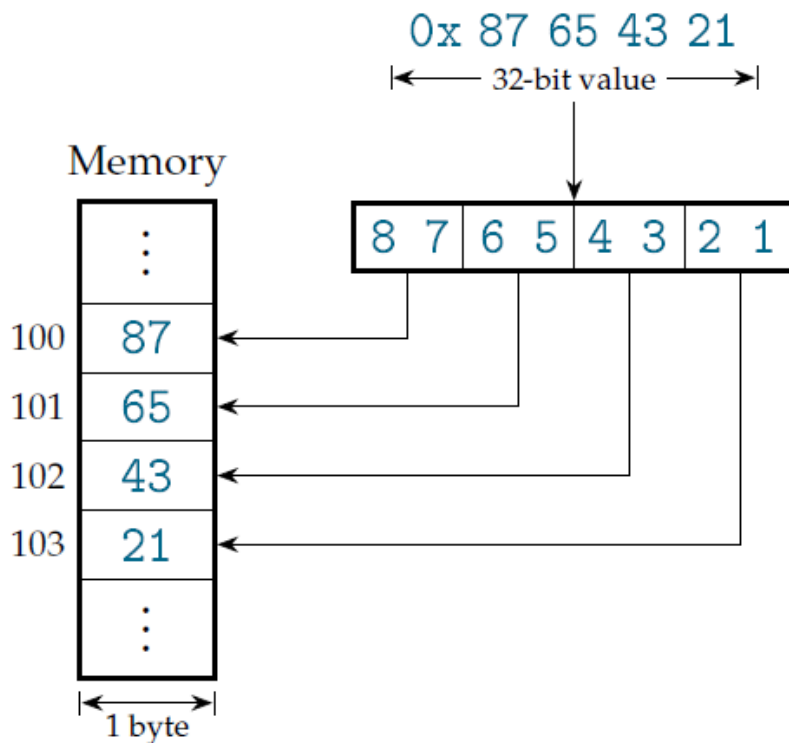
0x00000000	00
0x00000001	10
0x00000002	20
0x00000003	30
0x00000004	FF
0x00000005	FF
0x00000006	FF
0xFFFFFFFFD	00
0xFFFFFFFFE	00
0xFFFFFFFFF	00

Microprocessor & Computer Architecture (μpCA)

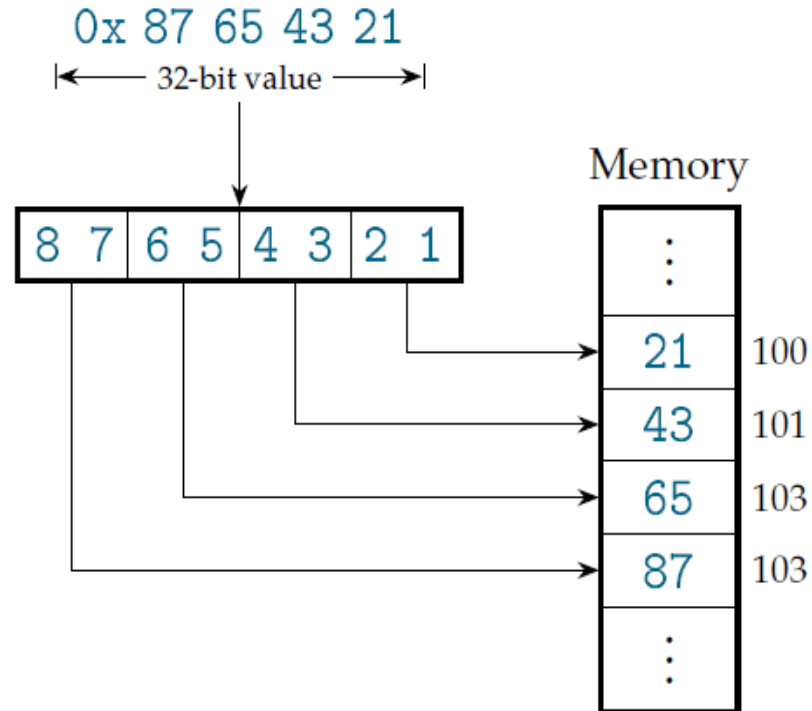
Little-Endian vs. Big-Endian

- Little-endian: least-significant byte first
- Big-endian: most-significant byte first

Big-Endian



Little-Endian



- Move data between registers and memory
- Basic forms
 - Single register load/store
 - Ex: **LDR**, **STR**
 - Multiple register load/store or Block Transfer
 - Ex: **LDM**, **STM**

Microprocessor & Computer Architecture (μpCA)

Single Register Load/Store



Syntax: <LDR/STR>{<cond>}{B} Rd, Addressing

LDR	Load word into register
STR	Save byte or word from register
LDRB	Load byte into register
STRB	Save byte from Register

Syntax: LDR{<cond>}SB/H/SH Rd, Addressing
STR{<cond>}H Rd, Addressing

LDRH	Load half word into register
STRH	Save half word from a register
LDRSB	Load signed byte into register
LDRSH	Load signed halfword into a Register

No **STRSB/STRSH** since **STRB/STRH** stores both signed/unsigned ones

Microprocessor & Computer Architecture (μpCA)

Copy A=B



	addr	data
	0x010	10
Let B →	0x014	
	0x018	30
Let A →	0x01C	40
	0x020	50
	0x024	60

LDR R0=A ;Where a is a variable i.e address is copied to Register R0= 0x1C

LDR R5, [R0] ; Copying the data in the address in R0 to R5=40

LDR R3=B ;Where b is a variable i.e address is copied to Register R3 = 0x14

STR R5,[R3] ; Store 40 in the address specified in R3=?

Microprocessor & Computer Architecture (μpCA)

Copy A=B



	addr	data
	0x010	10
Let B →	0x014	
	0x018	30
Let A →	0x01C	40
	0x020	50
	0x024	60

LDR R0=A ;Where a is a variable i.e address is copied to Register R0= 0x1C

LDR R5, [R0] ; Copying the data in the address in R0 to R5=40

LDR R3=B ;Where b is a variable i.e address is copied to Register R3 = 0x14

STR R5,[R3] ; Store 40 in the address specified in R3=40

Microprocessor & Computer Architecture (μpCA)

LDR and STR Example $B=A+9$



.text

LDR R0, =A → Address of A is copied to R0

LDR R3, =B → Address of B is copied to R3

MOV R4, #9 → Number 9 is copied to R4

LDR R5, [R0] → Value in the A i.e 10 is copied to R5

ADD R6, R4, R5 → $10+9=19$ copy result to R6

STR R6, [R3] → 19 is stored in the address in the register [R3], ie. $B=19$

.data

A: .word 10

B: word 0

Microprocessor & Computer Architecture (μpCA)

General Purpose

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 :00001020

R1 :00000000

R2 :00000000

R3 :00001024

R4 :00000009

R5 :0000000a

R6 :00000013

R7 :00000000

R8 :00000000

R9 :00000000

R10 (sl):00000000

R11 (fp):00000000

R12 (ip):00000000

R13 (sp):00005400

R14 (lr):00000000

R15 (pc):00011400

CPSR Register

Negative (N) :0

Zero (Z) :0

Carry (C) :0

Overflow (V) :0

IRQ Disable:1

FIQ Disable:1

Thumb (T) :0

CPU Mode :Svsystem

.text

00001000:E59F0010 Ldr R0,=a

00001004:E59F3010 LDR R3,=b

00001008:E3A04009 Mov r4,#9

0000100C:E5905000 Ldr R5,[r0]

00001010:E0846005 Add R6, R4, R5

00001014:E5836000 Str R6, [r3]

.data

00001020: a: .word 10

00001024: b: .word 0

MemoryView0

00001024

Word Size

8Bit

16Bit

32Bit

00001024 13 00 00 00 81 81 81 81 81 81 81 81 81 81 81 81 81 81 81 81

0000103A 81

00001050 81

00001066 81

OutputView

Console Stdin/Stdout/Stderr

Microprocessor & Computer Architecture (μpCA)

Example 2: C=A+B (Half Word)

```
.text
LDR R0, =a
LDR R1, =b
LDR R3, =c
LDRH R4, [R1]
LDRH R5, [R0]
ADD R6, R4, R5
STRH R6, [R3]
.data
a: .hword 10
b: .hword 20
c: .hword 0
```

```
.text
00001000:E59F0014 Ldr R0, =a
00001004:E59F1014 LDR R1, =b
00001008:E59F3014 LDR R3, =c
0000100C:E01140B0 LDRH R4, [r1]
00001010:E01050B0 LDRH R5, [r0]
00001014:E0846005 Add R6, R4, R5
00001018:E00360B0 STRH R6, [r3]
.data
00001028: a: .hword 10
0000102A: b: .hword 20
0000102C: c: .hword 0
```

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00001028
R1	: 0000102a
R2	: 00000000
R3	: 0000102c
R4	: 00000014
R5	: 0000000a
R6	: 0000001e
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 0000101c

MemoryView1		102C
00001028	0A 00 14 00 1E 00 00 00	

Microprocessor & Computer Architecture (μpCA)

Example 3: String (BYTE)

```
.text
Ldr      R0, =mystring
Swi 0x02
Swi 0x11
.data
mystring:.asciz "HELLO PESU"
```

jstring.s

```
.text
00001000:E59F0004    Ldr      R0, =mystring
00001004:EF000002    Swi 0x02
00001008:EF000011    Swi 0x11
.data
00001010:    mystring:.asciz "HELLO PESU"
```

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00001010
R1	: 00000000
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001008

MemoryView1

00001010 48 45 4C 4C 4F 20 50 45 53 55 00 00 81
00001027 81 81 81 81 81 81 81 81 81 81 81 81 81

OutputView

Console Stdin/Stdout/Stderr

HELLO PESU

Addressing or Indexing

- Pre Indexing Without Write Back
- **Syntax:** LDR Rd, [Rn,OFFSET]
- Pre Indexing With Write Back

Syntax: LDR Rd, [Rn,OFFSET]!

- Poste Indexing

Syntax: LDR Rd, [Rn] ,OFFSET



THANK YOU

Dr. D. C. Kiran

Department of Computer Science and Engineering

dckiran@pes.edu

9829935135



Microprocessor & Computer Architecture (μ pCA)

UE19CS252

Dr. D. C. Kiran

Department of
Computer Science and Engineering

Microprocessor & Computer Architecture (μ pCA)

Data Transfer Instructions

Dr. D. C. Kiran

Department of Computer Science and Engineering

Microprocessor & Computer Architecture (μpCA)

Syllabus



Unit 1: Basic Processor Architecture and Design

- ~~Microprocessor Overview~~
- ~~CISC VS RISC~~
- ~~Introduction to ARM Processor & Applications~~
- ~~ARM Architecture Overview~~
- ~~Different ARM processor Modes~~
- ~~Register Bank~~
- ~~ARM Program structure~~
- ~~ARM Instruction Format~~
- **ARM INSTRUCTION SET**

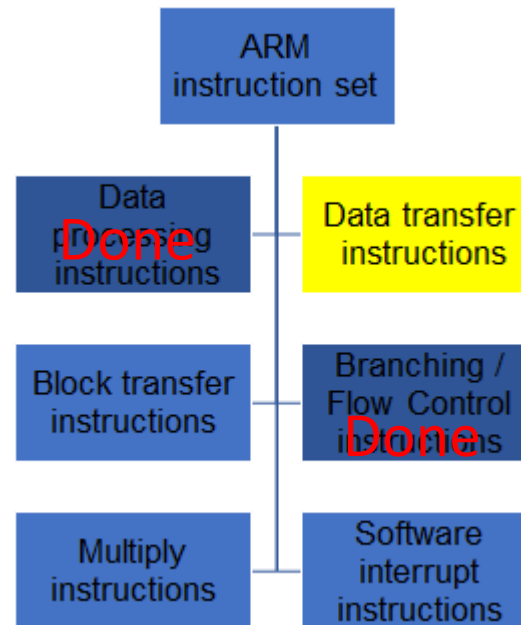
~~Data Processing Instructions~~

~~Flow Control Instructions~~

Data Transfer Instructions

~~LDR & STR~~

Indexing



- Pre Indexing Without Write Back
- **Syntax:** LDR Rd, [Rn,OFFSET]
- Pre Indexing With Write Back

Syntax: LDR Rd, [Rn,OFFSET]!

- Post Indexing

Syntax: LDR Rd, [Rn],OFFSET

- Memory is addressed by a register and **an offset**.

```
LDR  R0 , [R1]  @ mem[R1]
```

- Three ways to specify offsets:

- **Immediate**

```
LDR  R0 , [R1 , #4]      @ mem[R1+4]
```

- **Register**

```
LDR  R0 , [R1 , R2]      @ mem[R1+R2]
```

- **Scaled Register**

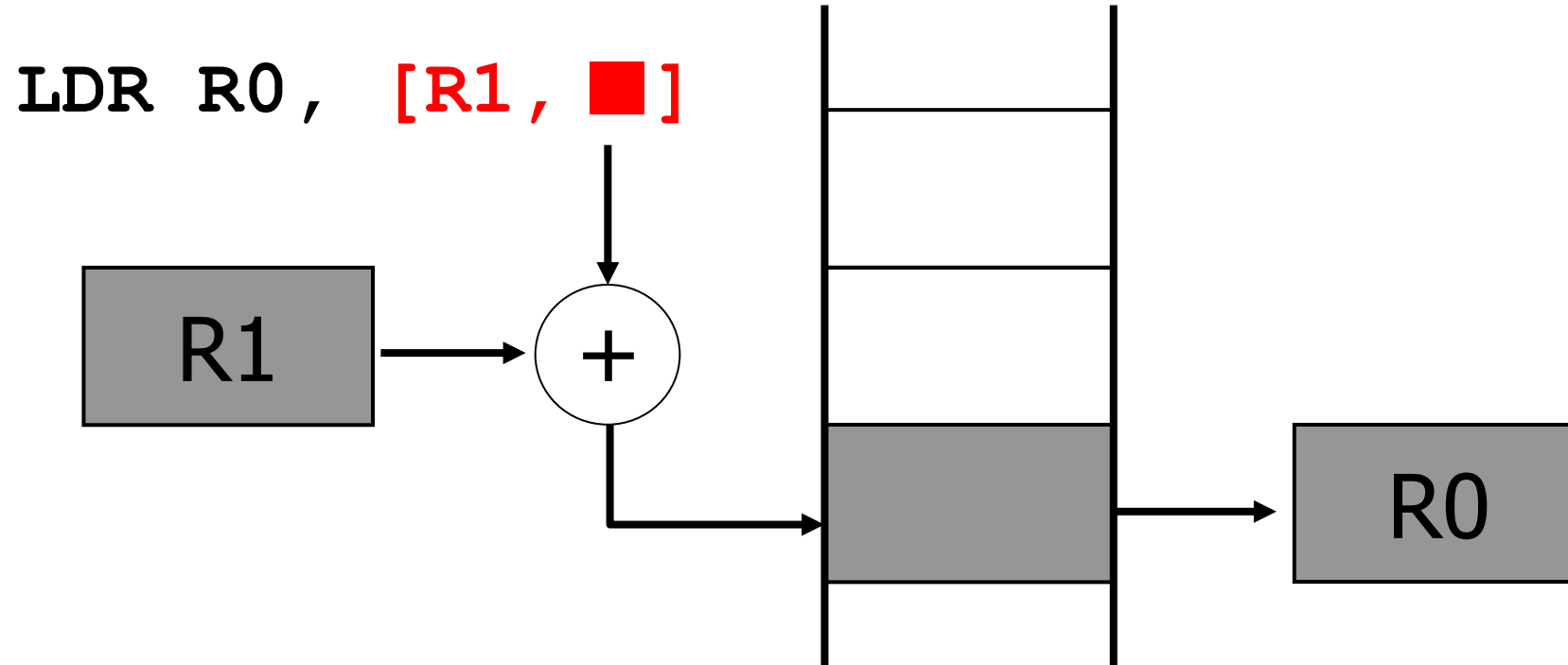
```
LDR  R0 , [R1 , R2 , LSL #2] @ mem[R1+4*R2]
```

- Pre-index addressing (`LDR R0, [R1, #4]`)
without a writeback.
- Auto-indexing addressing (`LDR R0, [R1, #4]!`)
Pre-index with writeback
calculation before accessing with a writeback
- Post-index addressing (`LDR R0, [R1], #4`)
calculation after accessing with a writeback

Microprocessor & Computer Architecture (μpCA)

Pre-index addressing

LDR R0, [R1, #4] @ R0=mem[R1+4]
 @ R1 unchanged



Microprocessor & Computer Architecture (μpCA)

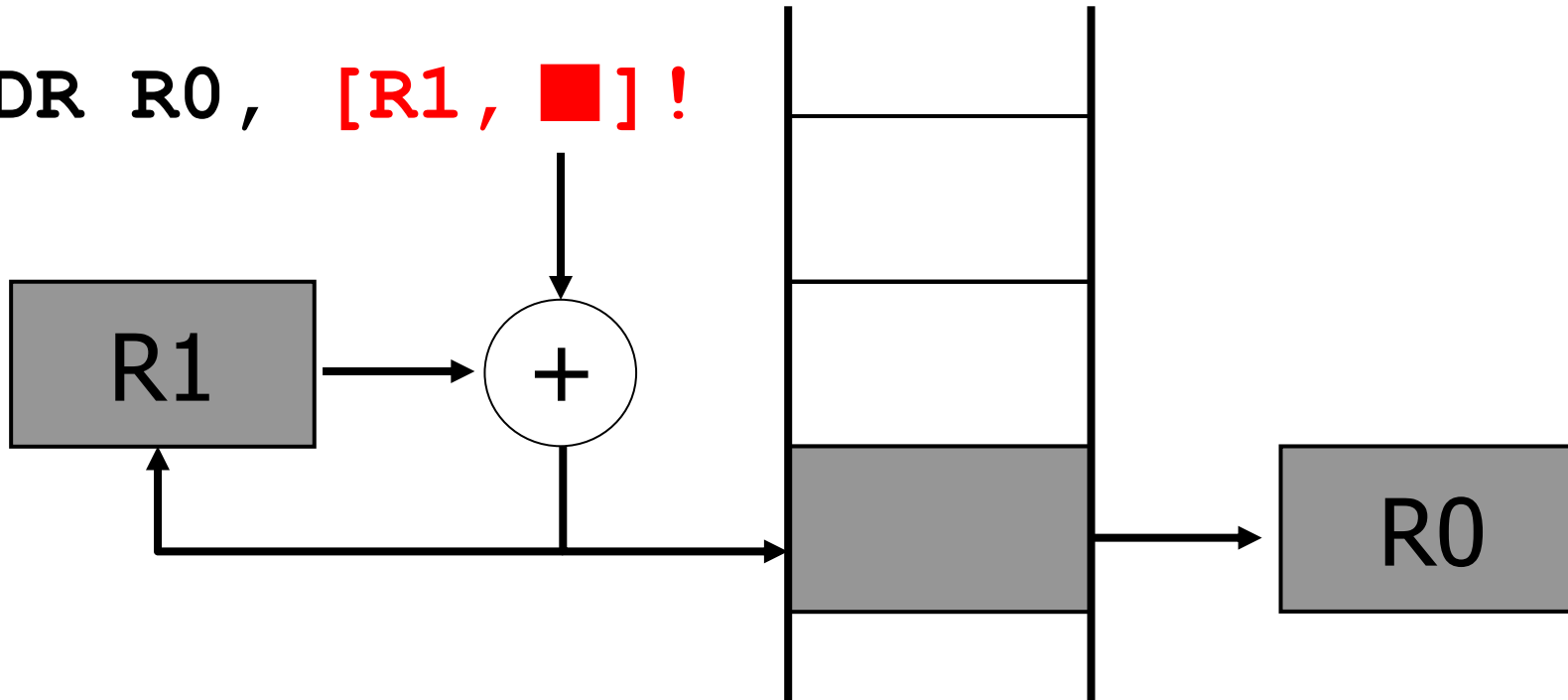
Auto-indexing addressing

LDR R0, [R1, #4]! @ R0=mem[R1+4]

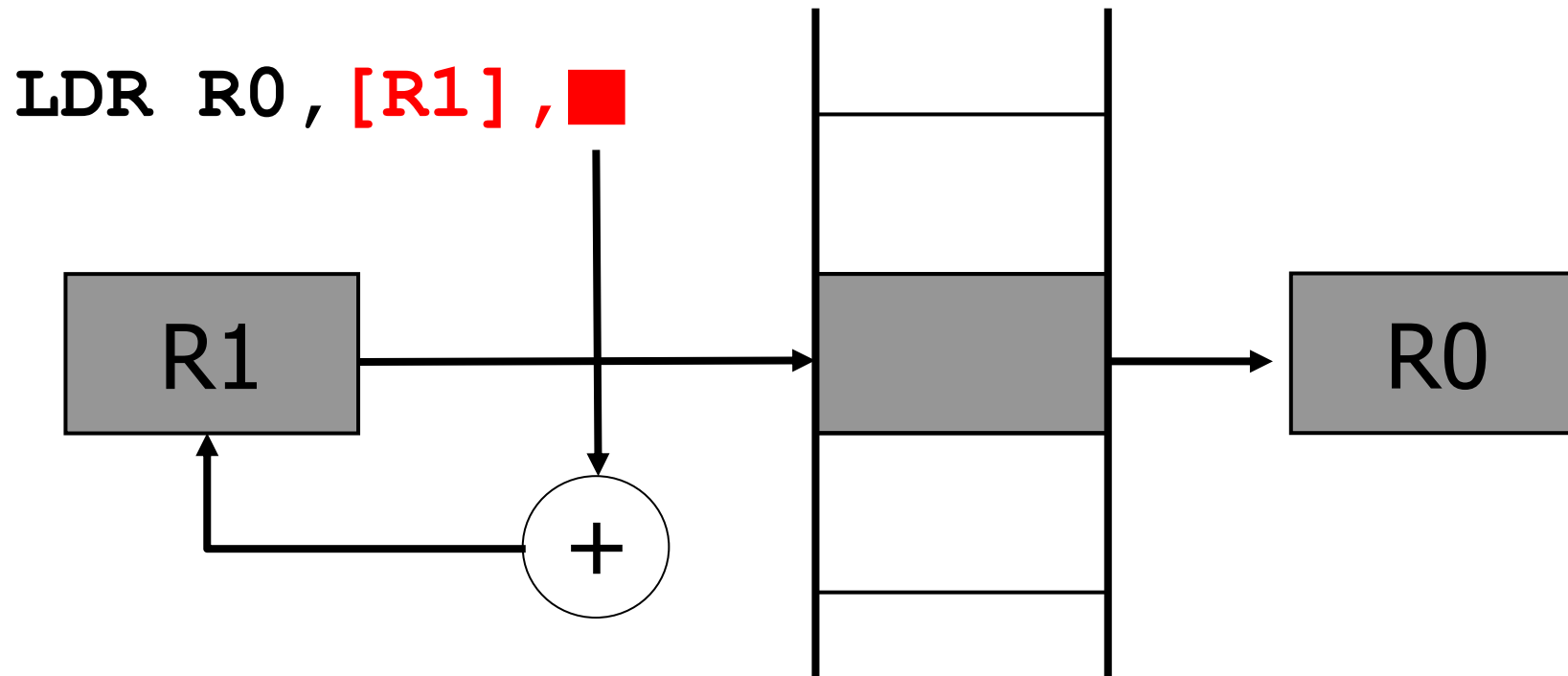
@ R1=R1+4

No extra time; Fast;

LDR R0, [R1, ■]!



LDR R0, [R1], #4 @ R0=mem[R1]
 @ R1=R1+4



Microprocessor & Computer Architecture (μpCA)

Pre-indexed addressing without Write Back or Auto Indexing

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:00000014
R1	:0000100C
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001008

```
.text
00001000:E59F1000  LDR R1,=A
00001004:E5910004  LDR R0,[R1,#4]
.data
0000100C:          A: .word 10,20,30,40
```

R0 get the 0x100C+4th Value

R1 is not Updated

MemoryView1				
0000100C				
0000100C	0000000A	00000014	0000001E	00000028
00001030	81818181	81818181	81818181	81818181
00001054	81818181	81818181	81818181	81818181

Microprocessor & Computer Architecture (μpCA)

Pre-indexed addressing with Write Back or Auto Indexing-2

```
.text
00001000:E59F1000    LDR R1,=A
00001004:E5B10004    LDR R0,[R1,#4]!
.data
0000100C:
A: .word 10,20,30,40
```

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:00000014
R1	:0001010
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001008

R0 get the 0x100C+4th i.e 0x1010th Value

R1 is Updated to 0x1010

MemoryView1				
0000100C				
0000100C	0000000A	00000014	0000001E	00000028
00001030	81818181	81818181	81818181	81818181

Microprocessor & Computer Architecture (μpCA)

Post-index addressing:2

```
.text
00001000:E59F1000  LDR R1,=A
00001004:E4910004  LDR R0,[R1],#4
.data
0000100C:          A: .word 10,20,30,40
```

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:0000000a
R1	:00001010
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001008

R0 get the 0x100Cth Value

R1 is Updated to 0x1010

MemoryView1				
0000100C				
0000100C	0000000A	00000014	0000001E	00000028
00001030	81818181	81818181	81818181	81818181
00001054	81818181	81818181	81818181	81818181

Block Transfer

- **LDRM**
- **STRM**



THANK YOU

Dr. D. C. Kiran

Department of Computer Science and Engineering

dckiran@pes.edu

9829935135



Microprocessor & Computer Architecture (μ pCA)

Multiple Load / Store Instructions or Block Transfer

Session - 7

MULTIPLE REGISTER LOAD/STORE

- Transfer a block of data more efficiently.
- Used for procedure entry and exit for saving and restoring workspace registers and the return address
- For ARM7, $2+Nt$ cycles
(N : #words, t : time for a word for sequential access).
- Increase interrupt latency since it can't be interrupted.
- Registers are considered in increasing order always.

```
LDMIA  R1, {R0, R2, R5}    @ R0 = mem[R1]
                                @ R2 = mem[R1+4]
                                @ R5 = mem[R1+8]
```

MICROPROCESSOR AND COMPUTER ARCHITECTURE

MULTIPLE REGISTER LOAD/STORE

Syntax: <LDM|STM>{<cond>}<addressing mode> $Rn\{!\}$, <registers>{^}

Addressing
mode

Description

Start address

End address

$Rn!$

IA	increment after	Rn	$Rn + 4 * N - 4$	$Rn + 4 * N$
IB	increment before	$Rn + 4$	$Rn + 4 * N$	$Rn + 4 * N$
DA	decrement after	$Rn - 4 * N + 4$	Rn	$Rn - 4 * N$
DB	decrement before	$Rn - 4 * N$	$Rn - 4$	$Rn - 4 * N$

LDM<mode> Rn, {<registers>}

IA: addr:=Rn

IB: addr:=Rn+4

DA: addr:=Rn-#<registers>*4+4

DB: addr:=Rn-#<registers>*4

For each Ri in <registers>

IB: addr:=addr+4

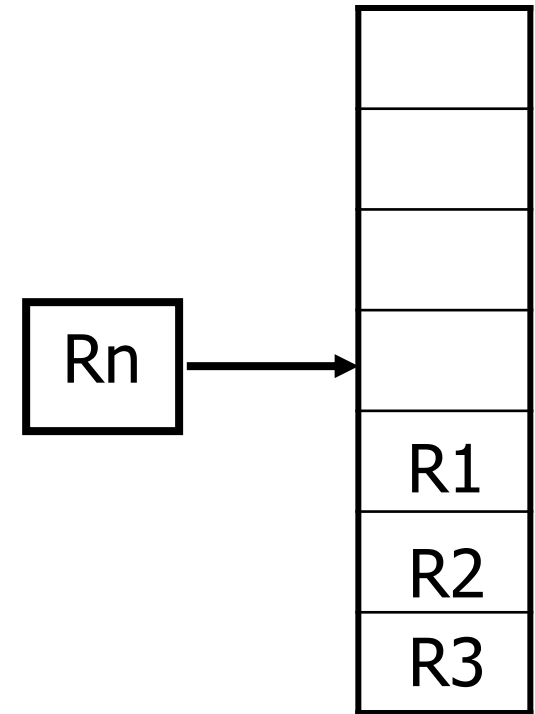
DB: addr:=addr-4

Ri:=M[addr]

IA: addr:=addr+4

DA: addr:=addr-4

<!:>: Rn:=addr



MULTIPLE LOAD/STORE REGISTER

LDMIA R0, {R1, R2, R3}

or

LDMIA R0, {R1-R3}

R1: 10

R2: 20

R3: 30

R0: 0x10



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

- Program to transfer data from memory locations to registers using single **Load Multiple Instruction using IA condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800

B: .WORD 0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R7,=B

LDMIA R0!,{R1,R8,R9,R11}

.END

MULTIPLE REGISTER **STORE**- EXAMPLE1 CONTD.,

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction** using **IA condition**
.DATA

A: .WORD 100,200,300,400,500,600,700,800

B: .WORD 0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R7,=B

LDmia R0!,{R1,R8,R9,R11}

STmia R7!,{R8,R9,R11,R1}

.END

MICROPROCESSOR AND COMPUTER ARCHITECTURE

MULTIPLE REGISTER LOAD/STORE

LDM<mode> Rn, {<registers>}

IA: addr:=Rn

IB: addr:=Rn+4

DA: addr:=Rn-#<registers>*4+4

DB: addr:=Rn-#<registers>*4

For each Ri in <registers>

IB: addr:=addr+4

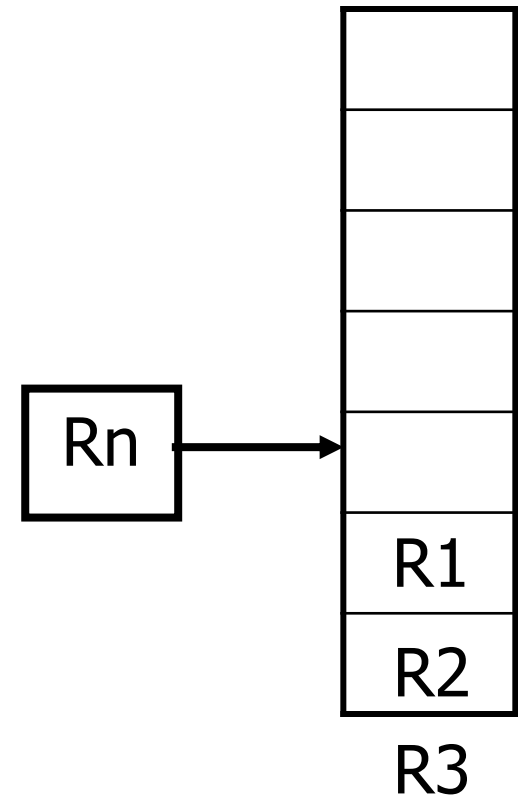
DB: addr:=addr-4

Ri:=M[addr]

IA: addr:=addr+4

DA: addr:=addr-4

<!:>: Rn:=addr



MULTIPLE LOAD/STORE REGISTER

LDMIB R0! , {R1,R2,R3}

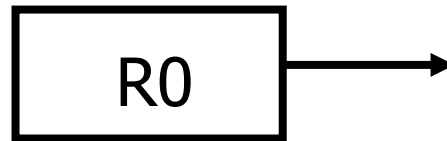
R1 : 20

R2 : 30

R3 : 40

R0 : 0x01C

R0

A rectangular box labeled 'R0' has an arrow pointing from its right side to the first row of a table, specifically to the 'addr' column.

addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD / STORE- EXAMPLE2

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction using IB condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R11,=B

LDMIB R0!,{R1-R10}

STMIB R11!,{R1-R10}

.END

MICROPROCESSOR AND COMPUTER ARCHITECTURE

MULTIPLE REGISTER LOAD/STORE

LDM<mode> Rn, {<registers>}

IA: $\text{addr} := \text{Rn}$

IB: $\text{addr} := \text{Rn} + 4$

DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$

DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$

For each R_i in <registers>

IB: $\text{addr} := \text{addr} + 4$

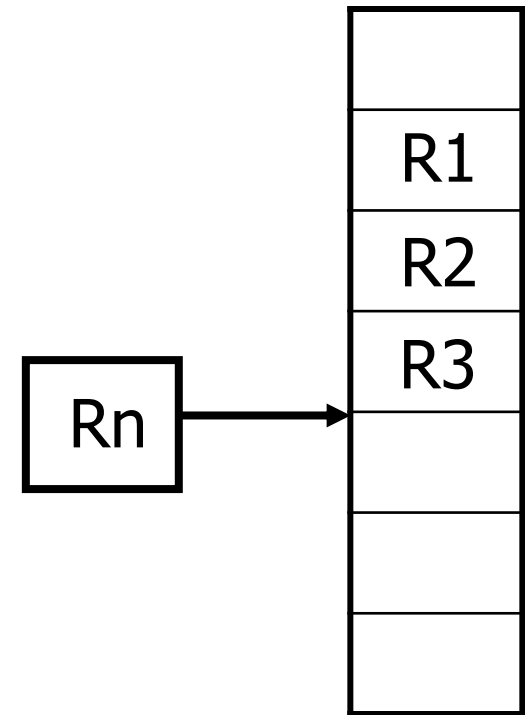
DB: $\text{addr} := \text{addr} - 4$

$R_i := M[\text{addr}]$

IA: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

<!:>: $\text{Rn} := \text{addr}$



MULTIPLE LOAD/STORE REGISTER

LDM**DA** R0! , {R1 , R2 , R3}

R1 : 40

R2 : 50

R3 : 60

R0 : 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD / STORE- EXAMPLE2

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction** using **DA condition**

..DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

ADD R0,R0,#36 ; ADD 36 TO POINT TO THE LOCATION STARTING AT A

LDR R11,=B

LDMDA R0!,{R1-R10}

STMDA R11!,{R1-R10}

.END

MULTIPLE LOAD/STORE REGISTER

LDM<mode> Rn, {<registers>}

IA: $\text{addr} := \text{Rn}$

IB: $\text{addr} := \text{Rn} + 4$

DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$

DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$

For each Ri in <registers>

IB: $\text{addr} := \text{addr} + 4$

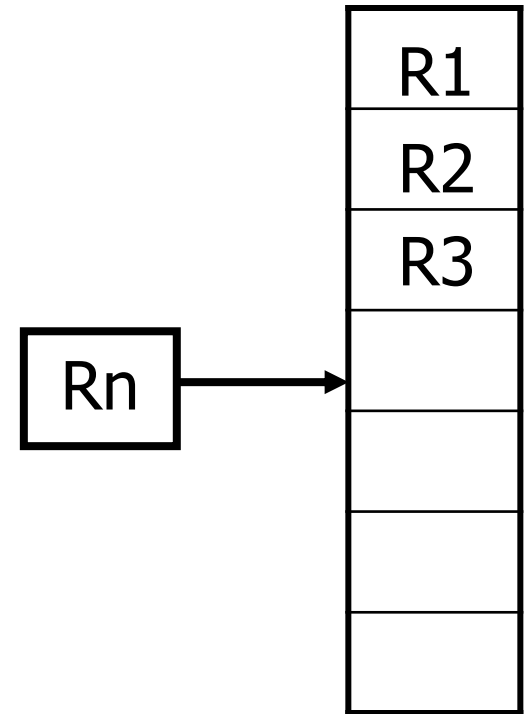
DB: $\text{addr} := \text{addr} - 4$

$\text{Ri} := \text{M}[\text{addr}]$

IA: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

<!:>: $\text{Rn} := \text{addr}$



Multiple load/store register

LDMDB R0!, {R1,R2,R3}

R1: 30

R2: 40

R3: 50

R0: 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD / STORE- EXAMPLE3

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction using DB condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

ADD R0,R0,#40

LDR R11,=B

ADD R11,R11,#-4

LDMDB R0!,{R1-R10}

STMDB R11!,{R1-R10}

.END

MULTIPLE REGISTER LOAD / STORE- EXAMPLE 4

PRE mem32[0x80018] = 0x03
 mem32[0x80014] = 0x02

 mem32[0x80010] = 0x01

 r0 = 0x00080010

 r1 = 0x00000000

 r2 = 0x00000000

 r3 = 0x00000000

 LDMIA r0!, {r1-r3}

POST r0 = 0x0008001c

 r1 = 0x00000001

 r2 = 0x00000002

 r3 = 0x00000003

MULTIPLE REGISTER LOAD / STORE- EXAMPLE 4

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIA $r0!$, { $r1-r3$ }

Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000003$
	0x80014	0x00000002	$r2 = 0x00000002$
	0x80010	0x00000001	$r1 = 0x00000001$
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD / STORE- EXAMPLE 5

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIB $r0!$, { $r1-r3$ }

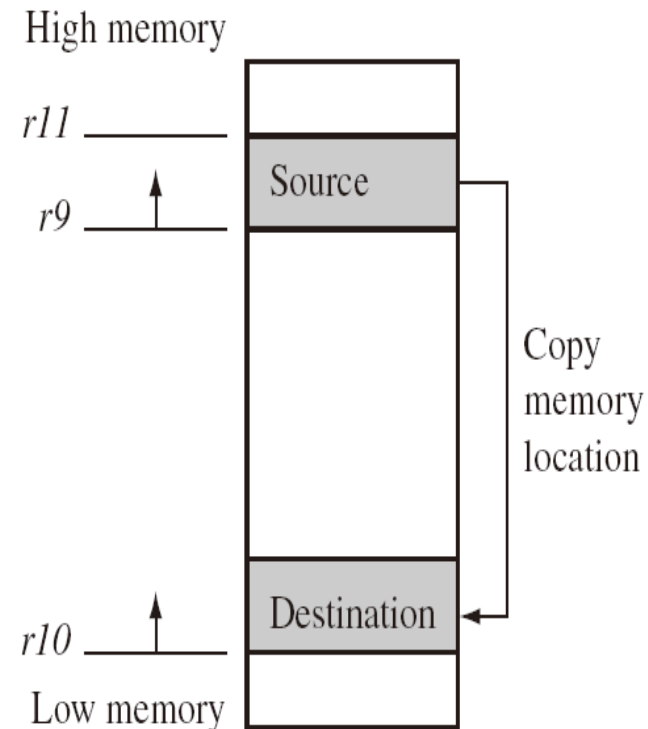
Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	$r3 = 0x00000004$
	0x80018	0x00000003	$r2 = 0x00000003$
	0x80014	0x00000002	$r1 = 0x00000002$
	0x80010	0x00000001	
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD / STORE- APPLICATION

Write a program using ARM7TDMI - ISA to copy a block 512 bytes of data from location A to location B if the rate of data transfer rate is 32 bytes.

- R9: address of the source A
- R10: address of the destination B
- R11: count : 16

```
loop: LDMIA R9!, {R0-R7}
      STMIA R10!, {R0-R7}
      CMP R11,#16
      BNE  loop
```





Microprocessor & Computer Architecture (μ pCA)

Block Transfer Instructions: Stack Operations

The Memory access can be in LIFO order.

i.e Can be treated like STACK.

R13 is a stack pointer which will keep the address of TOP of the STACK.

Mainly used in Procedural Call.

Stack can grow upward or downward direction based on the MODE used by the user in the program.

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	30
0x1024	40
0x1028	50
0x102C	60
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (Stack)

10 20 10 140 15 60 27

R13=0x1018

SP



Addr	data
0x1010	
0x1014	
0x1018	
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (Stack)

10 20 75 140 15 60 27

R13=0x101C

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	

Block Transfer Instructions (Stack)

10 20 75 140 15 60 27

R13=0x1020

SP →

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	

Block Transfer Instructions (Stack)

10 20 75 140 15 60 27

R13=0x1020

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	

Block Transfer Instructions (Stack)

10 20 75 140 15 60 27

R13=0x1024

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	140
SP → 0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions (Stack)

Syntax:

<LDM/STM> <Addressing Mode> R13 {!}, Registers

Addressing Mode	=LDM	=STM
Full ascending (FA)	LDMDA	STMIB
Full descending (FD)	LDMIA	STMDB
Empty ascending (EA)	LDMDB	STMIA
Empty descending (ED)	LDMIB	STMDA

- **STM** is used to **PUSH** on to the STACK
- **LDM** is used to **POP** from the STACK

Block Transfer Instructions (Stack)

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 00000000
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001000

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
.end
```

Initially Stack is Empty

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 81818181
00005404	: 81818181
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

Block Transfer Instructions (Stack)

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005408
R14 (lr)	: 00000000
R15 (pc)	: 00011400

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
.end
```

PUSH

StackView	
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181
00005440	: 81818181
00005444	: 81818181

Block Transfer Instructions (Stack)

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
LDMEA R13!,{R5,R6}
.end
```

POP

Again the Stack is Empty

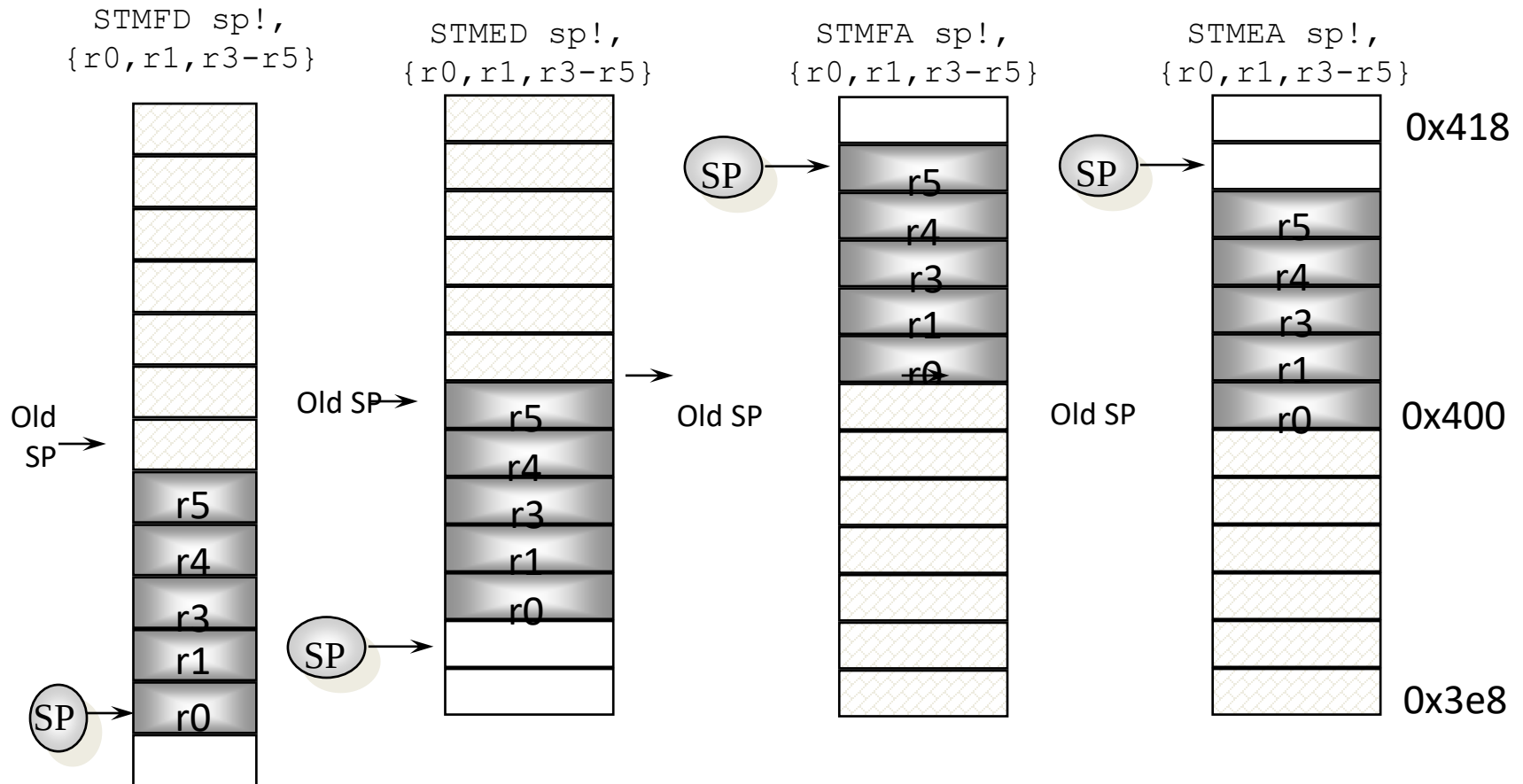
RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000004
R6	: 00000005
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001010

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

Stack Examples





THANK YOU



Team MPCA

Department of Computer Science
and Engineering



Microprocessor & Computer Architecture (μ pCA)

PSR & SWAP Instructions

Session - 8.2

Move to Register from Status Register (MRS): Read from either CPSR or SPSR

Example:

```
MRS R0, CPSR
```

```
MRS R1, SPSR
```

Move to Status Register from Register (MSR):

Example:

```
MSR CPSR_field, R0
```

```
MSR SPSR_field, R1
```

_field

_c: Control Field (0:7)

_f: Flag Field(24:31)

_x: Extension (8:15)

_s: Status (16:23)

Microprocessor & Computer Architecture (μpCA)

PSR Instructions

.text

MOVS R1,#-10

MRS R0,CPSR ; Take a copy of the CPSR.

AND R0,R0,#0000 ; Clear the mode bits.

MSR CPSR_F,R0 ; Write back the modified CPSR.

.end

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001004

CPSR Register	
Negative (N)	: 1
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

Microprocessor & Computer Architecture (μpCA)

PSR Instructions

.text

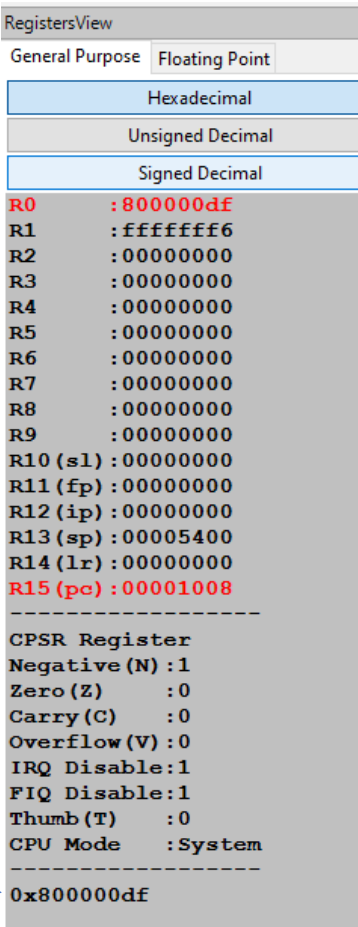
MOVS R1,#-10

MRS R0,CPSR ; Take a copy of the CPSR.

AND R0,R0,#0000 ; Clear the mode bits.

MSR CPSR_F,R0 ; Write back the modified CPSR.

.end



RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	: 800000df
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001008

CPSR Register

Negative (N) : 1

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable : 1

FIQ Disable : 1

Thumb (T) : 0

CPU Mode : System

0x800000df

Two blue arrows point from the assembly code to the register view. The first arrow points from the **MRS R0,CPSR** instruction to the R0 register value (800000df). The second arrow points from the **MSR CPSR_F,R0** instruction to the final CPSR value (0x800000df).

Microprocessor & Computer Architecture (μpCA)

PSR Instructions

.text

MOVS R1,#-10

MRS R0,CPSR ; Take a copy of the CPSR.

AND R0,R0,#0000 ; Clear the mode bits.

MSR CPSR_F,R0 ; Write back the modified CPSR.

.end



RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001000

CPSR Register	
Negative (N)	: 1
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x800000df	

Microprocessor & Computer Architecture (μpCA)

PSR Instructions

.text

MOVS R1,#-10

MRS R0,CPSR ; Take a copy of the CPSR.

AND R0,R0,#0000 ; Clear the mode bits.

MSR CPSR_F,R0 ; Write back the modified CPSR.

.end



RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:00000000
R1	:ffffff6
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001010

CPSR Register	
Negative (N)	:0
Zero (Z)	:0
Carry (C)	:0
Overflow (V)	:0
IRQ Disable	:1
FIQ Disable	:1
Thumb (T)	:0
CPU Mode	:System

0x000000df	

Microprocessor & Computer Architecture (μpCA)

SWAP :A & R1

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 5
R2	: 0
R3	: 0
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4100

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:          a:.word 10
```

MemoryView1

00001018

00001018 000A 0000 8181

Microprocessor & Computer Architecture (μpCA)

SWAP :A & R1

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 5
R2	: 4120
R3	: 10
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4112

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:          a:.word 10
```

MemoryView1	
00001018	
00001018	0005 0000 81
0000103C	8181 8181 81

Microprocessor & Computer Architecture (μpCA)

SWAP :A & R1

RegistersView	
General Purpose	Step Over Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 10
R2	: 4120
R3	: 10
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4116

CPSR	Register

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:                a:.word 10
```

MemoryView1	
00001018	
00001018	0005 0000 8

SWAP instruction

SWP <Swap Destination>, <Original>, [<address>]

.text

LDR R2, =a

MOV R1, #5

SWP R1, R1, [R2]

.data

a:.word 10

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 10
R2	: 4112
R3	: 0
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 70656

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 10
R2	: 4112
R3	: 0
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 70656

After Swapping

MemoryView1	
00001010	0005 0000 8181 8



THANK YOU



Team MPCA

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

Procedure Call or Subroutine Call or Function Call

Session - 8

Microprocessor & Computer Architecture (μpCA)

Procedure Call

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	0x0010
CPSR	

Main Procedure

0x0000 Instruction 1
0x0004 Instruction 2
0x0008 Instruction 3
0x000C Procedure Call
0x0010 Instruction 4
0x0014 Instruction 5

 Instruction Last

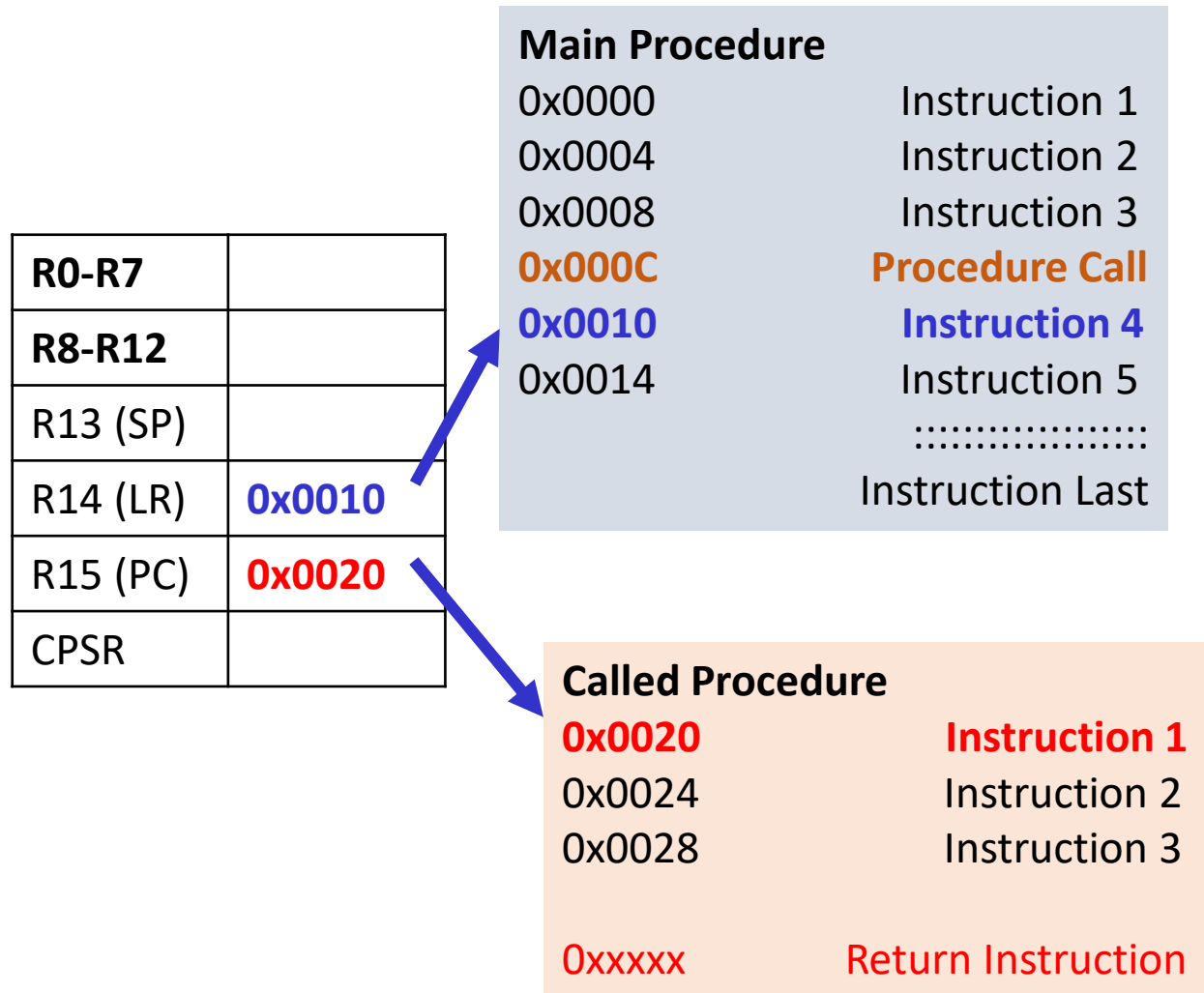
Called Procedure

0x0020 Instruction 1
0x0024 Instruction 2
0x0028 Instruction 3

0xxxxx Return Instruction

Microprocessor & Computer Architecture (μpCA)

General Structure of Procedure Call:



LR=PC

PC= Address of the 1st Instruction

Microprocessor & Computer Architecture (μpCA)

General Structure of Procedure Return

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	0x0010
CPSR	

Main Procedure

0x0000 Instruction 1
0x0004 Instruction 2
0x0008 Instruction 3
0x000C **Procedure Call**
0x0010 **Instruction 4**
0x0014 Instruction 5

 Instruction Last

Called Procedure

0x0020 **Instruction 1**
0x0024 Instruction 2
0x0028 Instruction 3

0xxxxx **Return Instruction**

PC=LR

Microprocessor & Computer Architecture (μpCA)

General Structure of Procedure Call & Return

Main Procedure

0x0000	Instruction 1
0x0004	Instruction 2
0x0008	Instruction 3
0x000C	BL Procedure
0x0010	Instruction 4
0x0014	Instruction 5
	
	Instruction Last

Called Procedure

0x0020	Procedure: Instruction 1
0x0024	Instruction 2
0x0028	Instruction 3
0xxxxx	MOV PC LR
	or
	BX LR

Procedure Call : Example 1

```
MOV R1, #3
BL FOOO
ADD R2, R0, R1
SWI 0x11
```

FOOO:

```
MOV R0, #2
BX LR
```

```
MOV R1, #3
BL FOOO
ADD R2, R0, R1
SWI 0x11
```

FOOO:

```
MOV R0, #2
MOV PC, LR
```

Microprocessor & Computer Architecture (μpCA)

Procedure Call : Example 1

RegistersView

General Purpose

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:00000000
R1	:00000000
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00000000
R15 (pc)	:00001000

jeven.s

```
00001000:E3A01003    main:  mov r1, #3
00001004:EB000001          bl foo
00001008:E0802001          add r2, r0, r1
0000100C:EF000011          swi 0x11
00001010:                foo:
00001010:E3A00002          mov r0, #2
00001014:E1A0F00E          mov pc, lr
```

Microprocessor & Computer Architecture (μpCA)

Procedure Call : Example 1

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:00000000
R1	:00000003
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00001008
R15 (pc)	:00001010

jeven.s

```
00001000:E3A01003    main:  mov r1, #3
00001004:EB000001          bl foo
00001008:E0802001          add r2, r0, r1
0000100C:EF000011          swi 0x11
00001010:                foo:
00001010:E3A00002          mov r0, #2
00001014:E1A0F00E          mov pc, lr
```

Microprocessor & Computer Architecture (μpCA)

Procedure Call : Example 1

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:00000002
R1	:00000003
R2	:00000000
R3	:00000000
R4	:00000000
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00001008
R15 (pc)	:00001008

jeven.s

```
00001000:E3A01003    main:  mov r1, #3
00001004:EB000001      bl foo
00001008:E0802001      add r2, r0, r1
0000100C:EF000011      swi 0x11
00001010:                foo:
00001010:E3A00002      mov r0, #2
00001014:E1A0F00E      mov pc, lr
```

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK



```
LDR  R6, =A
MOV  R1, #25           ; parameter1
MOV  R2, #25           ; parameter2
STMFD R13!, { R1, R2}  ; parameters are PUSHed on stack.
BL   LINK
STR  R0, [R6]         ; return value in Reg. R0.
SWI  0x11
```

```
LINK: LDMFD R13!, { R4, R5} ; parameters are POPed from the stack
      ADD R0, R4, R5       ; Result is in register R0.
      MOV PC, LR
```

```
A: .WORD 0
```

```
.END
```

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK

General Purpose

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 00000000

R1 : 00000000

R2 : 00000000

R3 : 00000000

R4 : 00000000

R5 : 00000000

R6 : 00000000

R7 : 00000000

R8 : 00000000

R9 : 00000000

R10 (s1) : 00000000

R11 (fp) : 00000000

R12 (ip) : 00000000

R13 (sp) : 00005400

R14 (lr) : 00000000

R15 (pc) : 00001000

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

FIQ Disable: 1

Thumb (T) : 0

CPU Mode : System

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:81818181

000053FC:81818181

00005400:81818181

00005404:81818181

00005408:81818181

0000540C:81818181

00005410:81818181

00005414:81818181

00005418:81818181

0000541C:81818181

OutputView

Console Stdin/Stdout/S

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

Register	Value
R0	:00000000
R1	:00000019
R2	:00000019
R3	:00000000
R4	:00001028
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:000053f8
R14 (lr)	:00000000
R15 (pc)	:00001010

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

FIQ Disable: 1

Thumb (T) : 0

jeven.s

```
.text
00001000:E59F4024    LDR    R4, =A
00001004:E3A01019    MOV    R1, #25
00001008:E3A02019    MOV    R2, #25
0000100C:E92D0006    STMFD  R13!, { R1, R2}
00001010:EB000001    BL     LINK
00001014:E5840000    STR    R0, [R4]
00001018:EF000011    SWI    0x11

0000101C:E8BD0030    LINK:  LDMFD R13!, { R4, R5}
00001020:E0840005    ADD    R0, R4, R5
00001024:E1A0F00E    MOV    PC, LR

00001028:00000000    A:     .WORD 0

0000102C:00001028    .END
```

StackView

Address	Value
000053D8	:81818181
000053DC	:81818181
000053E0	:81818181
000053E4	:81818181
000053E8	:81818181
000053EC	:81818181
000053F0	:81818181
000053F4	:81818181
000053F8	:00000019
000053FC	:00000019
00005400	:81818181
00005404	:81818181
00005408	:81818181
0000540C	:81818181
00005410	:81818181

OutputView

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	: 00000000
R1	: 00000019
R2	: 00000019
R3	: 00000000
R4	: 00001028
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 000053f8
R14 (lr)	: 00001014
R15 (pc)	: 0000101c

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

jeven.s

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053D8:81818181

000053DC:81818181

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:00000019

000053FC:00000019

00005400:81818181

00005404:81818181

00005408:81818181

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	: 00000000
R1	: 00000019
R2	: 00000019
R3	: 00000000
R4	: 00000019
R5	: 00000019
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00001014
R15 (pc)	: 00001020

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

jeven.s

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:00000019

000053FC:00000019

00005400:81818181

00005404:81818181

00005408:81818181

Microprocessor & Computer Architecture (μpCA)

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:00000032
R1	:00000019
R2	:00000019
R3	:00000000
R4	:00000019
R5	:00000019
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00001014
R15 (pc)	:00001014

CPSR Register

Negative (N) :0

Zero (Z) :0

Carry (C) :0

Overflow (V) :0

IRQ Disable:1

jeven.s

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:00000019

000053FC:00000019

00005400:81818181

00005404:81818181

00005408:81818181

0000540C:81818181

00005410:81818181

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)



.TEXT ; MAIN Procedure

LDR R4, =A

MOV R1, #11

MOV R2, #10

MOV R3, #02

STMFD R13!, {R1,R2,R3}

BL ADDFun ; Call to ADD Procedure

STR R0, [R4]

SWI 0x11

.DATA

A: .WORD 0

ADDFun: LDMFD R13!, { R4, R5,R6}

ADD R0, R4, R5

STMFD R13!, {R0,R6,LR}

BL MULFun

LDMFD R13!, { R4, R5,LR}

MOV PC, LR

; ADD Procedure

; Call to MUL Procedure

; Return to Main Procedure

MULFun: MUL R0, R4, R5

MOV PC, LR

; Return to ADD Procedure

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

R0 : 00000000
R1 : 0000000b
R2 : 0000000a
R3 : 00000002
R4 : 00001044
R5 : 00000000
R6 : 00000000
R7 : 00000000
R8 : 00000000
R9 : 00000000
R10 (s1) : 00000000
R11 (fp) : 00000000
R12 (ip) : 00000000
R13 (sp) : 00005400
R14 (lr) : 00000000
R15 (pc) : 00001010

```
.TEXT
00001000:E59F4038  LDR R4, =A
00001004:E3A0100B  MOV R1, #11
00001008:E3A0200A  MOV R2, #10
0000100C:E3A03002  MOV R3, #02
00001010:E92D000E  STMFD R13!, {R1,R2,R3}
00001014:EB000001  BL ADDFun
00001018:E5840000  STR R0, [R4]
0000101C:EF000011  SWI 0x11
00001020:E8BD0070  ADDFun: LDMFD R13!, { R4, R5,R6}
00001024:E0840005          ADD R0, R4, R5
00001028:E92D4041          STMFD R13!, {R0,R6,LR}
0000102C:EB000001          BL MULFun
00001030:E8BD4030          LDMFD R13!, { R4, R5,LR}
00001034:E1A0F00E          MOV PC, LR
00001038:E0000594  MULFun:  MUL R0, R4, R5
0000103C:E1A0F00E          MOV PC, LR

.DATA
00001044:      A:      .WORD 0
```

000053D4:81818181
000053D8:81818181
000053DC:81818181
000053E0:81818181
000053E4:81818181
000053E8:81818181
000053EC:81818181
000053F0:81818181
000053F4:81818181
000053F8:81818181
000053FC:81818181
00005400:81818181
00005404:81818181
00005408:81818181
0000540C:81818181
00005410:81818181
00005414:81818181
00005418:81818181
0000541C:81818181
00005420:81818181
00005424:81818181
00005428:81818181

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 00001044
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005f4
R14 (lr)	: 00001018
R15 (pc)	: 00001020

```
.TEXT
00001000:E59F4038    LDR R4, =A
00001004:E3A0100B    MOV R1, #11
00001008:E3A0200A    MOV R2, #10
0000100C:E3A03002    MOV R3, #02
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL ADDFun
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDFun: LDMFD R13!, { R4, R5,R6}
00001024:E0840005                ADD R0, R4, R5
00001028:E92D4041                STMFD R13!, {R0,R6,LR}
0000102C:EB000001                BL MULFun
00001030:E8BD4030                LDMFD R13!, { R4, R5,LR}
00001034:E1A0F00E                MOV PC, LR
00001038:E0000594    MULFun:  MUL R0, R4, R5
0000103C:E1A0F00E                MOV PC, LR

.DATA
00001044:      A:      .WORD 0
```

StackView	
000053A0:81818181	
000053A4:81818181	
000053A8:81818181	
000053AC:81818181	
000053B0:81818181	
000053B4:81818181	
000053B8:81818181	
000053BC:81818181	
000053C0:81818181	
000053C4:81818181	
000053C8:81818181	
000053CC:81818181	
000053D0:81818181	
000053D4:81818181	
000053D8:81818181	
000053DC:81818181	
000053E0:81818181	
000053E4:81818181	
000053E8:81818181	
000053EC:81818181	
000053F0:81818181	
000053F4:0000000B	
000053F8:0000000A	
000053FC:00000002	
00005400:81818181	
00005404:81818181	
00005408:81818181	
0000540C:81818181	
00005410:81818181	

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

R0 :00000015
R1 :0000000b
R2 :0000000a
R3 :00000002
R4 :0000000b
R5 :0000000a
R6 :00000002
R7 :00000000
R8 :00000000
R9 :00000000
R10(sl):00000000
R11(fp):00000000
R12(ip):00000000
R13(sp):000053f4
R14(lr):0000103c
R15(pc):00001038

```
.TEXT
00001000:E59F4038  LDR R4, =A
00001004:E3A0100B  MOV R1, #11
00001008:E3A0200A  MOV R2, #10
0000100C:E3A03002  MOV R3, #02
00001010:E92D000E  STMFD R13!, {R1,R2,R3}
00001014:EB000001  BL ADDFun
00001018:E5840000  STR R0, [R4]
0000101C:EF000011  SWI 0x11
00001020:E8BD0070  ADDFun: LDMFD R13!, { R4, R5,R6}
00001024:E0840005          ADD R0, R4, R5
00001028:E92D4041  STMFD R13!, {R0,R6,LR}
0000102C:EB000001  BL MULFun
00001030:E8BD4030  LDMFD R13!, { R4, R5,LR}
00001034:E1A0F00E  MOV PC, LR
00001038:E0000594  MULFun: MUL R0, R4, R5
0000103C:E1A0F00E  MOV PC, LR
.DATA
00001044:      A: .WORD 0
```

StackView

000053A0:81818181
000053A4:81818181
000053A8:81818181
000053AC:81818181
000053B0:81818181
000053B4:81818181
000053B8:81818181
000053BC:81818181
000053C0:81818181
000053C4:81818181
000053C8:81818181
000053CC:81818181
000053D0:81818181
000053D4:81818181
000053D8:81818181
000053DC:81818181
000053E0:81818181
000053E4:81818181
000053E8:81818181
000053EC:81818181
000053F0:81818181
000053F4:00000015
000053F8:00000002
000053FC:00001018
00005400:81818181
00005404:81818181

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

General Purpose		Floating Point
Hexadecimal		
Unsigned Decimal		
Signed Decimal		
R0	: 0000006e	
R1	: 0000000b	
R2	: 0000000a	
R3	: 00000002	
R4	: 0000000b	
R5	: 0000000a	
R6	: 00000002	
R7	: 00000000	
R8	: 00000000	
R9	: 00000000	
R10 (s1)	: 00000000	
R11 (fp)	: 00000000	
R12 (ip)	: 00000000	
R13 (sp)	: 000053f4	
R14 (lr)	: 00001030	
R15 (pc)	: 0000103c	

CPSR Register		
Negative (N)	: 0	
Zero (Z)	: 0	

```
.TEXT
00001000:E59F4038    LDR R4, =A
00001004:E3A0100B    MOV R1, #11
00001008:E3A0200A    MOV R2, #10
0000100C:E3A03002    MOV R3, #02
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL  ADDFun
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDFun:  LDMFD R13!, { R4, R5,R6}
                                ADD R0, R4, R5
00001024:E0840005                                STMFD R13!, {R0,R6,LR}
00001028:E92D4041                                BL MULFun
0000102C:EB000001                                LDMFD R13!, { R4, R5,LR}
00001030:E8BD4030                                MOV PC, LR
00001034:E1A0F00E
00001038:E0000594    MULFun:  MUL R0, R4, R5
0000103C:E1A0F00E                                MOV PC, LR

.DATA
00001044:      A:      .WORD  0
```

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

Step Into

Close

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 0000006e

R1 : 0000000b

R2 : 0000000a

R3 : 00000002

R4 : 0000000b

R5 : 0000000a

R6 : 00000002

R7 : 00000000

R8 : 00000000

R9 : 00000000

R10 (s1) : 00000000

R11 (fp) : 00000000

R12 (ip) : 00000000

R13 (sp) : 000053f4

R14 (lr) : 00001030

R15 (pc) : 00001030

CPSR Register

Negative (N) : 0

Zero (Z) : 0

.TEXT

00001000:E59F4038 LDR R4, =A

00001004:E3A0100B MOV R1, #11

00001008:E3A0200A MOV R2, #10

0000100C:E3A03002 MOV R3, #02

00001010:E92D000E STMFD R13!, {R1,R2,R3}

00001014:EB000001 BL ADDFun

00001018:E5840000 STR R0, [R4]

0000101C:EF000011 SWI 0x11

00001020:E8BD0070 ADDFun: LDMFD R13!, { R4, R5,R6}

00001024:E0840005 ADD R0, R4, R5

00001028:E92D4041 STMFD R13!, {R0,R6,LR}

0000102C:EB000001 BL MULFun

00001030:E8BD4030 LDMFD R13!, { R4, R5,LR}

00001034:E1A0F00E MOV PC, LR

00001038:E0000594 MULFun: MUL R0, R4, R5

0000103C:E1A0F00E MOV PC, LR

.DATA

00001044: A: .WORD 0

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

General Purpose

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:0000006e
R1	:0000000b
R2	:0000000a
R3	:00000002
R4	:00000015
R5	:00000002
R6	:00000002
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00001018
R15 (pc)	:00001034

CPSR Register

Negative (N) :0

Zero (Z) :0

```
.TEXT
00001000:E59F4038    LDR R4, =A
00001004:E3A0100B    MOV R1, #11
00001008:E3A0200A    MOV R2, #10
0000100C:E3A03002    MOV R3, #02
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL  ADDFun
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDFun:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                ADD R0, R4, R5
00001028:E92D4041                STMFD R13!, {R0,R6,LR}
0000102C:EB000001                BL  MULFun
00001030:E8BD4030                LDMFD R13!, { R4, R5,LR}
00001034:E1A0F00E                MOV PC, LR
00001038:E0000594    MULFun:  MUL R0, R4, R5
0000103C:E1A0F00E                MOV PC, LR

.DATA
00001044:    A:      .WORD 0
```

Microprocessor & Computer Architecture (μpCA)

Nested Procedure Call: MUL(ADD(a,b),c)

General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0 : 0000006e	
R1 : 0000000b	
R2 : 0000000a	
R3 : 00000002	
R4 : 00000015	
R5 : 00000002	
R6 : 00000002	
R7 : 00000000	
R8 : 00000000	
R9 : 00000000	
R10 (s1) : 00000000	
R11 (fp) : 00000000	
R12 (ip) : 00000000	
R13 (sp) : 00005400	
R14 (lr) : 00001018	
R15 (pc) : 00001018	

CPSR Register	
Negative (N) : 0	
Zero (Z) : 0	

00001000:E59F4038	.TEXT
00001004:E3A0100B	LDR R4, =A
00001008:E3A0200A	MOV R1, #11
0000100C:E3A03002	MOV R2, #10
00001010:E92D000E	MOV R3, #02
00001014:EB000001	STMFD R13!, {R1,R2,R3}
00001018:E5840000	BL ADDFun
0000101C:EF000011	STR R0, [R4]
00001020:E8BD0070	SWI 0x11
00001024:E0840005	ADDFun: LDMFD R13!, { R4, R5,R6}
00001028:E92D4041	ADD R0, R4, R5
0000102C:EB000001	STMFD R13!, {R0,R6,LR}
00001030:E8BD4030	BL MULFun
00001034:E1A0F00E	LDMFD R13!, { R4, R5,LR}
00001038:E0000594	MOV PC, LR
0000103C:E1A0F00E	MULFun: MUL R0, R4, R5
	MOV PC, LR
	.DATA
00001044:	A: .WORD 0



THANK YOU



Team MPCA

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE20CS252

Software Interrupts

Session - 8.2

WHAT IS INTERRUPT/EXCEPTION?

- Main ()

- {

- :

- Doing something

Can happen anytime
Depends on types of interrupts

- (e.g.

- browsing)

- :

- } ring



Phone rings

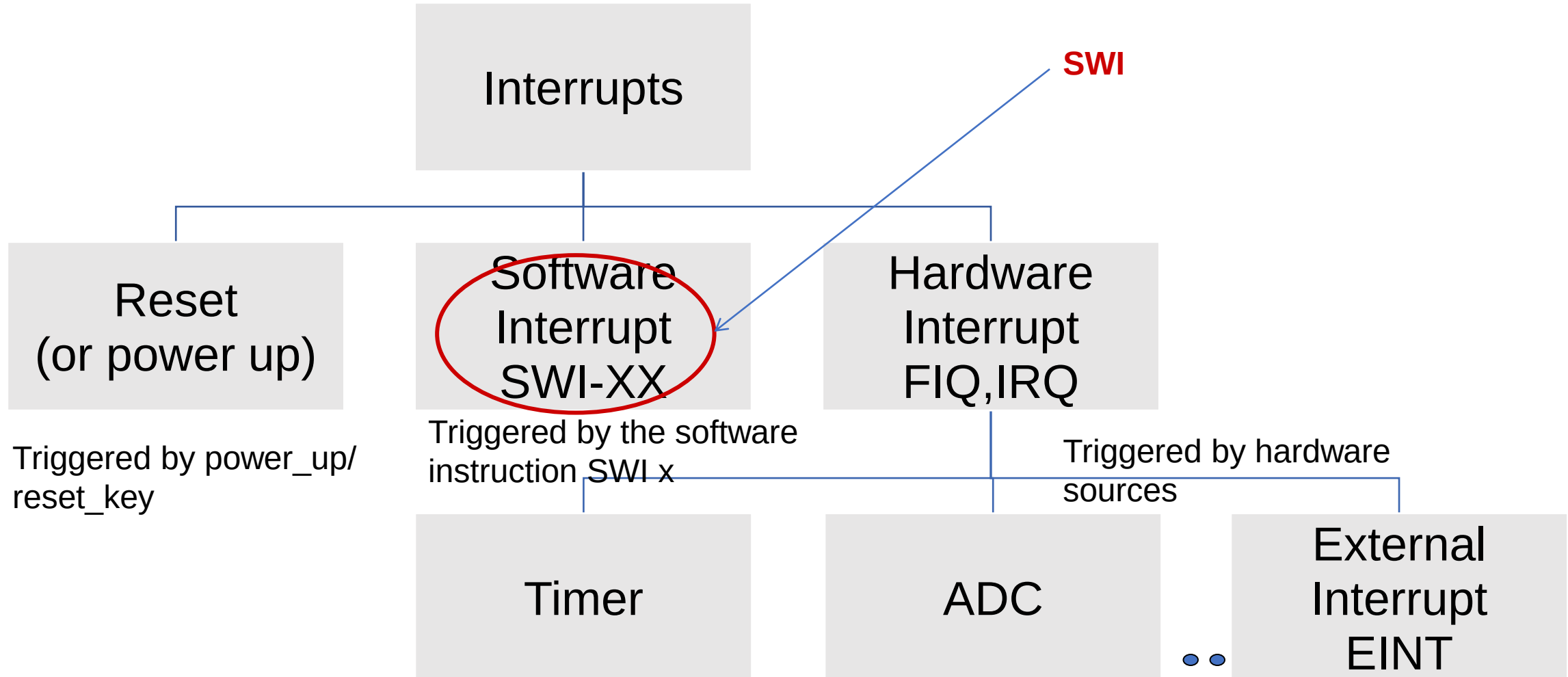


Phone rings

```
_isr() // Interrupt service routine
{
    some tasks (e.g. answer
                telephone)

} //when finished,
  //goes back to main
```



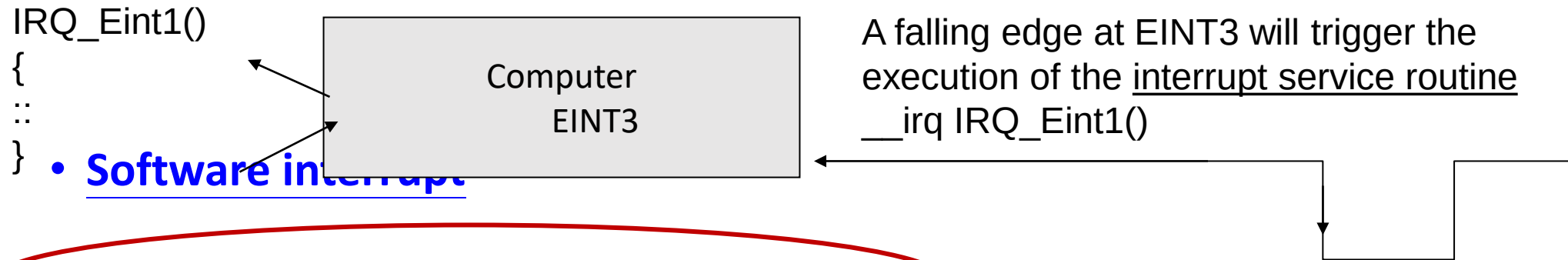


- The terms are used differently by various manufacturers
- Traditionally exception means
 - **The normal operation of a program is interrupted and the processor will execute another piece of software (exception handling) somewhere.**
 - Interrupt (hardware interrupt) is an exception caused by some hardware condition happening outside the processor (e.g. external hard interrupt, IRQ FIQ).
 - Software interrupt (SWI) is an exception caused by an assembly software instruction (SWI 0x?? exception call instruction) written in the software code.
 - Trap is an exception caused by a failure condition of the processor (e.g. abort “pre-fetch , data” , undefined instruction, divided_by_zero, or stack overflow etc)

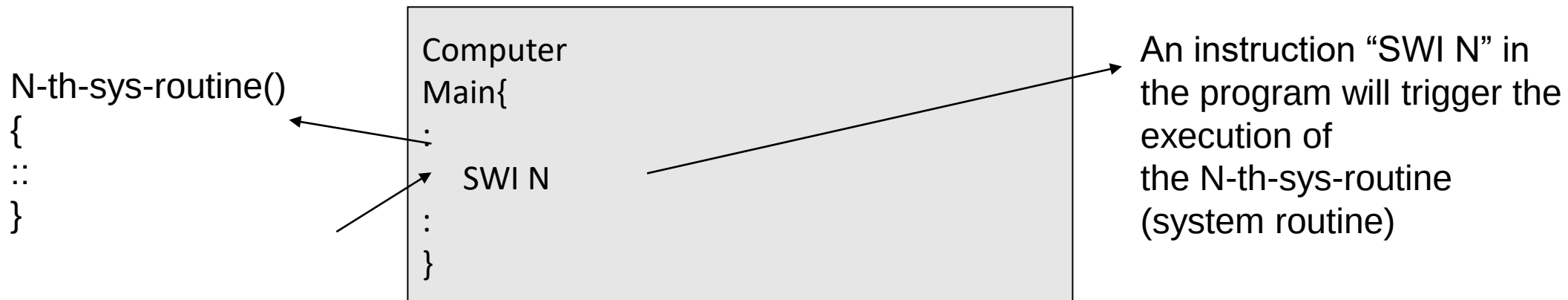
- Reset, a special interrupt to start the system– happens at power up , or reset button depressed)
- **Software interrupt SWI :**
**Similar to subroutine – happens when “SWI 0x??”
is written in the program**
- Hardware interrupt
 - FIQ (fast interrupt) or IRQ (external interrupt), when
 - the external interrupt request pin is pulled low, or
 - an analogue to digital conversion is completed, or
 - A timer/counter has made a regular request

COMPARE HARDWARE AND SOFTWARE INTERRUPT

- Hardware interrupt, e.g.



- Software interrupt



- ARM supports 7 types of exceptions and has a privileged processor mode for each type of exception.
- ARM Exception (interrupt) vectors

INTERRUPT VECTOR TABLE:

	Address	Exception	Mode in Entry
1	0x00000000	Reset	Supervisor
2	0x00000004	Undefined instruction	Undefined
3	0x00000008	Software Interrupt	Supervisor
4	0x0000000C	Abort (prefetch)	Abort
5	0x00000010	Abort (data)	Abort
x	0x00000014	Reserved	Reserved
6	0x00000018	IRQ (external interrupt)	IRQ
7	0x0000001C	FIQ (fast interrupt)	FIQ

SWI



- **SWI (software interrupt)**
 - Caused by “**SWI 0x??**” in your program
 - Arm completes the current instruction.
 - **Goto SWI** exception address **0x08** (short form for 0x 0000 0008)
 - {Change to supervisor op. mode :CPSR (bit0-4)
 - :
 - :
 - Return from interrupt
 - **MOVS pc, lr**
 - return to main}

- Whenever a program requires **input or output**, for instance to send some **text to the display**, it is normal to **call** a supervisor routine.
 - **To read data from the keyboard or to display the contents on the console**
- The supervisor is a program which operates at a privileged level, which means that it can do things that a user-level program cannot do directly.
- The limitations on the capabilities of a user-level program vary from system to system, but in many systems the user cannot access hardware facilities directly.
- **The supervisor** provides trusted ways to access system resources which appear to the user-level program rather like **special subroutine** accesses.

- The instruction set includes a special instruction, **SWI**, to call these functions, (SWI stands for '**Software Interrupt**', but is usually pronounced '**Supervisor Call**').
- **Note:** In **privileged modes** there is another register for each mode called **Saved Processor Status Register (SPSR)**.

The SPSR is used to **save the current CPSR** before changing modes.

- SWI interrupt occurs when the SWI instruction has been fetched and decoded successfully, and none of the other higher priority exceptions/interrupts have been flagged.
- Upon entry to the handler the CPSR will be set to SVC mode.
- **Note:** If a **SWI calls another SWI** (which is a common occurrence), then to avoid corruption, **the link register (LR & SPSR)** must be **stacked away** before branching to the nested SWI.

Example 1 :

A procedure to compute the statement in high level language using ARM ALP.

if (R0==R1) R2++;

MOV R0, #10

MOV R1, #10

BL GREAT

SWI 0x11 ; terminate the program / logical end.

GREAT: CMP R0, R1
ADDEQ R2, R2, #1
MOV PC, LR

Example 2 :

A program to display a string on the screen using ARM ALP.

System.out.print (“ Hello World”);

LDR R1, =A

LOOP: LDRB R0, [R1], #1

CMP R0, #0

SWINE 0x00 ; display a character on the screen.

BNE LOOP

SWI 0x11 ; terminate the program.

.DATA

A: .ASCIZ “HELLO WORLD”

Example 3 :

A procedure to display a string on the screen using ARM ALP.

```
// System.out.print (" Hello World");
```

```
LDR    R1, =A
```

```
BL  strprint      ; call display routine
```

```
SWI 0x11          ; terminate the program.
```

```
strprint: LDRB R0, [R1], #1
```

```
CMP R0, #0
```

```
SWINE 0x00        ; display a character on the screen.
```

```
BNE strprint
```

```
MOV PC, LR
```

```
.DATA
```

```
A: .ASCIZ "HELLO WORLD"
```

Example 3 :

A procedure to display a string on the screen using ARM ALP using routine 0x02.

```
LDR    R0, =A
SWI     0x02      ; display a string on the screen
SWI     0x11
```

.DATA

A: .ASCIZ "HELLO WORLD"



THANK YOU



Team MPCA

Department of Computer Science and Engineering



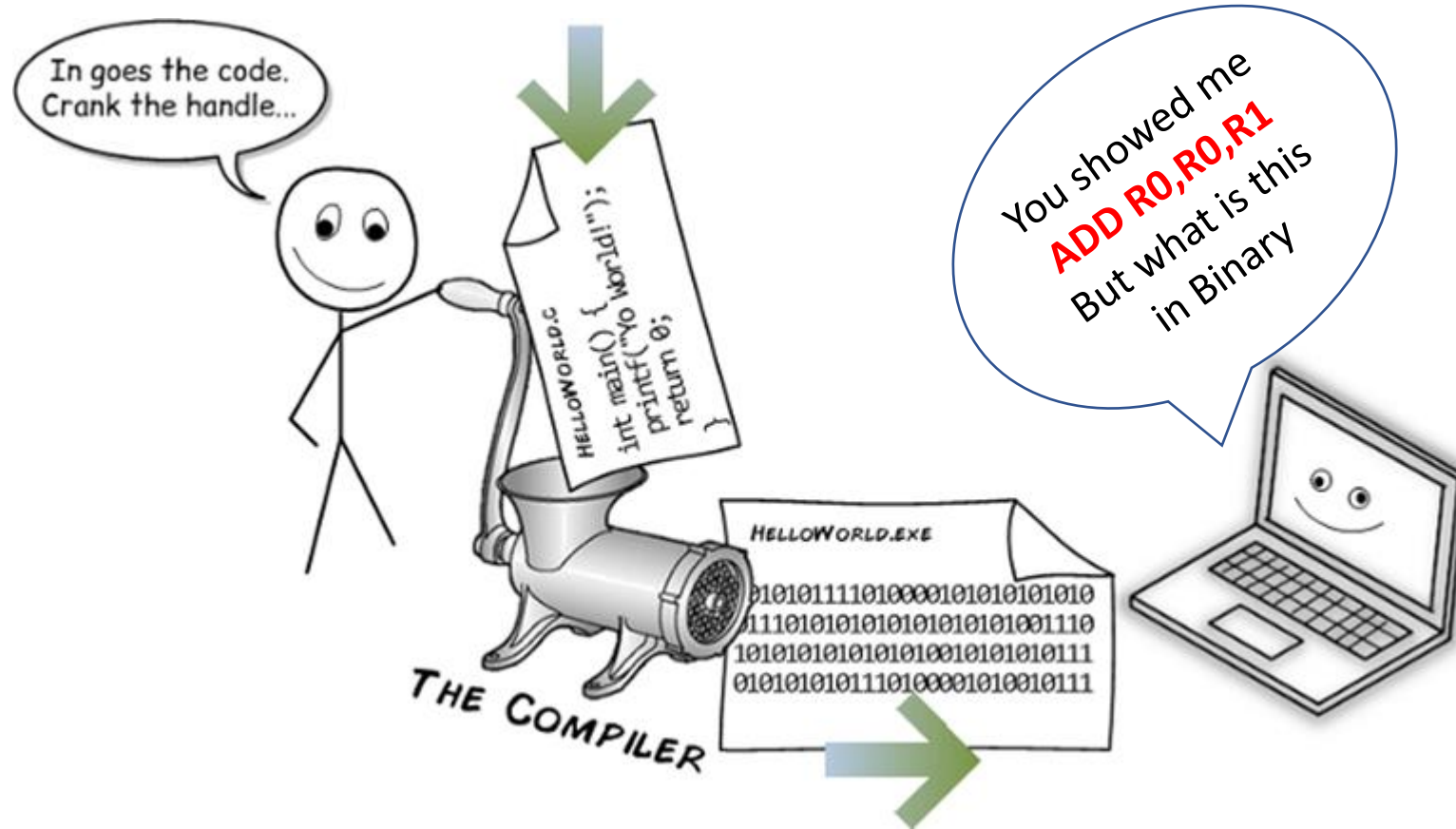
Microprocessor & Computer Architecture (μ pCA)

Instruction Encoding

Session - 9

Microprocessor & Computer Architecture (μpCA)

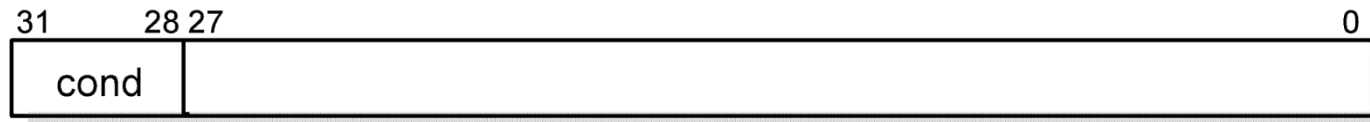
Instruction Encoding



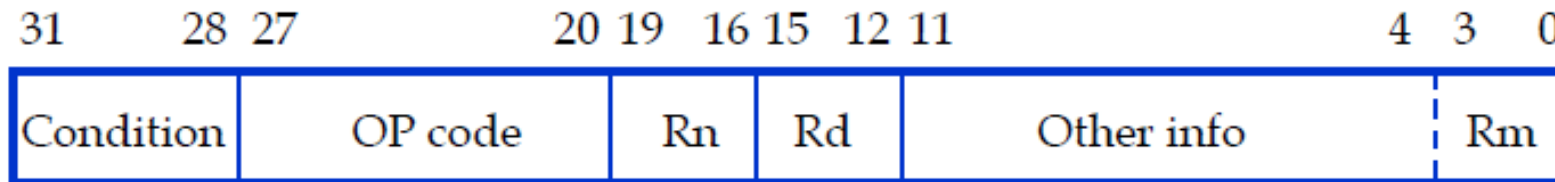
Microprocessor & Computer Architecture (μpCA)

What is an ARM Instruction?

OPcode{condition}{S} Rd,Operand1,Operand2



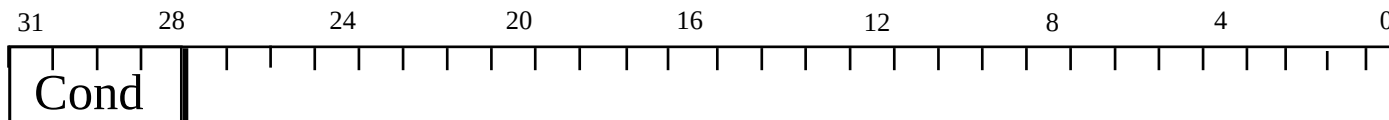
Example: Data Processing Instruction



An instruction specifies a **Conditional Execution Code** (Condition), the **OP Code**, **Two or Three Registers** (Rn, Rd, and Rm), and some other information

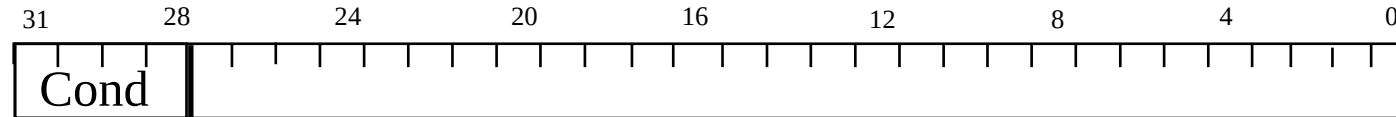
- Fifteen different conditions may be used: these are listed in **Condition code summary**.
- The **sixteenth (1111) is reserved**, and must not be used.
- In the **absence** of a suffix, the condition field of most instructions is set to "**Always**" (suffix **AL**). This means the instruction will always be executed regardless of the CPSR condition codes.

The ARM condition code field



Microprocessor & Computer Architecture (μpCA)

The Condition Field



0000 = EQ - Z set (equal)

0001 = NE - Z clear (not equal)

0010 = HS / CS - C set (unsigned higher or same)

0011 = LO / CC - C clear (unsigned lower)

0100 = MI - N set (negative)

0101 = PL - N clear (positive or zero)

0110 = VS - V set (overflow)

0111 = VC - V clear (no overflow)

1000 = HI - C set and Z clear (unsigned higher)

1001 = LS - C clear or Z (set unsigned lower or same)

1010 = GE - N set and V set, or N clear and V clear (>or =)

1011 = LT - N set and V clear, or N clear and V set (>)

1100 = GT - Z clear, and either N set and V set, or N clear and V set (>)

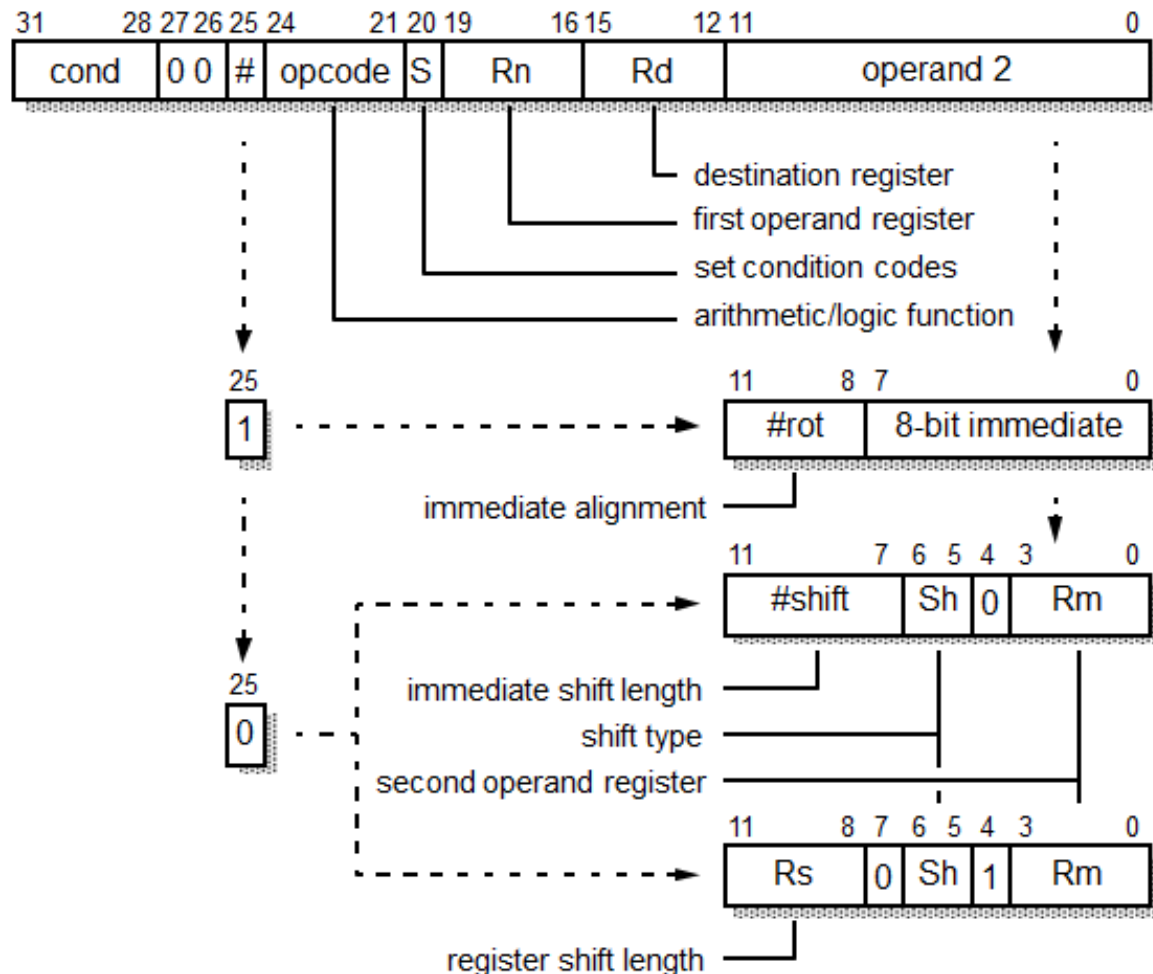
1101 = LE - Z set, or N set and V clear, or N clear and V set (<, or =)

1110 = AL - always

1111 = NV - reserved.

Microprocessor & Computer Architecture (μpCA)

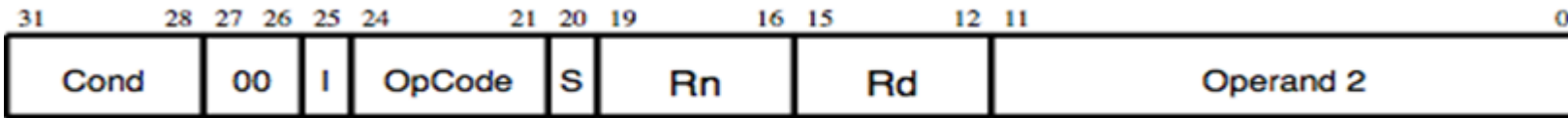
Data Processing Instruction



Opcode [24:21]	Mnemonic
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

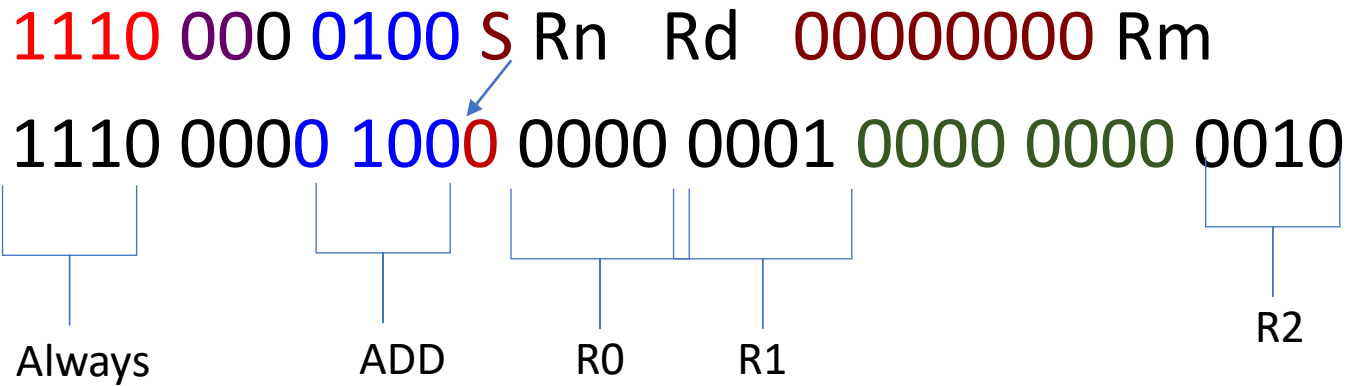
Microprocessor & Computer Architecture (μpCA)

Data Processing Instruction: Example-1



■ Ex:

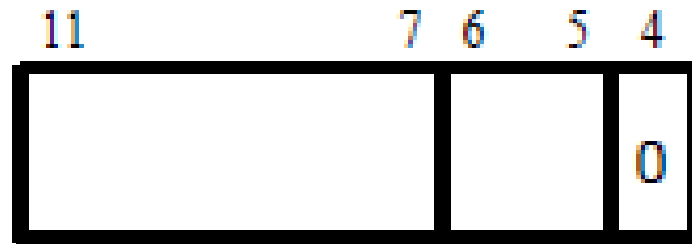
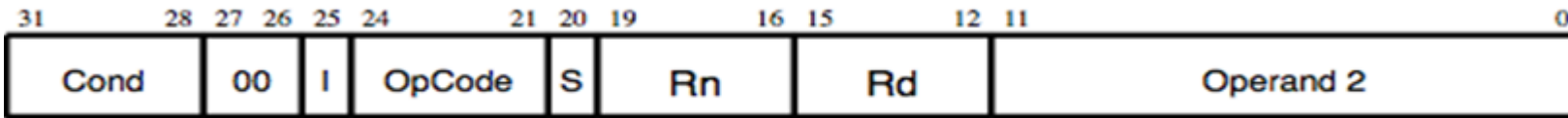
ADD R1, R0, R2



E0801002

Microprocessor & Computer Architecture (μpCA)

Data Processing Instruction



Shift type

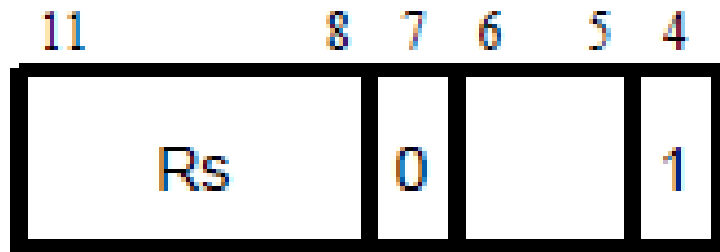
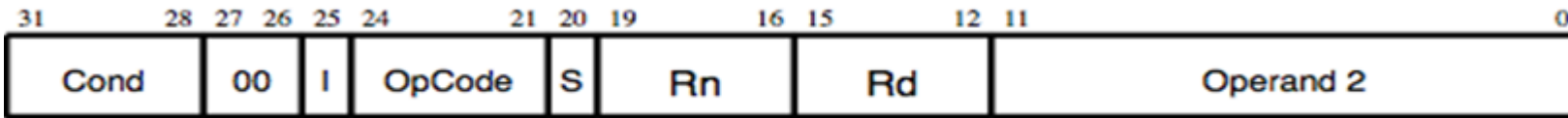
- 00 = logical left
- 01 = logical right
- 10 = arithmetic right
- 11 = rotate right

Shift amount

5 bit unsigned integer

Microprocessor & Computer Architecture (μpCA)

Data Processing Instruction



Shift type

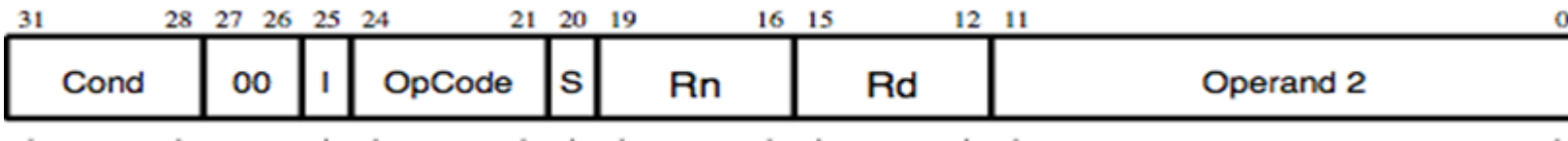
- 00 = logical left
- 01 = logical right
- 10 = arithmetic right
- 11 = rotate right

Shift register

Shift amount specified in
bottom byte of Rs

Microprocessor & Computer Architecture (μpCA)

Data Processing Instrucion: Example-2



■Ex:

ADDS R1, R0, R2, LSR #2

1110 000 0100 S Rn Rd Shift Rm

1110 000 0100 S Rn Rd 00010 01 0 Rm

1110 0000 1001 0000 0001 0001 0010 0010

└──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘

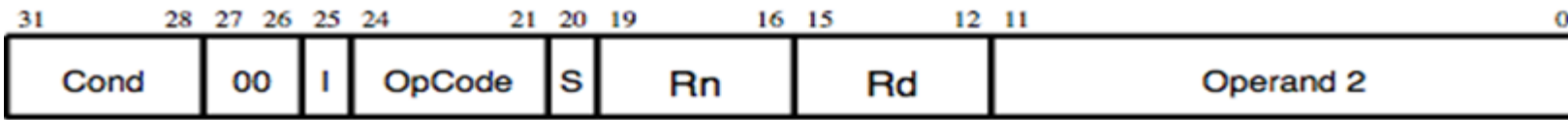
ADDS R0 R1 #2 LSR R2

E0901122

Shift is specified as immediate

Microprocessor & Computer Architecture (μpCA)

Data Processing Instruction: Example-3



■ Ex:

ADDS R1, R0, R2, LSR R4

1110 000 0100 S Rn Rd Shift Rm

1110 000 0100 S Rn Rd 0100 0 01 1 Rm

1110 0000 1001 0000 0001 0100 0011 0010

└──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘ └──┬──┘

ADDS R0 R1 R4 LSR R2

PES
UNIVERSITY
ONLINE

[illegible]

Microprocessor & Computer Architecture (μpCA)

Cond	0	0	1	Opcode				S	Rn			Rd			Operand 2										
Cond	0	0	0	0	0	0	A	S	Rd			Rn			Rs			1	0	0	1	Rm			
Cond	0	0	0	0	1	U	A	S	RdHi			RdLo			Rn			1	0	0	1	Rm			
Cond	0	0	0	1	0	B	0	0	Rn			Rd			0	0	0	0	1	0	0	1	Rm		
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn		
Cond	0	0	0	P	U	0	W	L	Rn			Rd			0	0	0	0	1	S	H	1	Rm		
Cond	0	0	0	P	U	1	W	L	Rn			Rd			Offset			1	S	H	1	Offset			
Cond	0	1	1	P	U	B	W	L	Rn			Rd			Offset										
Cond	0	1	1																		1				
Cond	1	0	0	P	U	S	W	L	Rn			Register List													
Cond	1	0	1	L	Offset																				

Data Processing / PSR Transfer

Multiply

Multiply Long

Single Data Swap

Branch and Exchange

Halfword Data Transfer: register offset

Halfword Data Transfer: immediate offset

Single Data Transfer

Undefined

Block Data Transfer

Branch



THANK YOU



Team MPCA

Department of Computer Science and Engineering



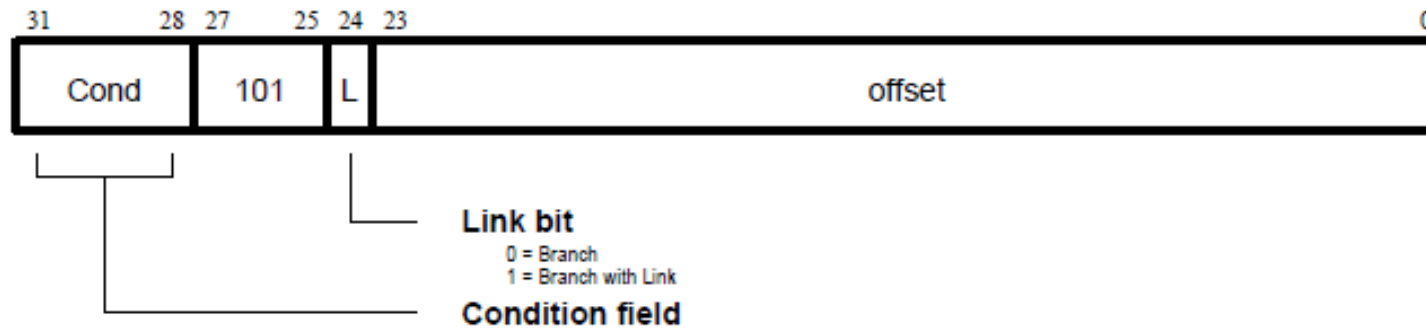
Microprocessor & Computer Architecture (μ pCA)

Instruction Encoding

Session - 10

Microprocessor & Computer Architecture (μpCA)

BRANCH INSTRUCTIONS

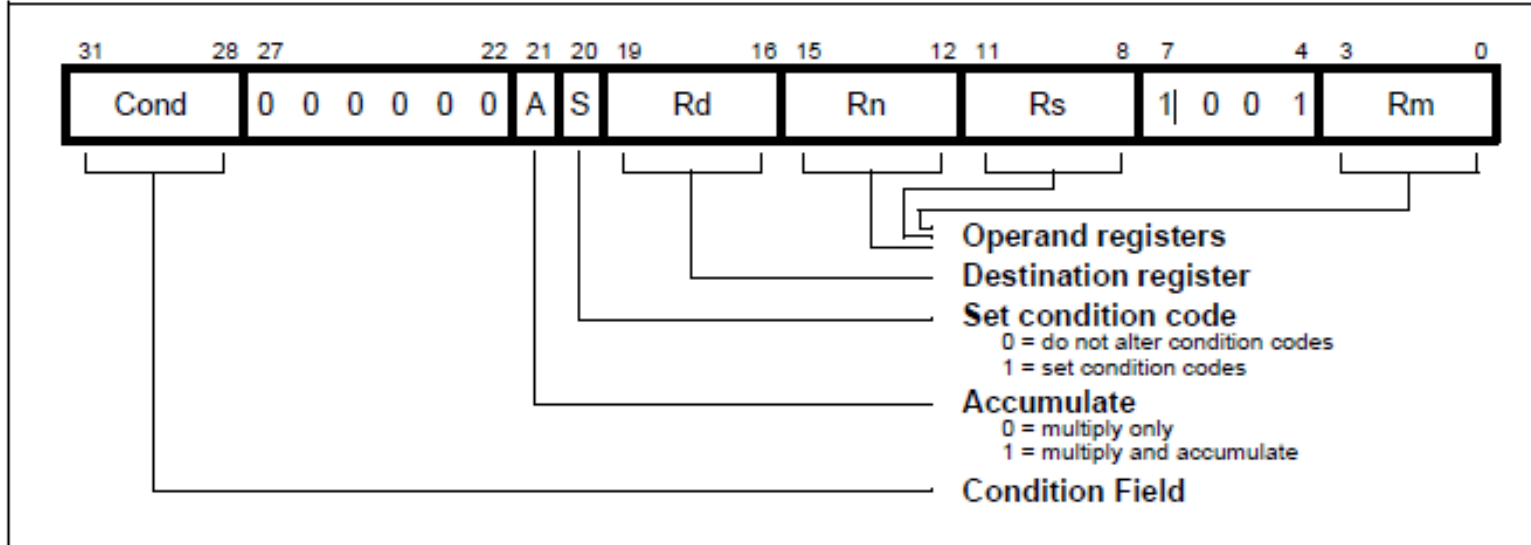


Ex 1: B LOOP

Ex 2: BL SUBROUTINE

Microprocessor & Computer Architecture (μpCA)

MULTIPLICATION INSTRUCTIONS

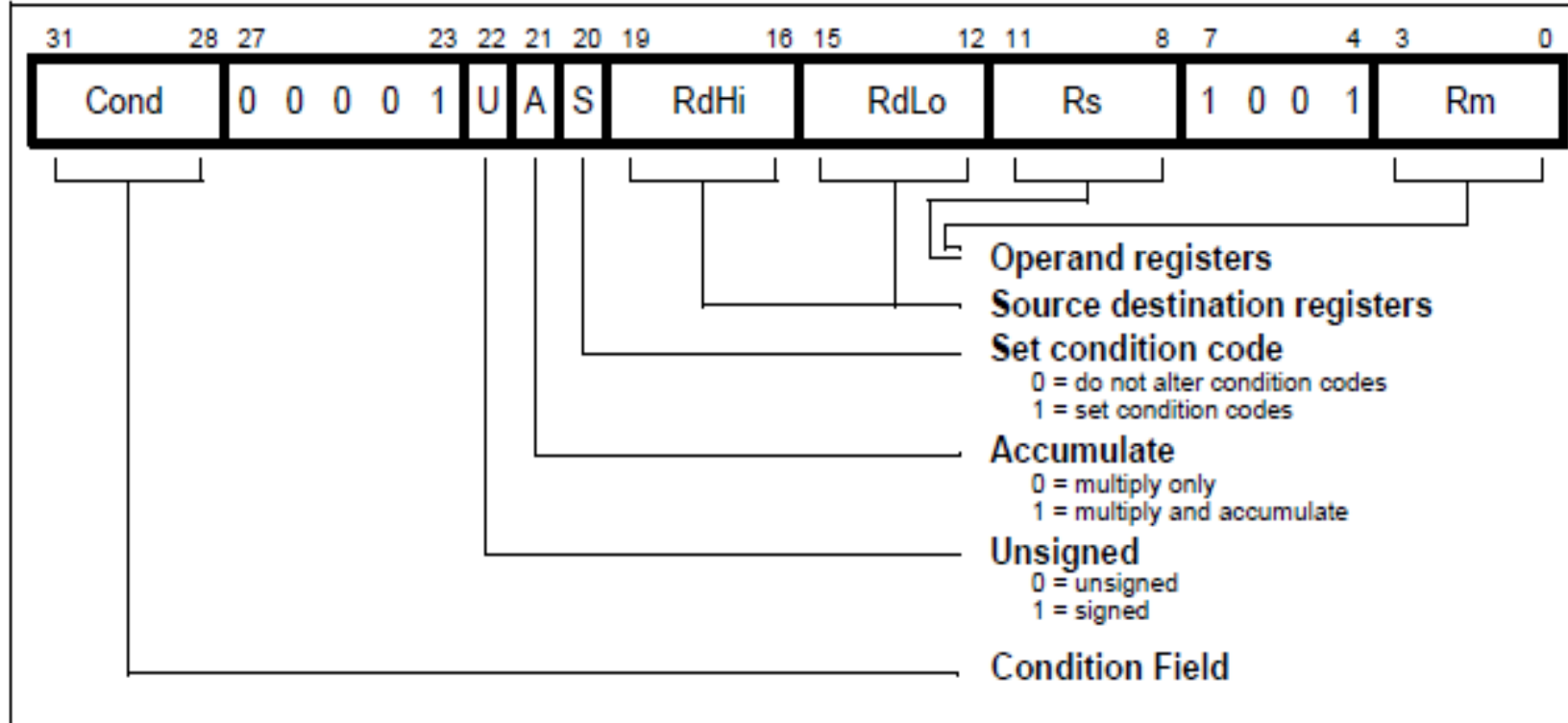


Ex 1: MUL R0, R1, R2

Ex 2: MLA R0, R1, R2, R3

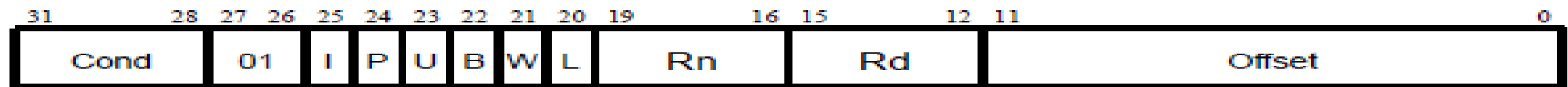
Microprocessor & Computer Architecture (μpCA)

MULTIPLICATION INSTRUCTIONS



Ex 3: **SMULL R0, R1, R2, R3**

Ex 4: **UMLAL R0, R1, R2, R3**



Data Transfer Instruction

Source/Destination register

Base register

Load/Store bit

0 – Store to memory
1 – Load from memory

Write-back bit

0 – no write-back
1 – write address into base

Byte/Word bit

0 – transfer word quantity
1 – transfer byte quantity

Up/Down bit

0 – down; subtract offset from base
1 – up; add offset to base

Pre/Post indexing bit

0 – post; add offset after transfer
1 – pre; add offset before transfer

Immediate offset

11 0 – offset is an Immediate value



Unsigned 12 bit immediate offset



shift applied to Rm

Offset register

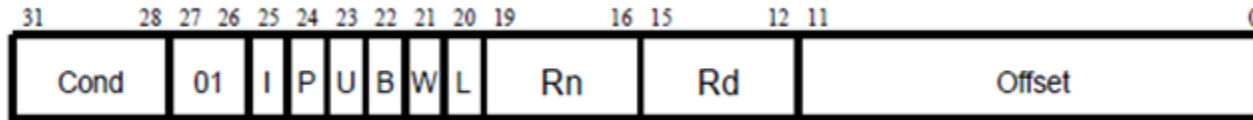
Condition field

Microprocessor & Computer Architecture (μpCA)

Data Transfer Instruction: STR



■ Ex:



STR R0, [R1]

1110 01 0 1 1 0 0 0 0001 0000 0000 0000 0000

1110 01 I P U B W L 0001 0000 0000 0000 0000

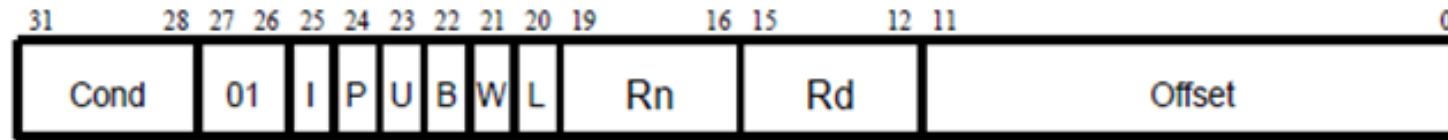
Binary Code: E5810000

Note: Default pre-indexing with no write back when offset is not specified

Microprocessor & Computer Architecture (μpCA)

Data Transfer Instruction: LDR

■ Ex:



LDR R0, [R1], R2

1110 01 I P U B W L 0001 0000 0000 0000 0010

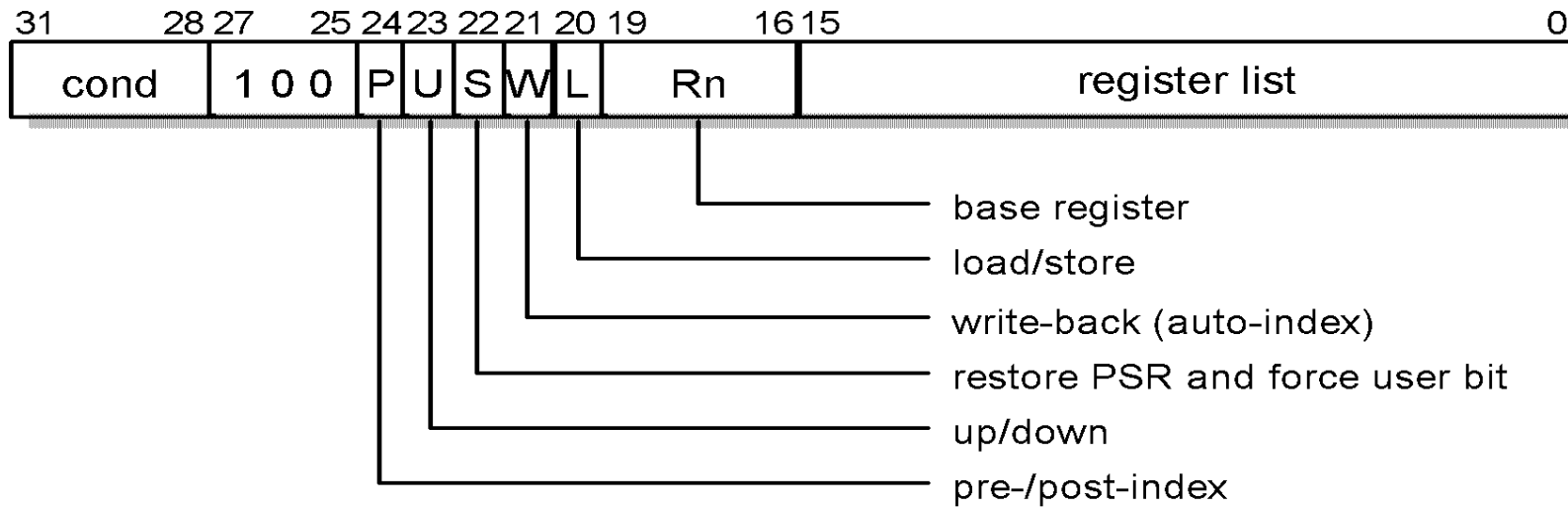
1110 01 1 0 1 0 0 1 0001 0000 0000 0000 0010

E6910002

Note: In the case of post-indexed addressing, the write back bit is redundant and must be set to zero

Microprocessor & Computer Architecture (μpCA)

Block Transfer Instruction



Ex 1: LDMIA R13! , { R0, R5 - R8, R11}

Ex 2: STMIA R13! , { R8, R4- R6, R12}

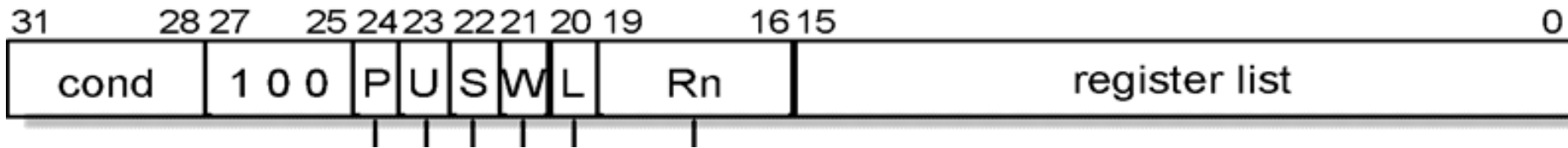
Microprocessor & Computer Architecture (μpCA)

Block Transfer Instruction: Addressing Mode

Name	Stack	Other	L bit	P bit	U bit
pre-increment load	LDMED	LDMIB	1	1	1
post-increment load	LDMFD	LDMIA	1	0	1
pre-decrement load	LDMEA	LDMDB	1	1	0
post-decrement load	LDMFA	LDMDA	1	0	0
pre-increment store	STMFA	STMIB	0	1	1
post-increment store	STMEA	STMIA	0	0	1
pre-decrement store	STMFD	STMDB	0	1	0
post-decrement store	STMED	STMDA	0	0	0

Microprocessor & Computer Architecture (μpCA)

Block Transfer Instruction: LDM



LDMIA R13! , { R0, R5 - R8, R11}

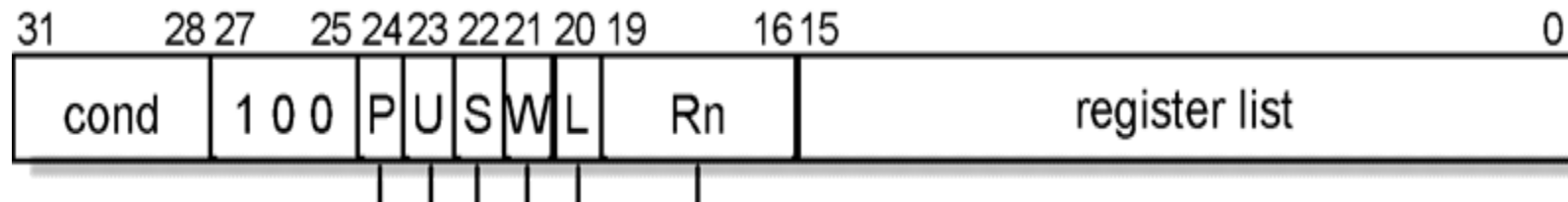
1110 100 **P** **U** **S** **W** **L** 1101 **0000** **R11** **00****R8****R7****R6****R5** **0000****R0**

1110 100 **0** **1** **0** **1** **1** 1101 **0000** **1** **00****1** **1** **1** **1** **0000****1**

E8BD09E1

Microprocessor & Computer Architecture (μpCA)

Block Transfer Instruction: STM



STMIB R13! , { R8, R4- R6, R12}

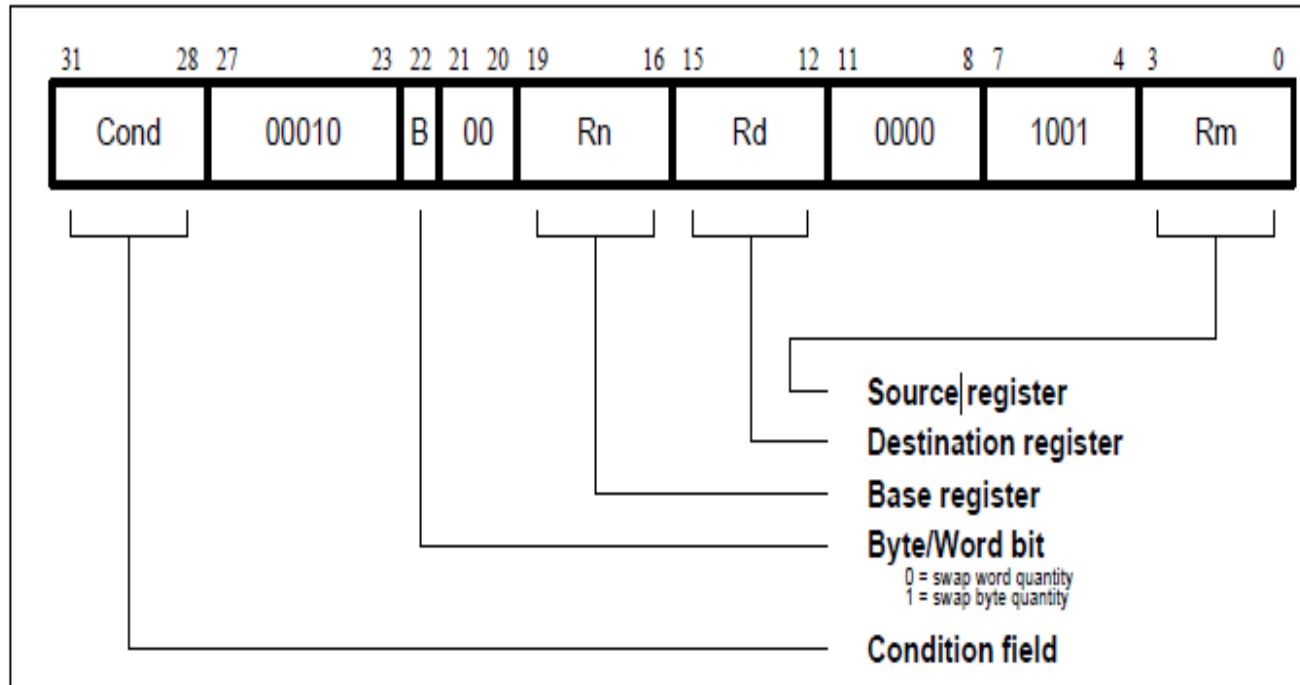
1110 100 **1** **10** **10** 1101 000**10** 00**10111**0000

1110 100 **P** **U** **S** **W** **L** 1101 000**R12**0 00**R8** 0**R6R5R4** 0000

E9AD1170

Microprocessor & Computer Architecture (μpCA)

SWAP





THANK YOU



Team MPCA

Department of Computer Science and Engineering